



Урок 1. Введение в SQL

1. Базы данных (БД) и СУБД
2. Язык управления запросами SQL
3. Работа в Redash
4. Резюме

Урок 1. Введение в SQL

Базы данных (БД) и СУБД

С использованием данных мы сталкиваемся каждый день: при заказе еды, в библиотеке, при использовании банковского приложения. В сфере IT с базами данных прежде всего приходится работать дата-инженерам, аналитикам, дата-сайентистам, бэкэнд-разработчикам, а иногда даже менеджерам.

В курсе мы рассматриваем работу с реляционными базами данных. Слово «*реляционный*» происходит от английского “*relation*” и означает отношение, логическую связь между таблицами, которые друг с другом связаны.

В основе любых реляционных баз данных лежат таблицы. Каждая таблица состоит из полей (столбцов) и записей (строк), на пересечении которых находятся значения (значение поля для записи). Применительно к полям существует два понятия:

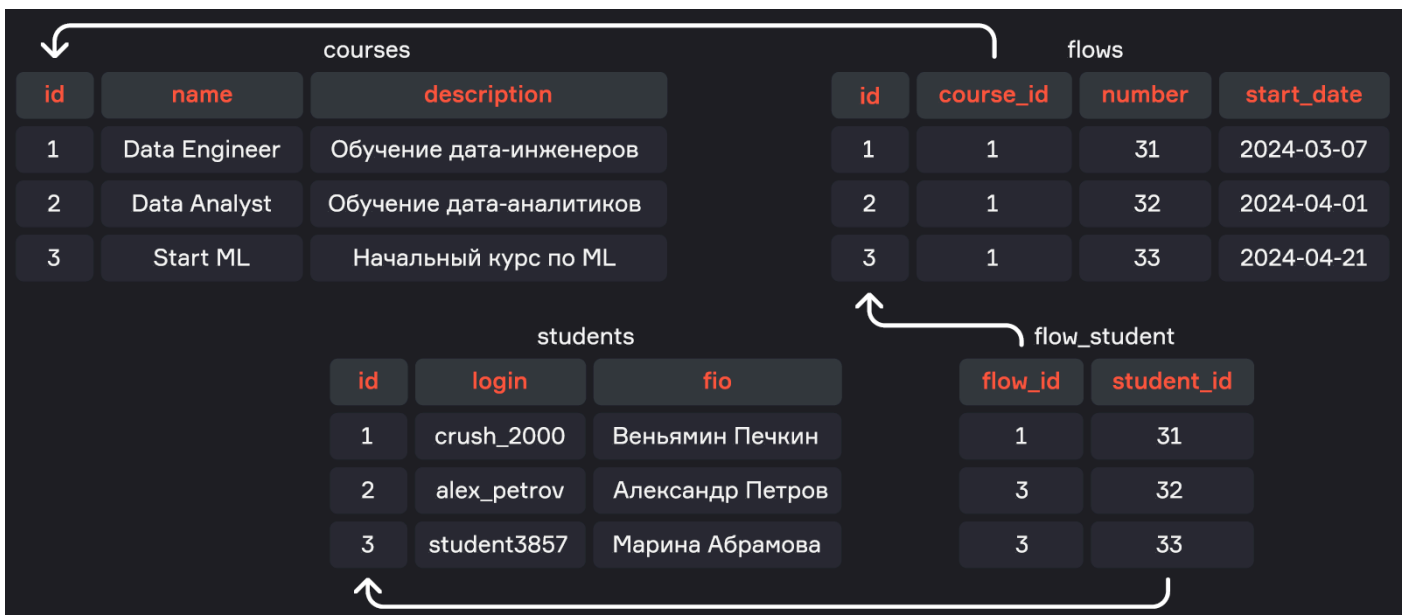
1. **Первичный ключ (primary key)**. Он нужен для того, чтобы уникально идентифицировать какую-либо строку в таблице. Для всех записей в одной таблице первичным ключом является одно или несколько полей данной таблицы.

Первичным ключом могут быть:

- реальные уникальные идентификаторы пользователя в рамках одной таблицы. Например, номер паспорта, номер заказа, email, СНИЛС;
- синтетические значения идентификаторов. Такие значения не имеют смысла в физическом мире и используются только для различения строк в таблице.

2. **Внешний ключ (foreign key)**. Это ссылка из одной таблицы на первичный ключ другой таблицы. Например, у нас есть ID заказа в одной таблице, а в другой — набор заказанных товаров. У каждого из заказанных товаров будет стоять ID заказа, к которому он относится.

ПРИМЕР



Рассмотрим набор таблиц, содержащих информацию о студентах и курсах, на которых они обучаются:

- таблица **courses** имеет первичный ключ — уникальный синтетический **id** ;
- таблица **flows** (потоки обучения) имеет первичный ключ — свой **id** , не зависящий от идентификаторов других таблиц. А также внешний ключ **course_id** , который ссылается на таблицу **courses** . В примере на схеме все потоки имеют **course_id** = 1 и относятся к курсу Data Engineer из таблицы **courses** ;
- таблица **students** имеет первичный ключ — **id** ;
- таблица **flow_student** имеет два внешних ключа — **flow_id** и **student_id** .

Выше мы рассматривали связь **один ко многим**. Одному первичному ключу (например ID) может соответствовать несколько внешних ключей в соседней таблице. Один ID-шник — много ссылок на него, поэтому связь называется «один ко многим».

courses			flows			
id	name	description	id	course_id	number	start_date
1	Data Engineer	Обучение дата-инженеров	1	1	31	2024-03-07
2	Data Analyst	Обучение дата-аналитиков	2	1	32	2024-04-01
3	Start ML	Начальный курс по ML	3	1	33	2024-04-21

Также у нас в базе студентов есть и связь, которая называется **многие ко многим**. Это поля с потоками и студентами, ведь каждый из этих студентов может учиться на нескольких потоках, а в каждый поток может зачисляться несколько студентов. Такая связь реализуется с помощью дополнительной вспомогательной таблицы `flow_student`. Это таблица, в которой один внешний ключ — это ссылка на таблицу Flows, а второй внешний ключ — это ссылка на таблицу `students`.

flow_student	
flow_id	student_id
1	31
3	32
3	33

Работой базы данных управляет система управления базами данных (СУБД). Это программное обеспечение, которое берет на себя ответственность за консистентность, оптимизацию, репликацию, блокировки, доступ и прочие функции. Систем управления базами данных очень много, и их работа немного отличается. Наиболее распространенными СУБД являются Postgres, MySQL, Greenplum, Vertica, Oracle, MS SQL, Clickhouse, Redis, MongoDB. С некоторыми из них мы познакомимся в следующих модулях курса.



Язык управления запросами SQL

Мы можем получать данные из базы данных с помощью запросов к СУБД, написанных на языке управления запросами SQL (Structured Query Language). Это *декларативный унифицированный язык*. Определение «декларативный» значит, что в запросе мы описываем результат выполнения запроса, а не логику извлечения данных системой. А благодаря унифицированности языка, его основные конструкции можно использовать для работы с разными базами данных. Тем не менее, стоит помнить, что для разных СУБД язык может немного отличаться синтаксически или за счет введения дополнительных конструкций.

Итак, мы отправляем в СУБД SQL-запрос, система управления оптимизирует этот запрос, смотрит, насколько он правильный, есть ли у нас доступ к этим таблицам, есть ли вообще такие таблицы и поля, а затем обращается к базе данных за нужными строками, которые мы хотим получить. Это строки с нужной нам информацией возвращаются, например, в интерфейсе приложения, или в командной строке, или в дашборде.

SQL-запрос представляет собой конструкцию из операторов, которые также называют инструкциями или ключевыми словами. Благодаря им СУБД «понимает», какие действия с данными ей необходимо произвести.

Рассмотрим простой запрос `SELECT * FROM courses;`

- `SELECT ... FROM ...` — основная конструкция в SQL, которая даёт команду базе данных **ВЫБЕРИ** <колонки> **ИЗ** <таблица>
- - используется после `SELECT` и сообщает, что нужно выбрать все столбцы из указанной после `FROM` таблицы

Таким образом, в результате запроса мы получим все строки из таблицы `courses`.

id	name	description
1	Data Engineer	Обучение дата-инженеров
2	Data Analyst	Обучение дата-аналитиков
3	Start ML	Начальный курс по ML

Запрос `SELECT COUNT(id) FROM students;` считает количество строк в таблице `students`.

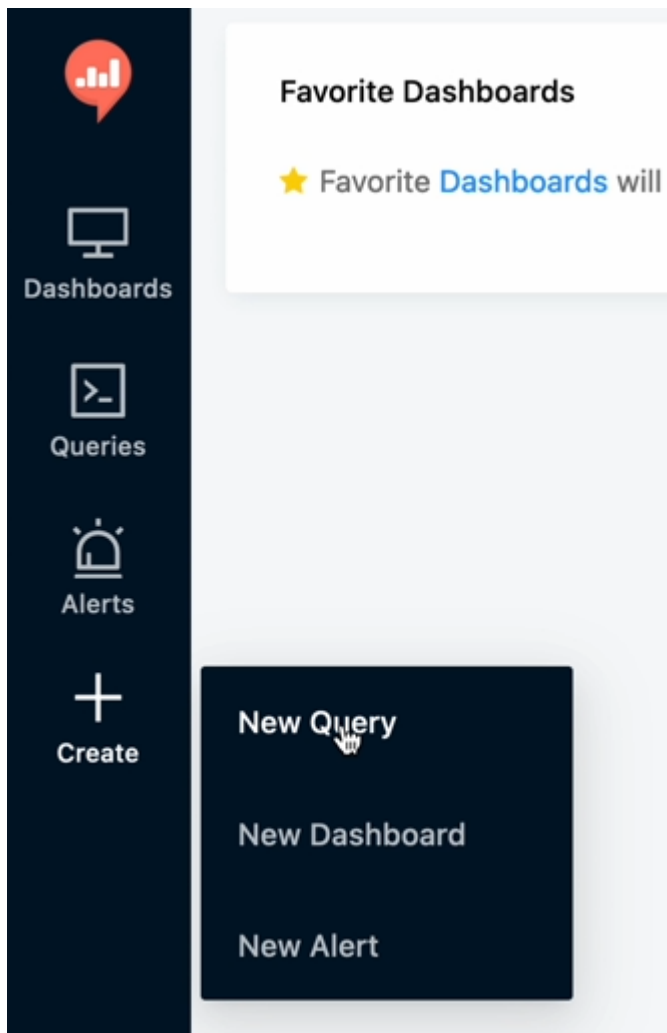
Рассмотренные выше запросы относятся к категории DQL (Data Query Language), которая только забирает данные из таблицы. Сам SQL — это такое общее название для семейства подязыков, отличающихся операторами и назначением запросов. Полный перечень подязыков, на которые делят SQL следующий:

- **DQL (Data Query Language)** — язык, который используется для выполнения запросов к данным, поиска и извлечения данных из базы;
- **DML (Data Manipulation Language)** — язык, с помощью которого добавляются, изменяются или удаляются строки;
- **DDL (Data Definition Language)** — язык, с помощью которого создаются или изменяются какие-то сущности, например, таблицы;
- **DCL (Data Control Language)** — язык, который позволяет управлять доступами к данным;
- **TCL (Transaction Control Language)** — язык управления транзакциями, который используется для контроля обработки транзакций в базе данных.

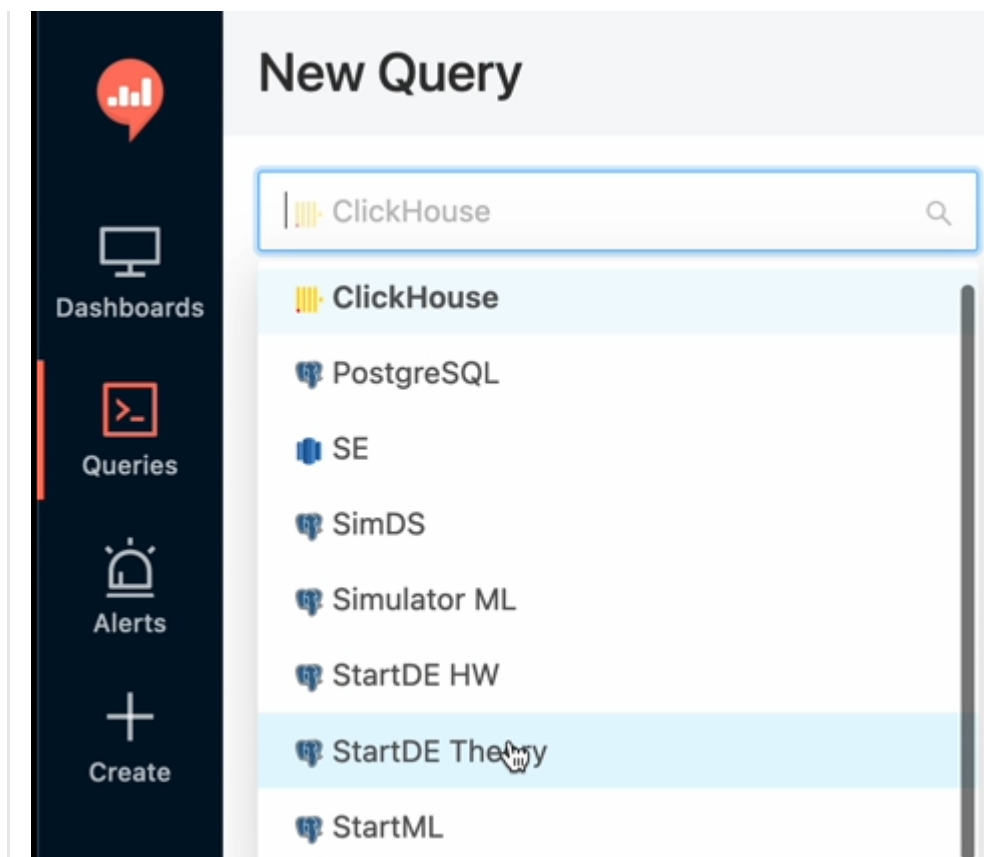
Работа в Redash

Ниже представлена краткая инструкция работы с инструментом Redash, который является сервисом для работы с базами данных и предоставляет возможность не только написать запросы, но и визуализировать результат в дашбордах.

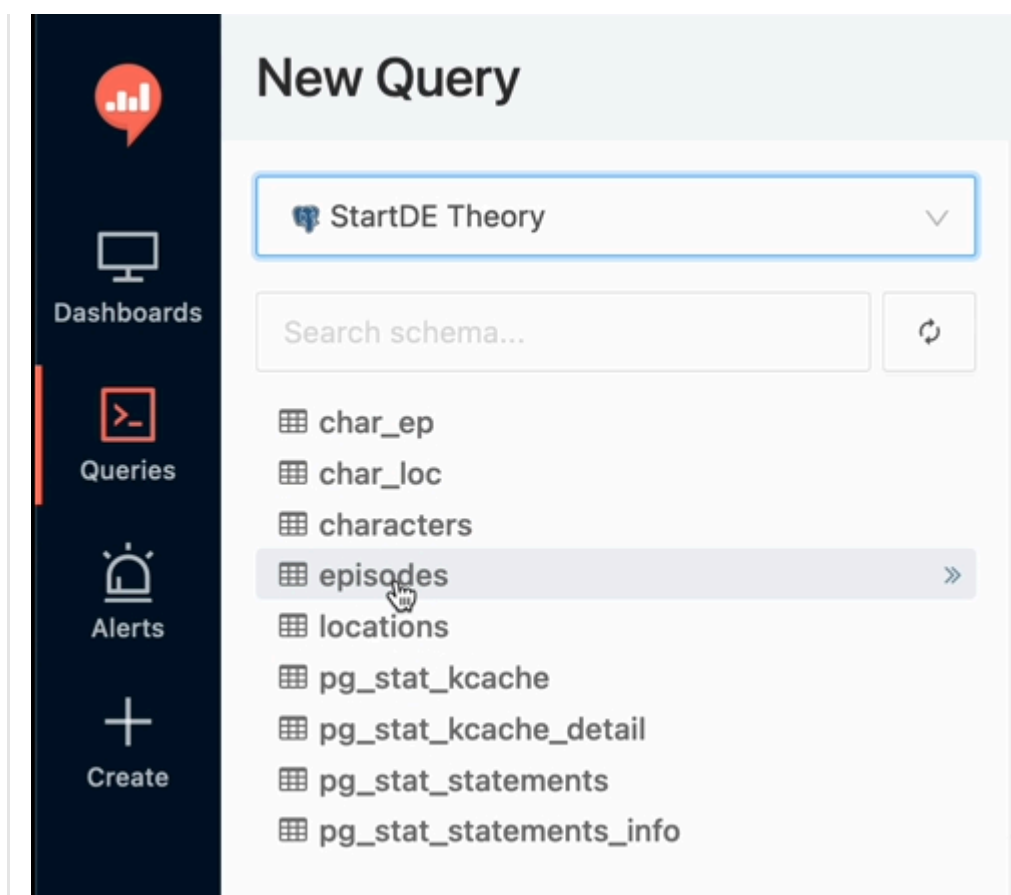
1. Для того чтобы начать писать запрос, необходимо нажать на кнопку **Create** и выбрать пункт New Query.



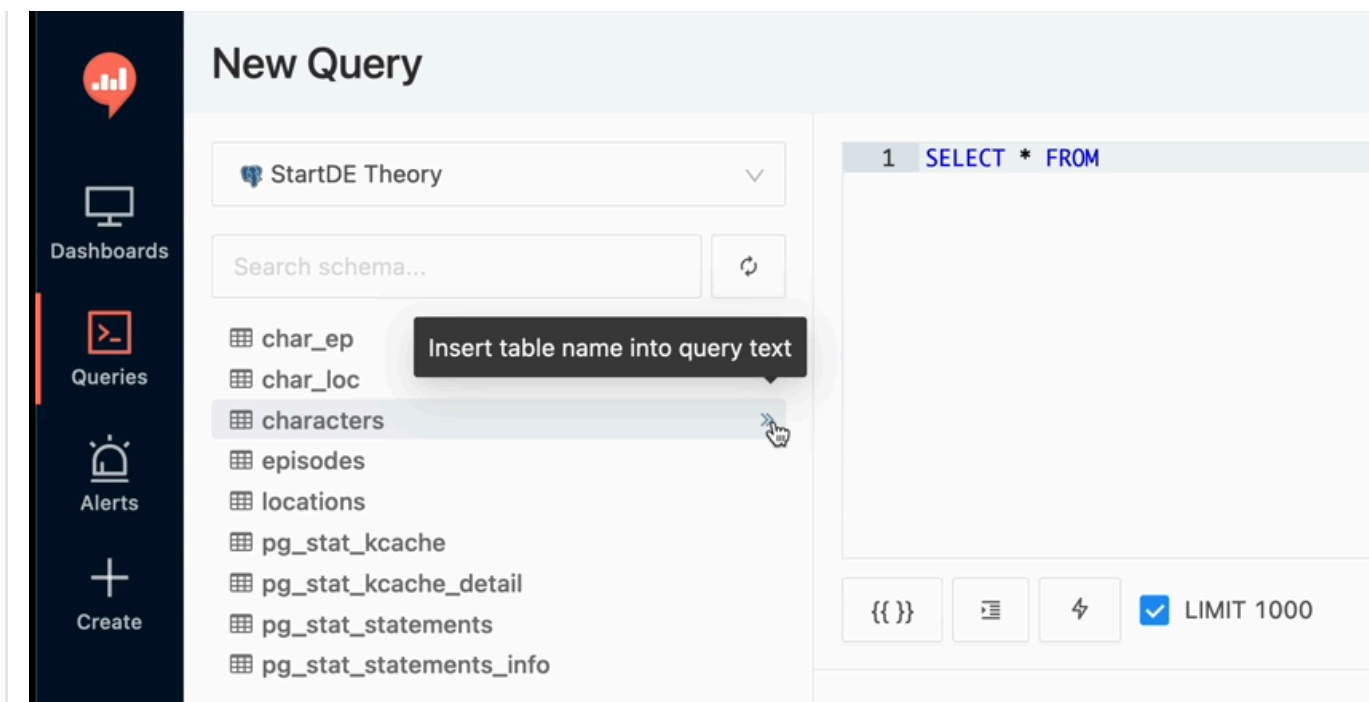
1. Появится набор баз данных, к которым у нас есть доступ. Нас интересуют база данных **StartDE Theory**.



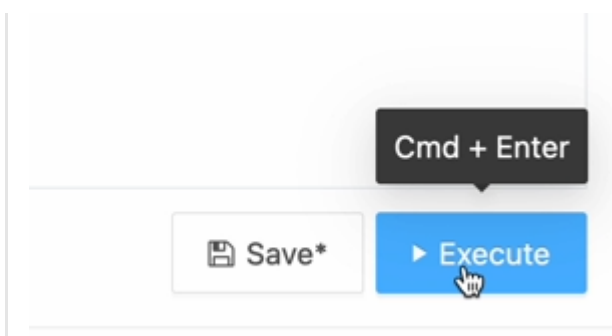
2. Мы видим несколько таблиц, в том числе системных.



3. Если нажать на правую стрелочку около таблицы, мы сможем получить ее название сразу в нашем `SELECT`.

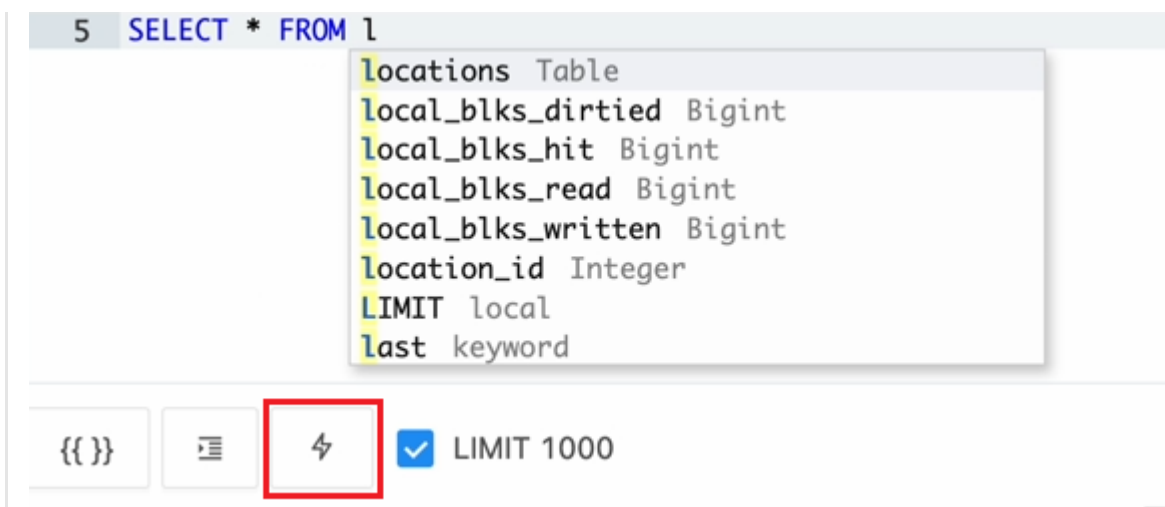


4. Внизу справа у нас есть кнопка **Execute**. С ее помощью запрос отправляется в СУБД.

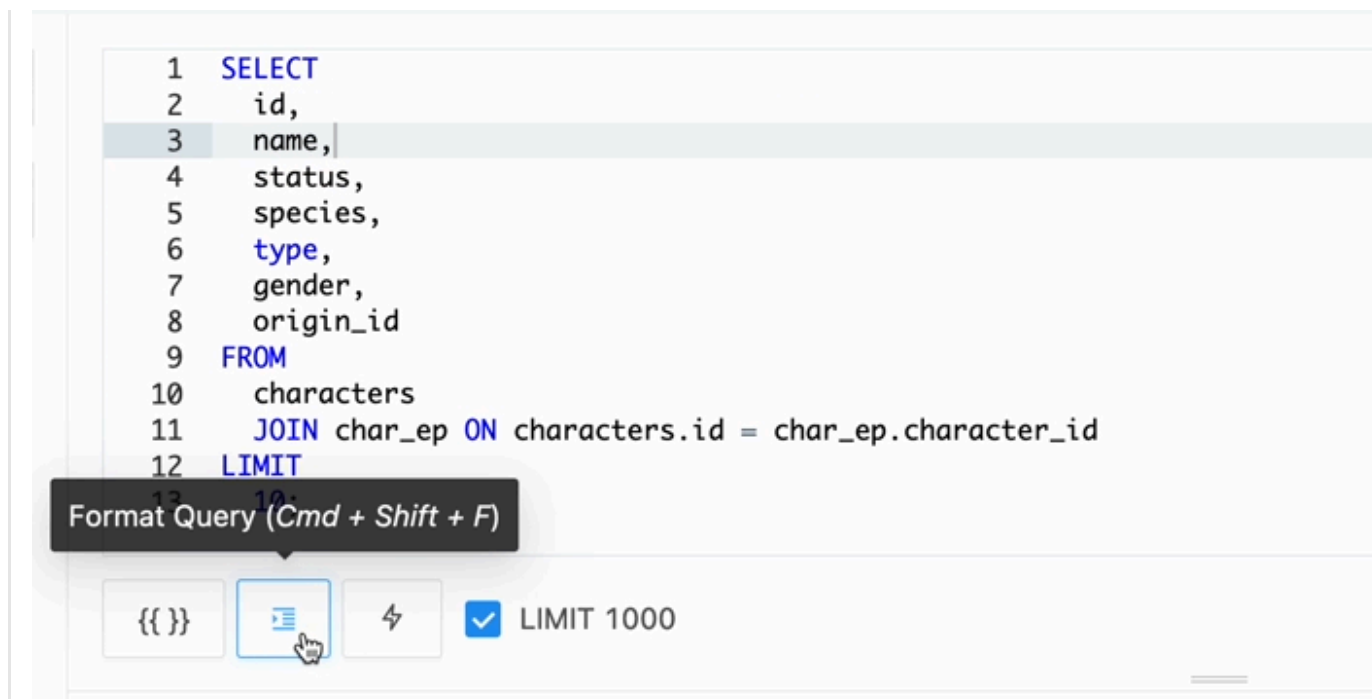


Некоторые фишки в Redash:

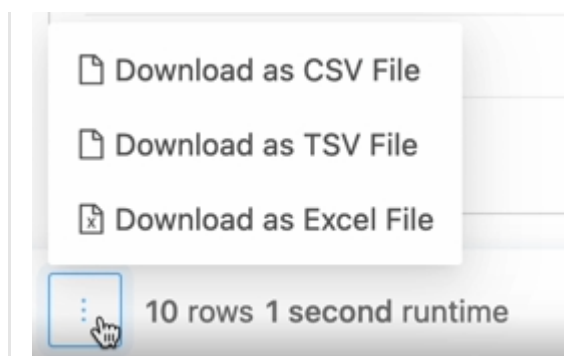
- **Автокомплит** (автозаполнение). Если мы пишем SELECT звездочка FROM I, видим, что у нас вылезает подсказка, из которой можно выбрать нужное нам значение. Автокомплит можно отключить.



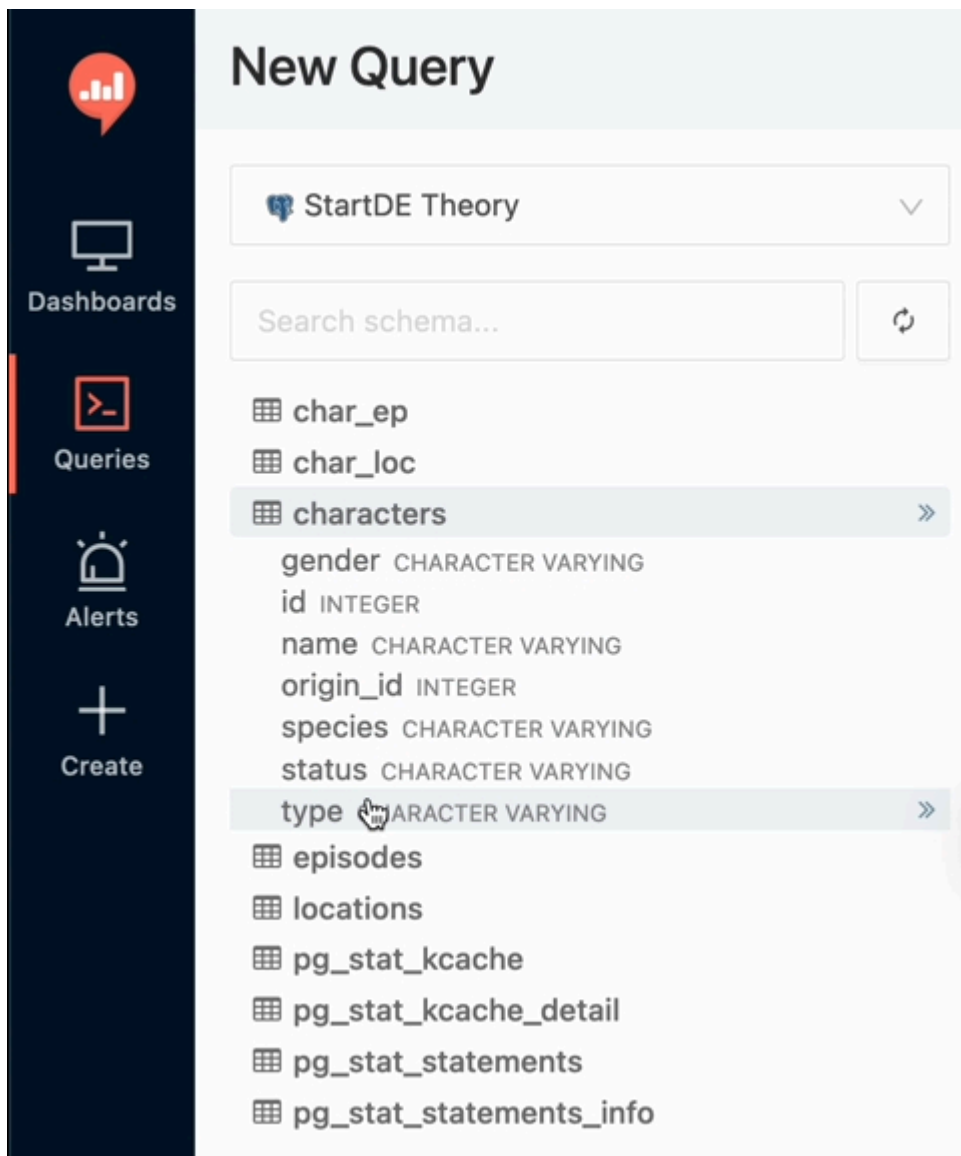
- **Форматирование.** Актуально для больших запросов на много строк. Когда необходимо привести все в порядок форматирование, мы нажимаем на соответствующую кнопку и получаем более аккуратный код, которым уже не стыдно поделиться со своими коллегами.



- Возможность **сохранить файл** в нужном формате для дальнейшей обработки с помощью других инструментов.



- **Просмотр** таблиц и наборов их полей.



Нюансы отображения в Redash:

- Redash «обрубает» до двух чисел после запятой при отображении результата вычисления. Можно убедиться в этом, преобразовав результат в текстовый тип.

```
1 SELECT 10.0 / 7|
```

{{ }}



LIMIT 1000

Table

+ Add Visualization

?column?

1.43

1 SELECT (10.0 / 7)::text|

I

{{ }}

☰

⚡

☒ LIMIT 1000

Table

+ Add Visualization

text

1.4285714285714286

- Redash видоизменяет дату, которая хранится в PostgreSQL. На самом деле в PostgreSQL, как и в большинстве баз данных, даты хранятся в следующем формате: сначала год, потом месяц, потом дата.


```

1 SELECT *
2 FROM episodes
3 WHERE air_date = '2013-12-02';
4 -- '02/12/13';

```

{{ }}



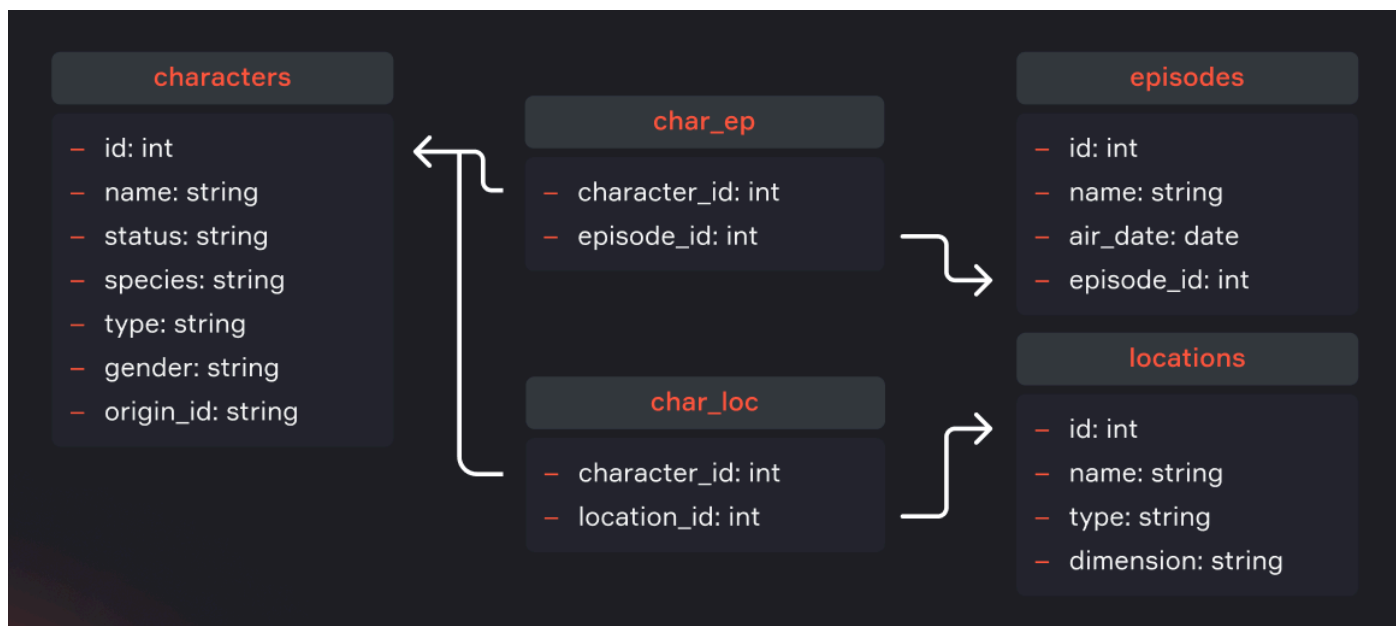
LIMIT 1000

Table

+ Add Visualization

id	name	air_date	episode_id
1	Pilot	02/12/13	S01E01

ПРИМЕР



Это схема по мультсериалу «Рик и Морти». Она включает в себя следующие таблицы:

- персонажи (таблица `characters`);
- эпизоды (таблица `episodes`);
- локации, в которых происходит действие (таблица `locations`);

- таблица, показывающая, в каких эпизодах и какие были персонажи (связывающая таблица `char_ep`);
- таблица, показывающая, в каких локациях и какие были персонажи (связывающая таблица `char_loc`).

Выполним запрос `SELECT * FROM characters;` и проанализируем результат.

Table

+ Add Visualization

id	name	status	species	type	gender	origin_id
1	Rick Sanchez	Alive	Human		Male	1
2	Morty Smith	Alive	Human		Male	
3	Summer Smith	Alive	Human		Female	20
4	Beth Smith	Alive	Human		Female	20
5	Jerry Smith	Alive	Human		Male	20

<

1

2

3

4

5

...

34

>

Мы написали максимально неоптимальный запрос для любой СУБД по следующим причинам:

1. Вывели все поля (столбцы)

Мы выбрали звездочку, то есть все поля, которые есть у нас в таблице именно в том виде, в котором они хранятся в базе данных. Однако очень часто необходимо выгрузить эти данные с помощью запроса и отнести в какое-то большое хранилище данных. И здесь уже очень важно, какой именно набор полей мы получаем, в каком именно порядке они располагаются. Разработчики базы данных могут добавить новое поле в таблицу, удалить существующее, поменять их местами и просто забыть нам об этом сказать. В таком случае наш пайплайн обрушится. Поэтому лучше всего всегда использовать список полей, которые нам нужны, и в нужном порядке. Это `id`, `name`, `status`, `species`, `type`, `gender` и `origin_id`. Все эти поля перечисляются через запятую.



```
1 SELECT id, name, status, species, type, gender, origin_id
2 FROM characters
```

2. Вывели все записи (строки)

Как мы видим, нам вернулось 34 страницы по 25 записей в каждой. Вряд ли мы сейчас будем работать со всеми. Поэтому, если нам не нужны все строки,

необходимо в конце любого SELECT писать LIMIT, например, 10. Это ограничивает набор возвращающихся записей, и мы можем работать с тем количеством, которое указали. Таким образом мы не нагружаем сеть, не забираем всю таблицу (которая может занимать десятки гигабайт).



```
1 SELECT id, name, status, species, type, gender, origin_id
2     FROM characters
3 LIMIT 10
```

<aside>



Необходимо ограничивать количество выбранных строк только в том случае, если они действительно нужны не все.

Ниже приведены операторы DQL (Data Query Language), которые использовались в данном уроке:

SELECT	Основной оператор для запроса данных из одной или нескольких таблиц. Позволяет выбрать конкретные столбцы и строки на основе заданных условий
FROM	Указывает на таблицу в базе данных, к которой обращается запрос
LIMIT	Ограничивает количество

возвращаемы х строк

Резюме

- Таблица в реляционной БД состоит из полей (столбцов) и записей (строк), на пересечении которых находятся значения (значение поля для записи)
- Применительно к полям существует два понятия:
 - **Первичный ключ.** Он нужен для того, чтобы уникально идентифицировать какую-либо строку в таблице. Для всех записей в одной таблице первичным ключом является одно или несколько полей данной таблицы
 - **Внешний ключ.** Это ссылка из одной таблицы на первичный ключ другой таблицы
- Применительно к таблицам в реляционной БД существуют следующие связи:
 - **Один к одному.** Одной записи таблицы соответствует только одна запись другой таблицы.
 - **Один ко многим.** Одному первичному ключу может соответствовать несколько внешних ключей в соседней таблице
 - **Многие ко многим.** Каждая запись в одной таблице может соответствовать множеству записей в другой таблице и наоборот
- **SQL (Structured Query Language)** — это декларативный унифицированный язык. В разных СУБД он может незначительно отличаться. Перечень подязыков SQL:
 - **DQL (Data Query Language)**
 - **DML (Data Manipulation Language)**
 - **DDL (Data Definition Language)**
 - **DCL (Data Control Language)**
 - **TCL (Transaction Control Language)**
- При работе в Redash можно использовать:
 - **Автокомплит** (автозаполнение)

- **Форматирование**
- **Сохранение файла** в нужном формате
- **Просмотр** таблиц и наборов их полей
- При работе в Redash важно помнить:
 - Redash «обрубает» до двух чисел после запятой при отображении результата вычисления
 - Redash видоизменяет дату, которая хранится в Postgres
- При написании SQL-запроса к базе данных необходимо придерживаться следующих правил:
 - Всегда использовать список полей в том порядке, которые нам нужны (приписывать их в явном виде)
 - Ограничивать количество выбранных строк в том случае, если они действительно нужны не все



Урок 2

[О типах данных и функциях](#)

[Числа](#)

[Операторы](#)

[Функции](#)

[Символьные типы](#)

[Функции](#)

[Дата и время](#)

[Функции](#)

[Флаг \(Boolean\)](#)

[Функции](#)

[Оператор Case](#)

[Преобразование типов](#)

[Дополнительные типы](#)

[Алиас](#)

[Резюме](#)

О типах данных и функциях

В большинстве баз данных существуют следующие основные **типы данных**:

- **числовые;**
- **символьные;**
- **дата-время;**
- **флаги.**

Также бывают дополнительные типы данных, такие как массивы, JSON, UUID, бинарные типы, геоданные.

Типы данных играют важную роль в БД по нескольким причинам:

- помогают обеспечить **целостность данных**, гарантируя, что в колонках хранятся только допустимые значения (например, колонка с числовым типом данных не будет принимать строковые значения);
- **оптимизируют хранение**, позволяя эффективно использовать память. Например, хранение целочисленного типа требует меньше памяти, чем хранение этой же информации в виде строки (набора символов). Кроме того, и для числовых, и для символьных типов также существуют подтипы, ограничивающие размер, который может занимать значение колонки в памяти. Например, это может быть ограничение по максимальному количеству символов для строкового типа `VARCHAR(255)` или ограничение по максимальному значению для целочисленного типа данных (тип данных `int` имеет ограничение в 2147483647);
- **оптимизируют производительность**, позволяя быстрее выполнять нужные нам операции. Например, операции сравнения и сортировки выполняются быстрее для числовых типов данных по сравнению с символьными;
- **упрощают работу** с данными, проверяя корректность данных, которые вносятся в базу, а также делая базу данных более читаемой и понятной для разработчиков и администраторов.

Типы данных могут отличаться в зависимости СУБД, поэтому необходимо сверяться с документацией той базы, с которой вы работаете.

Функция — это подпрограмма, которая принимает на вход аргументы, выполняет определенные операции (например, вычисления или манипуляции с данными) и возвращает результат. Например, функция суммирования с двумя входными параметрами будет брать значения первого и второго аргумента, вычислять и затем возвращать их сумму.

При желании результат работы любой функции можно использовать далее, в следующей функции. Для этого необходимо вложить одну функцию в другую. В качестве примера такой вложенности можно представить функцию, которая вычисляет квадрат суммы, где сначала вычисляется сумма первой функцией, а результат уже возводится в квадрат с помощью второй.

Числа

Как и в математике, числа бывают целые и дробные. В отличие от целых чисел, в дробных есть какое-то количество знаков после запятой, которые могут быть и нулями (в таком случае дробное число будет равно целому). В разных СУБД числовые типы обозначаются по-разному, а также имеют различные диапазоны допустимых значений и размер занимаемой памяти:

- **Целочисленные** (TINYINT, SMALLINT, MEDIUMINT, INT (или INTEGER), BIGINT)
- **Десятичные дроби** (FLOAT, DOUBLE, DECIMAL, NUMERIC)

Операторы

У числовых типов есть операторы и функции. Операторы выполняют математические операции, такие как сложение, вычитание, произведение, деление. Кроме этого, в SQL есть дополнительные операторы — остаток от деления, возведение в степень и модуль числа.

Сложение	+	$3 + 2 \rightarrow 5$ $3 + 2.0 \rightarrow 5.0$	Остаток от деления	%	$3 \% 2 \rightarrow 1$ $3 \% 2.0 \rightarrow 1.00$
Вычитание	-	$3 - 2 \rightarrow 1$ $3 - 2.0 \rightarrow 1.0$	Возведение в степень	^	$3 ^ 2 \rightarrow 9$ $3 ^ 2.0 \rightarrow 9.00$
Произведение	*	$3 * 2 \rightarrow 6$ $3 * 2.0 \rightarrow 6.0$	Модуль числа	@	$@ -3 \rightarrow 3$ $@ -3.0 \rightarrow 3.0$
Деление	/	$3 / 2 \rightarrow 1$ $3 / 2.0 \rightarrow 1.50$			

Обратите внимание, что когда в операторах вы используете только целое значение — результат будет целочисленным. Если какой-нибудь из аргументов у нас дробный, то и результат будет дробным.

Функции

Для выполнения различных вычислений и преобразований числовых типов в большинстве СУБД реализованы следующие функции:

Остаток от деления	mod (числитель, знаменатель)	mod (5, 2) → 1 mod (11, 3) → 2
Целочисленное деление	div (числитель, знаменатель)	div (5, 2) → 2 div (11, 3) → 3
Округление	round (число)	round (1.54) → 2 round (27.3) → 27
Ближайшее большее целое	ceil (число)	ceil (1.54) → 2 ceil (27.3) → 28
Ближайшее меньшее целое	floor (число)	floor (1.54) → 1 floor (27.3) → 27
Возведение в степень	power (число, степень)	power (3, 3) → 27 sqrt (1.5, 3) → 3.38
Квадратный корень	sqrt (число)	sqrt (2) → 4 sqrt (1.5) → 1.22



Если вы были внимательны, то могли заметить, что остаток от деления можно выполнить и с помощью оператора `%`, и с помощью функции `DIV`, на результат вычисления это не повлияет. Аналогично несколькими способами можно выполнить возведение в степень.

Если мы хотим использовать нашу СУБД в качестве калькулятора, то нам необходимо использовать оператор `SELECT`, после которого перечислить через запятую необходимые нам выражения и функции. При этом нет необходимости указывать оператор `FROM`, так как мы не запрашиваем данные из какой-либо таблицы.

Пример использования числовых операторов и функций:

```
SELECT 1 + 2 AS simple_sum,      -- результат 3
       1 + 2.0 AS second_sum,   -- результат 3.00
       4 - 8 AS negative_diff,  -- результат -4
       5 * 6 AS abs,            -- результат 30
```

```

round(2.5) as our_round,      -- результат 3.00
ceil(2.5) AS ceil,           -- результат 3.00
floor(2.5) AS floor,         -- результат 2.00
power(3, 5) AS power,        -- результат 243
sqrt(25) AS square_root     -- результат 5.00

```

Символьные типы

Символьные типы включают в себя любые символы, строки, буквы или даже предложения. Как и числовые типы, символьные в разных СУБД описываются по-разному (TEXT, STRING, CHAR, VARCHAR), а также имеют различные диапазоны длин и размер занимаемой памяти. Поэтому необходимо сверяться с документацией той базы данных, с которой вы работаете.

Функции

Объединение	concat (a, b, c)	concat ('У Вали было ', 2, ' апельсина') → 'У Вали было 2 апельсина' concat (1, 2, 3) → '123'
Длина строки	length (строка)	length ('У Вали было 2 апельсина') → 23 length ('123') → 3
Обрезание пробелов	trim (строка)	trim (' текст ') → 'текст' trim (' 123 ') → '123'
Поиск подстроки	position (подстрока in строка)	position ('ель' in '2 апельсина') → 5 position ('4' in '123') → 0
Верхний регистр	upper (текст)	upper ('текст') → 'ТЕКСТ' upper ('123') → '123'
Нижний регистр	lower (текст)	lower ('ТЕКСТ') → 'текст' lower ('123') → '123'

Обратите внимание, что строки необходимо заключать в кавычки, которые могут быть как одинарными, так и двойными. Также следует помнить, что при подсчете длины строки считаются и пробелы.

Пример использования символьных функций:

```

SELECT upper(name),           -- имя в верхнем регистре
       length(name),         -- количество символов в
       concat(gender, ' ', species), -- объединение пола и вида
       position('Smith' in name) -- позиция подстроки
FROM public.characters
LIMIT 10;

```

```

1 SELECT upper(name),
2     length(name),
3     concat(gender, ' ', species),
4     position('Smith' in name)
5 FROM public.characters
6 LIMIT 10;

```

{{ }}



LIMIT 1000

StartDE Theory



Table



upper	length	concat	position
RICK SANCHEZ	12	Male Human	0
MORTY SMITH	11	Male Human	7
SUMMER SMITH	12	Female Human	8
BETH SMITH	10	Female Human	6

Дата и время

В SQL есть четыре типа даты и времени:

- Date — просто дата
- DateTime — дата и время

- Time — только время
- Timestamp

Timestamp — это число, которое содержит количество секунд, прошедших с 1 января 1970 года. Этот формат принят в Unix-системах и часто он полезнее, чем дата или время. Как правило, пользователи не задумываются над тем, в каком часовом поясе находится тот или иной сервер. Timestamp всегда отсчитывает секунды по UTC, при каждом запросе мы получаем абсолютное время, которое сервер переводит в текущий для нас часовой пояс. Но под капотом оно не будет отличаться в зависимости от места нахождения сервера.

Функции

Текущая дата и время	now ()	now() → date '2026-05-04 12:44:00'
Прибавление	дата + число	date '2001-09-28' + 7 → date '2001-10-05'
Убавление	дата - число	date '2001-09-28' - 7 → date '2001-09-21'
Разница	дата - дата	date '2001-10-01' - date '2001-09-28' → 3
Часть	extract(часть from дата)	extract(year from date '2001-10-01') → 2001

В примерах выше обратите внимание, что `DATE` является конструктором даты, который явно указывает, что строка '2001-09-28' должна интерпретироваться как дата.

Пример использования функций даты и времени:

```
SELECT name,                -- имя
       extract(year from air_date)::int  -- год
FROM episodes;
```

```
1 SELECT name,  
2     extract(year from air_date)::int  
3 FROM episodes  
4 LIMIT 10;
```

[[]] [] [⚡] ☒ LIMIT 1000 StartDE Theory [] [📄] [▶]

Table +

name	extract
Pilot	2,013
Lawnmower Dog	2,013
Anatomy Park	2,013
M. Night Shaym-Aliens!	2,014

Флаг (Boolean)

Данный тип чаще всего описывается как Boolean или Bool. Внутри базы данных он хранится как 1 или 0, а в интерфейсе обычно показывается как True или False. Это флаг, который отвечает на определенный вопрос, например:

- сдал ли студент зачет;
- оплачена ли покупка;
- есть ли пользователю 18 лет;
- пройден ли онбординг.

Функции

Логическое "Не"	NOT	NOT true → false	NOT false → true
Логическое "И"	AND	true AND false → false	true AND true → true
Логическое "Или"	OR	true OR false → true	false OR false → false
Равно	=	5 = 5 → true	false = false → true
Больше	>	5 > 5 → false	7 > 5 → true
Больше или равно	>=	5 >= 5 → true	7 >= 5 → true
Меньше	<	5 < 5 → false	5 < 7 → true
Меньше или равно	<=	5 <= 5 → true	5 <= 7 → true
Между	BETWEEN	5 BETWEEN 3 AND 7 → true	3 BETWEEN 5 AND 7 → false

В SQL приоритетность операторов выполнения логических операций определяет порядок, в котором эти операции будут выполняться. Знание приоритетности операторов помогает правильно формировать запросы и избегать ошибок в логике.

Приоритетность операторов в SQL:

- Операторы сравнения `=`, `!=`, `<`, `>`, `<=`, `>=`, `<>`, `BETWEEN`, `LIKE`, `IN`, `IS NULL`, `IS NOT NULL` имеют самый высокий приоритет среди логических операций и выполняются первыми при оценке выражений
- Логический оператор `NOT` имеет более высокий приоритет, чем `AND` и `OR`, и используется для инверсии логического значения
- Логический оператор `AND` имеет средний приоритет и используется для объединения двух логических выражений, оба из которых должны быть истинными, чтобы результат работы оператора вернул True, иначе он вернет False
- Логический оператор `OR` имеет самый низкий приоритет и используется для объединения двух логических выражений, одно из которых должно быть истинным, чтобы результат работы оператора вернул True, иначе он вернет False

Оператор Case

Оператор **CASE** в SQL является мощным инструментом для создания условных логических выражений, позволяющих выполнять различные действия в зависимости от заданных условий. Он может быть применен в **SELECT**, **UPDATE** и других операторах для выбора данных, их агрегации, обновления и создания вычисляемых полей. Синтаксис оператора похож на условные конструкции **if-else** в языках программирования и может быть применен в различных сценариях.

```
CASE                                -- оператор
  WHEN condition_1 THEN result_1   -- условие_1
  WHEN condition_2 THEN result_2   -- условие_2
  ...
  ELSE result_N                    -- условие_N
END                                -- конец логического выражения
```

Пример применения логических функций и оператора CASE:

```
SELECT name,
       CASE WHEN dimension = 'Dimension C-137' THEN 'Earth'
            ELSE 'not Earth' END AS Earth,
       type = 'Planet' AS is_planet,
       id > 10 AS not_top10,
       (type = 'Planet')::int AS is_planet_int
FROM locations
LIMIT 100;
```

```

1 SELECT name,
2     CASE WHEN dimension = 'Dimension C-137' THEN 'Earth'
3     ELSE 'not Earth' END AS earth,
4     type = 'Planet' AS is_planet,
5     id > 10 AS not_top10,
6     (type = 'Planet')::int AS is_planet_int
7 FROM locations
8 LIMIT 100;

```



name	earth	is_planet	not_top10	is_planet_int
Earth (C-137)	Earth	true	false	1
Abadango	not Earth	false	false	0
Citadel of Ricks	not Earth	false	false	0
Worldender's lair	not Earth	true	false	1

Преобразование типов

Мы уже сталкивались с тем, что результатом сложения целого и дробного чисел является дробное число. А также с преобразованием числа в текст при конкатенации текста и числа. Это примеры **неявного преобразования**, когда СУБД понимает, что тип данных одного столбца можно преобразовать в тип данных другого без явного указания.

Также мы можем явно указать, что тип данных должен быть преобразован в другой тип, например, преобразовав текст в число, и работать с ним как с числом. Для этого используется **явное преобразование**, которое осуществляется помощью двух двоеточий `::` или с помощью функции `CAST`.

Правилом хорошего разработчика является «явное лучше неявного». Поэтому даже если СУБД преобразовывает все внутри себя, лучше сделать это преобразование явным. Важно помнить это правило, поскольку не все СУБД поддерживают одинаковые правила неявного преобразования типов, что может привести к проблемам при переносе кода между различными

СУБД. Также неявные преобразования могут быть менее производительными, так как СУБД перед преобразованием должна дополнительно определить тип.



Явное преобразование делает работу кода более предсказуемой.

Пример преобразования:

```
SELECT
  CAST('123' AS INTEGER),
  '123' :: INTEGER;
```

```
1  SELECT
2  CAST('123' AS INTEGER) AS result_1,
3  '123' :: INTEGER AS result_2;
```

{{ }}



LIMIT 1000

StartDE Theory



Table



result_1

result_2

123

123

Дополнительные типы

- **Массивы** — это набор значений одного типа. Например, мы не хотим добавлять 255 колонок и складываем все в единый массив в одну колонку;

- **JSON** — это текстовый формат для работы со структурированными данными. Некоторые СУБД позволяют работать с этим типом не как со строкой, а как с полноценной структурой, быстро находя нужные значения (как в Python);
- **UUID** — это стандарт идентификации. Обычно он генерируется программно, и процент совпадений таких UID-ов очень низок. Это альтернатива ID, который мы заполняем возрастающими значениями;
- **Бинарные типы** позволяют хранить картинки, видео и прочие файлы. С данными типами нет смысла работать с помощью SQL, поэтому на практике файлы хранят в отдельном файловом хранилище, а в таблице хранят ссылку на файл;
- **Геоданные** позволяют указывать, например, точку или область на карте. Некоторые СУБД предоставляют такой функционал для работы.

Разные СУБД представляют разные типы, и это только самые распространенные дополнительные типы.

Алиас

Чтобы дать название тому, что возвращает нам функция или выражение, мы можем использовать псевдоним или алиас (alias). Для этого нам необходимо использовать инструкцию «AS» после нашей функции или нашего выражения. Технически можно её не использовать, это не будет ошибкой. Однако в таком случае ухудшается читаемость больших запросов, поэтому все же рекомендуется использовать псевдонимы.

Посмотрим на один из примеров, приведенных в этом уроке:

```
SELECT name,
       CASE WHEN dimension = 'Dimension C-137' THEN 'Earth'
            ELSE 'not Earth' END AS Earth,
       type = 'Planet' AS is_planet,
       id > 10 AS not_top10,
       (type = 'Planet')::int AS is_planet_int
```

```
FROM locations  
LIMIT 100;
```

В полученной таблице для четырех из пяти столбцов мы явно прописали их названия, то есть алиасы: `Earth`, `is_planet`, `not_top10`, `is_planet_int`. А что будет выведено, если их не прописать, предлагаем вам проверить самостоятельно



Резюме

- В большинстве баз данных существует несколько основных **типов данных**:
 - **Числовые**. Среди них выделяют целочисленные (`TINYINT`, `SMALLINT`, `MEDIUMINT`, `INTEGER`, `BIGINT`) и десятичные дроби (`FLOAT`, `DOUBLE`, `DECIMAL`, `NUMERIC`)
 - **Символьные** (`TEXT`, `STRING`, `CHAR`, `VARCHAR`)
 - **Дата-время** (`DATE`, `DATETIME`, `TIME`, `TIMESTAMP`)
 - **Флаги** (`BOOLEAN` или `BOOL`)
- Разные типы предназначены для разных диапазонов значений (или количества символов) и количества занимаемой памяти
- Преобразование типов в СУБД бывает явное и неявное. Явное преобразование (с помощью оператора `CAST` или `::`) делает работу кода более предсказуемой
- Функция — это подпрограмма, которая принимает на вход аргументы, выполняет определенные операции (например, вычисления или манипуляции с данными) и возвращает результат. При желании результат работы любой функции можно использовать далее, в следующей функции. У числовых типов помимо функций есть операторы, которые выполняют математические операции
- Оператор `CASE` — это мощный инструмент в SQL, который позволяет создавать условия для выполнения различных операций в зависимости от значения столбца или выражения

- Хорошей практикой считается присвоение алиаса (псевдонима) результату вычисления выражения или работы функции



Урок 3

[Фильтрация \(WHERE\)](#)

[Основные методы фильтрации текста](#)

[Сортировка \(ORDER BY\)](#)

[Ограничение выборки \(LIMIT, OFFSET\)](#)

[Code Style](#)

[Работа с отсутствующими данными \(NULL\)](#)

[Вопросы на собеседовании](#)

[Резюме](#)

Фильтрация (WHERE)

Фильтрация данных является одной из ключевых задач при работе с базами данных. Она необходима для выбора одной или более строк по заданному условию. Тем самым фильтрация позволяет сосредоточиться только на интересующих нас данных.

Общий вид запроса с фильтрацией:

```
SELECT *           -- выбор полей
  FROM table        -- выбор таблицы
 WHERE <условие>    -- условие
```

Выражение, записанное в качестве условия фильтрации, должно возвращать булево значение (True или False). Синтаксис похож на работу с булевыми переменными:

- **Равенство и неравенство** (= , <>)
- **Сравнения** (> , >= , < , <=)
- **Инструкция IN** — проверка вхождения значения в список значений

- **Инструкция NOT** — инверсия условия (`NOT IN` , `NOT BETWEEN` , `NOT LIKE` , `NOT EXISTS` , `IS NOT NULL` , а также комбинированные условия `NOT (department = 'HR' AND age >= 30)`)
- **Инструкция BETWEEN** — проверка вхождения значения в диапазон

Для фильтрации по нескольким условиям используются логические операторы `AND` и `OR` . При их использовании следует помнить, что **AND** имеет более высокий приоритет, чем **OR**.

Пример использования фильтрации:

```
SELECT * FROM characters
WHERE species = 'Human' AND status = 'Alive'
LIMIT 10;
```

```
1 SELECT * FROM characters
2 WHERE species = 'Human' AND status = 'Alive'
3 LIMIT 10;
```

{{ }}

☰

⚡

☒ LIMIT 1000

StartDE Theory ▾

💾*

▶

Table +

id	name	status	species	type	gender
1	Rick Sanchez	Alive	Human		Male
2	Morty Smith	Alive	Human		Male
3	Summer Smith	Alive	Human		Female

Если в фильтрации используется несколько условий, то для явного указания приоритетности выполнения операций можно использовать скобки (без них СУБД будет выполнять операции согласно приоритетам, которые мы рассмотрели в предыдущем уроке).

Использование скобок важно для правильного выполнения запросов и получения ожидаемых результатов, также оно улучшает читаемость. Поэтому их лучше использовать всегда, даже если порядок выполнения с ними и без них будет одинаков.

Следующие два запроса выдадут одинаковый результат, однако запрос с явным указанием приоритетности с помощью скобок легче читается и интерпретируется:

```
SELECT * FROM employees  
WHERE department = 'Sales' OR department = 'Marketing' AND salary > 50000;
```

```
SELECT * FROM employees  
WHERE department = 'Sales' OR (department = 'Marketing' AND salary > 50000);
```

Запрос выберет всех сотрудников из отдела продаж и тех сотрудников из отдела маркетинга, у которых зарплата больше 50000.

А в следующем запросе скобки изменили порядок выполнения операций. Запрос выберет сотрудников, которые работают либо в отделе продаж, либо в отделе маркетинга, при этом зарплата которых зарплата больше 50000:

```
SELECT * FROM employees  
WHERE (department = 'Sales' OR department = 'Marketing') AND salary > 50000;
```



Для избегания ошибок рекомендуется задавать приоритет выполнения логических операторов с помощью круглых скобок

Основные методы фильтрации текста

Особое внимание в SQL следует уделить фильтрации символьных типов данных, которая предоставляет множество возможностей для поиска и отбора строк, которые соответствуют определённым текстовым условиям. Использование операторов сравнения, шаблонов, встроенных функций и логических операторов позволяет создавать гибкие и мощные запросы для работы с текстовыми данными.

Наиболее часто используемые из них:

1. Оператор равенства (=) и неравенства (<> или !=)

Оператор равенства используется для поиска строк, которые точно соответствуют указанному тексту.


```
SELECT * FROM customers
WHERE city = 'New York';
```

Этот запрос выбирает всех клиентов, у которых город проживания указан как «New York».

Оператор неравенства используется для поиска строк, которые не соответствуют указанному тексту.

```
SELECT * FROM customers
WHERE city != 'New York';
```

Этот запрос выбирает всех клиентов, у которых город проживания не «New York».

2. Оператор **LIKE**

Оператор **LIKE** используется для поиска строк по шаблону. В нем используются два специальных символа:

- **%**: заменяет любое количество символов (в том числе ноль символов).
- **_**: заменяет один символ.

Примеры:

```
SELECT * FROM customers
WHERE name LIKE 'J%';
```

Этот запрос выбирает всех клиентов, чьи имена начинаются с буквы «J».

```
SELECT * FROM customers
WHERE email LIKE '%@gmail.com';
```

Этот запрос выбирает всех клиентов, чьи email-адреса заканчиваются на «@gmail.com».

```
SELECT * FROM products
WHERE code LIKE 'A_1';
```

Этот запрос выбирает все продукты, чей код начинается с буквы «А», имеет любой один символ на втором месте, и заканчивается на «1».

3. Оператор **ILIKE** (в некоторых СУБД, например, PostgreSQL)

В некоторых системах управления базами данных (СУБД) оператор **ILIKE** используется для регистронезависимого поиска строк по шаблону. Например, в PostgreSQL:

```
SELECT * FROM customers
WHERE name ILIKE 'j%';
```

Этот запрос выберет всех клиентов, чьи имена начинаются с буквы «j» или «J».

4. Оператор **IN**

Оператор **IN** позволяет фильтровать строки, которые соответствуют одному из значений из списка.

```
SELECT * FROM customers
WHERE city IN ('New York', 'Los Angeles', 'Chicago');
```

Этот запрос выбирает всех клиентов, которые живут в одном из трех указанных городов.

5. Использование функций для работы с текстом

Многие СУБД предоставляют встроенные функции для работы с текстом, такие как **LOWER()**, **UPPER()**, **TRIM()**, **SUBSTRING()** и другие. Эти функции могут использоваться в условиях **WHERE** для выполнения более сложных фильтров.

Примеры:

- Преобразование текста к нижнему регистру для регистронезависимого поиска:

```
SELECT * FROM customers
WHERE LOWER(name) = 'john doe';
```

- Поиск строк, содержащих определенную подстроку:

```
SELECT * FROM courses
WHERE SUBSTRING(description, 1, 4) = 'Data';
```

- Удаление пробелов из начала и конца строки:

```
SELECT * FROM customers
WHERE TRIM(name) = 'John Doe';
```

Комбинирование условий

Вы можете комбинировать различные условия фильтрации текста с помощью логических операторов **AND**, **OR**, и **NOT** для создания сложных запросов.

Пример:

```
SELECT * FROM customers
WHERE (city = 'New York' OR city = 'Los Angeles')
AND name LIKE 'J%'
AND email LIKE '%@gmail.com';
```

Этот запрос выбирает всех клиентов, которые живут в Нью-Йорке или Лос-Анджелесе, их имена начинаются с буквы «J» и email-адреса заканчиваются на «@gmail.com».

Когда в SQL-запросе есть вычисляемые поля (поля, значения которых вычисляются во время выполнения запроса), для фильтрации по такому полю необходимо продублировать вычисляемое выражение. Использование алиаса в данном случае вызовет ошибку в СУБД.

Пример правильного использования:

```
SELECT
    product_id,
    price,
    discount,
    price * discount AS discounted_price
FROM
    products
WHERE
    price * discount > 3000;  -- Условие фильтрации дублирует
вычисляемое выражение
```

Пример неправильного использования:

```
SELECT
    product_id,
    price,
    discount,
    price * discount AS discounted_price
FROM
    products
WHERE
    discounted_price > 3000;  -- Использование алиаса вызовет
ошибку
```



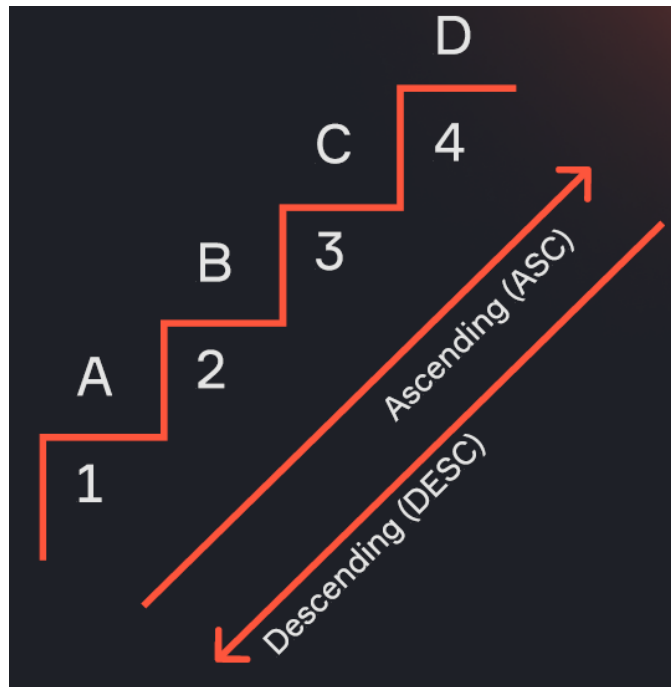
Не используйте алиасы для фильтрации по вычисляемым полям в SQL-запросах

Сортировка (ORDER BY)

Сортировка данных позволяет упорядочивать строки в порядке возрастания или убывания. Сортировка происходит по одному или нескольким столбцам, указанным в SQL-запросе. Для этого используется инструкция `ORDER BY`:

- **ASC** (используется по умолчанию) — сортировка по возрастанию

- **DESC** — сортировка по убыванию



Общий вид запроса с сортировкой:

```
SELECT *                -- выбор полей
FROM table              -- выбор таблицы
WHERE <условие>         -- условие фильтрации
ORDER BY <поле_1> DESC, <поле_2> -- сортировка по убыванию поля
```

В качестве указания поля сортировки используется наименование поля, либо его алиас, либо порядковый номер поля (начиная с 1) среди всех полей к выводу, указанных в **SELECT**.

Сортировка чисел осуществляется в порядке возрастания (от меньшего к большему), а сортировка строковых типов данных осуществляется в лексикографическом порядке.

Ограничение выборки (LIMIT, OFFSET)

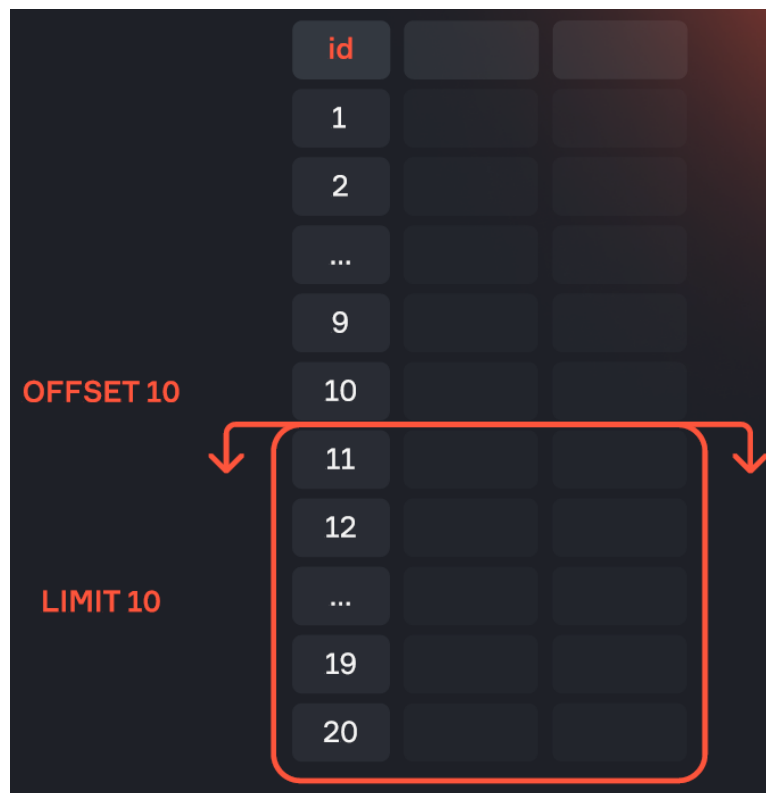
Для ограничения количества возвращаемых строк используется инструкция **LIMIT**. В сочетании с **OFFSET** она позволяет реализовать пагинацию

(разделение данных на страницы для удобства отображения и управления большим объемом информации).

Общий вид запроса с ограничением выборки:

```
SELECT *           -- выбор полей
  FROM table        -- выбор таблицы
 WHERE <условие>    -- условие
 ORDER BY <поле_1>, <поле_2> -- сортировка
  LIMIT n1          -- выводим n1 строк
  OFFSET n2         -- пропускаем первые n2 строк
```

Например, `LIMIT 10 OFFSET 10` выберет 10 строк, начиная с 11-й.



Пример использования сортировки и ограничения выборки:

```
SELECT *
  FROM locations
```

```
WHERE type in ('TV', 'Fantasy town', 'Planet')
ORDER BY name LIMIT 10 OFFSET 10;
```

```
1 SELECT *
2 FROM locations
3 WHERE type in ('TV', 'Fantasy town', 'Planet')
4 ORDER BY name LIMIT 10 OFFSET 10;
```

{{ }}



LIMIT 1000

StartDE Theory ▾



Table



id	name	type	dimension
69	Earth (C-35)	Planet	Dimension C-35
23	Earth (C-500A)	Planet	Dimension C-500A
74	Earth (Chair Dimension)	Planet	Chair Dimension
59	Earth (D716)	Planet	Dimension D716

Code Style

Хороший стиль написания SQL-запросов делает их более читаемыми и поддерживаемыми. Необходимо придерживаться следующих простых правил:

- **Инструкции** пишутся **заглавными буквами** (SELECT, FROM, WHERE, ORDER BY)



Использование верхнего регистра для ключевых слов — это вопрос стиля и соглашений в команде. Это необязательное требование SQL, но широко принятое соглашение, которое помогает улучшить читаемость и поддерживаемость кода.

- **Каждая инструкция** начинается с **новой строки**
- **Использование пробелов** для выравнивания улучшает читаемость (правило «коридора», когда ключевые элементы выровнены и легко различимы)
- В сложных запросах можно использовать **комментарии** для лучшей читаемости:
 - `--` используется для комментария на одной строке
 - `/* */` используется для комментария на нескольких строках

Пример написания SQL-запроса с использованием правила «коридора»:

```
1  SELECT *
2    FROM locations
3   WHERE type in ('TV', 'Fantasy town', 'Planet')
4   ORDER BY name LIMIT 10 OFFSET 10;
```

Работа с отсутствующими данными (NULL)

NULL — это специальное значение, которое обозначает отсутствие данных в ячейке таблицы. Оно отличается от пустой строки или нулевого значения и требует особого подхода при работе:

- **NULL не участвует в обычных сравнениях.** Вместо операторов сравнения необходимо использовать конструкции `IS NULL` и `IS NOT NULL`, которые возвращают True или False.


```
-- Выбор всех сотрудников, у которых не указана электронная почта
SELECT *
  FROM employees
 WHERE email IS NULL;

-- Выбор всех сотрудников, у которых указана электронная почта
SELECT *
  FROM employees
 WHERE email IS NOT NULL;
```

- Если **при выполнении арифметических операций** используются поля со значениями `NULL` в строках, то результатом вычисления для данных строк будет также `NULL`.

```
-- Расчет итоговой цены с учетом скидки
SELECT
  sale_id,
  price,
  discount,
  price - discount AS final_price
FROM
  sales;

-- Для строк со значениями NULL в discount результат в final_price
```

- **Функция** `COALESCE` возвращает первое не `NULL` значение из переданных ей аргументов. Она применится к каждому значению в колонке, и если это значение окажется `NULL` — заменяет его на значение, указанное вторым аргументом. В противном случае — функция просто вернёт значение колонки.

```
-- Выбор данных сотрудника с использованием COALESCE для обработки NULL
SELECT
  employee_id,
  first_name,
  COALESCE(last_name, '') AS last_name
```

```
FROM
    employees;
-- Если last_name NULL, то возвращается пустая строка
```

- **Сортировка.** В разных СУБД NULL может сортироваться как первое или последнее значение. Для явного указания необходимо использовать `NULLS FIRST` или `NULLS LAST`.

```
-- Сортировка продуктов по цене с NULL в начале
SELECT
    product_id,
    product_name,
    price
FROM
    products
ORDER BY
    price NULLS FIRST;
```

Пример обработки `NULL` с помощью функции `COALESCE` :

```
SELECT id, name, type,
       COALESCE(dimension, 'unknown')
FROM locations
WHERE type in ('TV', 'Fantasy town', 'Planet');
```

```

1 SELECT id, name, type,
2     COALESCE(dimension, 'unknown')
3 FROM locations
4 WHERE type in ('TV', 'Fantasy town', 'Planet');

```

{{ }}



LIMIT 1000

StartDE Theory ▾



Table



id	name	type	coalesce
1	Earth (C-137)	Planet	Dimension C-137
4	Worldender's lair	Planet	unknown
6	Interdimensional Cable	TV	unknown
8	Post-Apocalyptic Earth	Planet	Post-Apocalyptic Dimension
9	Purge Planet	Planet	Replacement Dimension
10	Venzenulon 7	Planet	unknown
11	Bepis 9	Planet	unknown
12	Cronenberg Earth	Planet	Cronenberg Dimension

Вопросы на собеседовании

1. Что вернет `NULL = NULL` ?

Ответ: `NULL = NULL` вернет `NULL`, потому что сравнение с `NULL` всегда неопределенно.

2. Что вернет `NULL OR TRUE` ?

Ответ: `NULL OR TRUE` вернет `TRUE`, так как оператор `OR` возвращает true, если хотя бы одно из условий истинно.

3. Что вернет `NULL AND TRUE` ?

Ответ: `NULL AND TRUE` вернет `NULL`, так как результат зависит от неизвестного значения `NULL`.

4. Как обрабатывается `NULL` при сортировке и как это изменить?

Ответ: В разных СУБД `NULL` может сортироваться по-разному. Для изменения порядка используйте `NULLS FIRST` или `NULLS LAST`.

Резюме

- Для выбора строк по заданному условию необходимо использовать фильтрацию данных в запросе с помощью инструкции `WHERE`. Для объединения нескольких условий используются `AND` и `OR`. Приоритет выполнения `AND` выше, чем у `OR`. Для задания приоритета в явном виде логические выражения помещают в круглые скобочки.
- Для сортировки данных в запросе используется инструкция `ORDER BY`, после которой указываются поля сортировки и порядок (`ASC` — по возрастанию, `DESC` — по убыванию).
- Для ограничения количества возвращаемых строк используются инструкции `LIMIT` и `OFFSET`, которые позволяют реализовать пагинацию выдачи.
- Хорошей практикой при написании кода считается руководство общепринятыми негласными правилами:
 - написание инструкций заглавными буквами
 - написание инструкций с новой строки
 - использование выравнивания для читаемости

- NULL обозначает отсутствие данных (в отличие от цифры 0 или пустой строки ""). Особенности работы с NULL:
 - При сравнениях используются операторы `IS NULL` и `IS NOT NULL`
 - При участии `NULL` в качестве одного из слагаемых при выполнении арифметических операций результатом будет также `NULL`
 - Для замены значений `NULL` на константные значения используется функция `COALESCE`
 - При сортировке полей значения `NULL` могут сортироваться как первое или последнее значение в зависимости от СУБД

К концу текущего урока вы можете написать SQL-запрос с использованием инструкций:

SELECT	Основной оператор для запроса данных из одной или нескольких таблиц. Позволяет выбрать конкретные столбцы и строки на основе заданных условий.
FROM	Указывает на таблицу в базе данных, к которой обращается запрос
WHERE	Указывает условия, по которым строки будут выбраны
ORDER BY	Упорядочивает строки в результате запроса по одному или нескольким столбцам. Может быть использован для сортировки по возрастанию (ASC) или убыванию (DESC)
LIMIT и OFFSET	LIMIT ограничивает количество возвращаемых строк. OFFSET пропускает указанное количество строк перед возвратом оставшихся строк



Урок 4. Сложные запросы

1. Инструкция `DISTINCT`
2. Объединение запросов (`UNION` и `UNION ALL`)
3. Объединение таблиц (`JOIN`)
4. Вопросы на собеседовании
5. Резюме

Инструкция `DISTINCT`

Инструкция `DISTINCT` используется для удаления дублирующихся строк из результата выполнения запроса. Применяя `DISTINCT`, мы можем получить только уникальные строки из таблицы.

Например, запрос ниже выведет уникальные значения в поле `column1`:

```
▼  
1 SELECT DISTINCT column1 FROM table_name;
```

В случае, когда нам необходимы уникальные комбинации значений в двух и более полях, мы указываем поля через запятую, а инструкцию `DISTINCT` указываем один раз, также после `SELECT`:

```
▼  
1 SELECT DISTINCT column1, column2 FROM table_name;
```

Когда мы используем `DISTINCT`, СУБД запоминает каждое уникальное значение и сравнивает его с последующими, что может замедлить выполнение запроса. Это может быть полезно, например, в случае сетевых ошибок, когда источник данных может отправить для записи в СУБД одно и то же сообщение дважды.

Пример использования инструкции **DISTINCT** :

▼

```
1 SELECT DISTINCT type
2   FROM locations;
```

1 SELECT DISTINCT type

2 FROM locations;

{{ }}

☰

⚡

☒ LIMIT 1000

StartDE Theory ▼

💾*

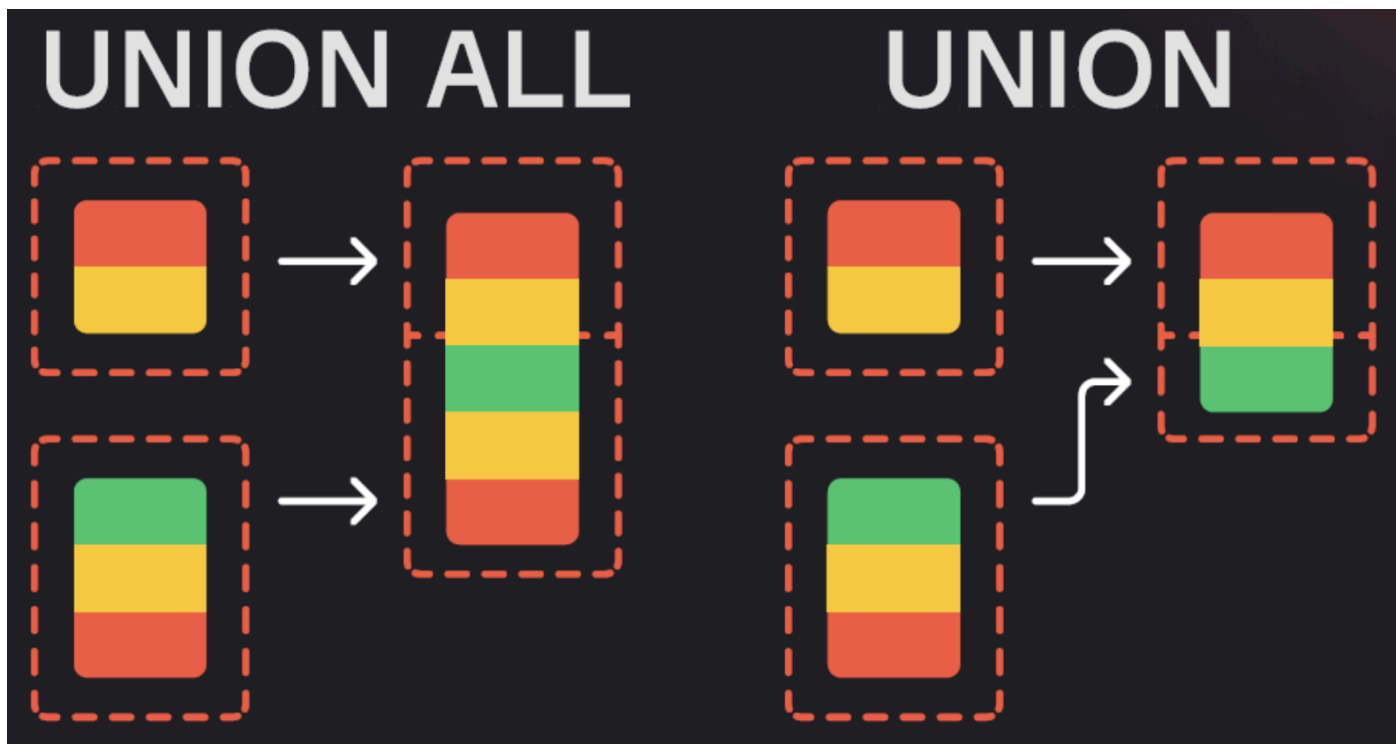
▶

Table	+
type	
Non-Diegetic Alternative Reality	
Death Star	
Arcade	

Объединение запросов (UNION и UNION ALL)

Когда мы работаем с SQL, нам часто нужно объединить результаты двух или более запросов. Для этого используются операции **UNION** и **UNION ALL**. Хотя эти две операции схожи, их отличие в том, как они обрабатывают дубликаты.

- **UNION** объединяет результаты, исключая дублирующиеся строки.
- **UNION ALL** работает аналогично **UNION**, но не исключает дубликаты.



Таким образом, `UNION` (в отличие от `UNION ALL`) выполняет дедупликацию**.*



Дедупликация — исключение повторяющихся данных

Дедупликация — исключение повторяющихся данных

Дедупликация требует дополнительных ресурсов и времени на выполнение, поэтому `UNION ALL` работает быстрее, так как он просто объединяет результаты без удаления дубликатов.

Общий вид SQL-запроса с использованием UNION:

```
1 SELECT <поле_1>, <поле_2>, <поле_3>      -- запрос на выборку из таблицы 1
2 FROM table_1
3 UNION [ALL]                                -- инструкция для объединения
4 SELECT <поле_1>, <поле_2>, <поле_3>      -- запрос на выборку из таблицы 2
5 FROM table_2;
```

У операций `UNION` и `UNION ALL` есть ограничение: **количество полей** в объединяемых результатах запросов **должно быть одинаковым**. При этом названия полей могут различаться, так как итоговые названия полей берутся из первого запроса. Типы данных в соответствующих полях также могут отличаться, но рекомендуется объединять только поля с совместимыми типами данных.

Пример запроса с использованием UNION:

```
1 SELECT name
```



```
2 FROM locations
3
4 UNION ALL
5
6 SELECT name
7 FROM characters
8 ORDER BY 1;
```

```
1 SELECT name
2 FROM locations
3
4 UNION ALL
5
6 SELECT name
7 FROM characters
8 ORDER BY 1;
```

{{ }}



LIMIT 1000

StartDE Theory ▾



Table



name

26 Years Old Morty

40 Years Old Morty

7+7 Years Old Morty

80's snake

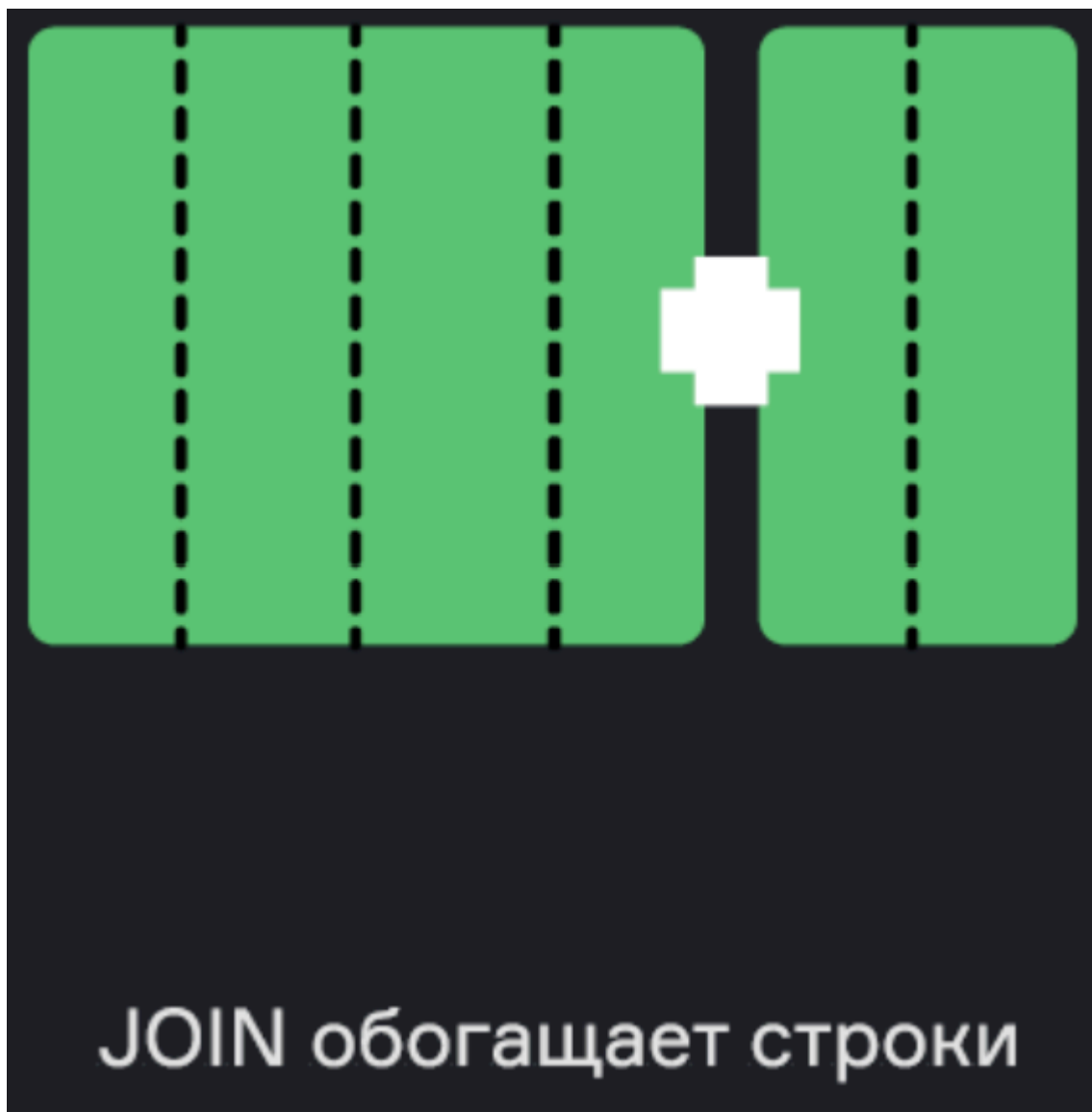
Когда использовать **UNION** и **UNION ALL** ?

- Используйте **UNION** , когда нужно устранить дубли в результатах запросов.
- Используйте **UNION ALL** , когда вам не важна дедупликация и необходимо повысить производительность запроса.

Важно: Если вы уверены, что данные в обоих запросах уникальны, используйте **UNION ALL** , так как это будет быстрее.

Объединение таблиц (JOIN)

Операция **JOIN** используется для обогащения данных из одной таблицы данными из другой. При этом каждой строке в одной таблице подбирается соответствие во второй таблице по некоторому условию, затем происходит объединение.



Таким условием, например, является равенство значений в определенном поле (или полях), когда во второй таблице ищутся записи, соответствующие значению поля из первой таблицы, после чего происходит их объединение в результирующую таблицу.

В отличие от **UNION**, где объединяются только строки, при **JOIN** результатом будет таблица с новыми столбцами, полученными из разных таблиц.

Общий вид SQL-запроса с использованием JOIN:

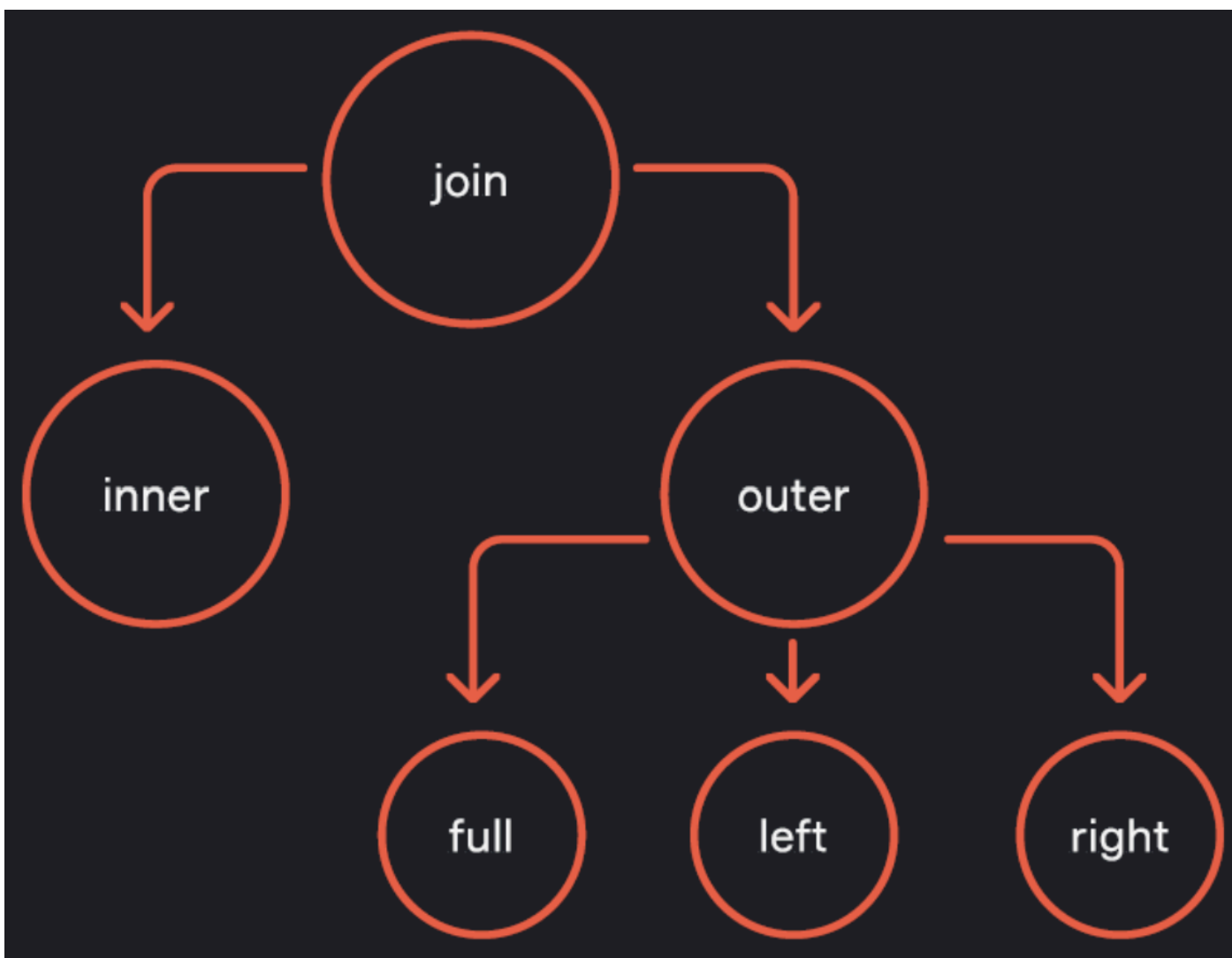
```
1 SELECT a.column1, b.column2
2 FROM table1 a
```

```
3 JOIN table2 b ON a.id = b.table2_id;
```

Этот запрос объединяет таблицы `table1` и `table2` по полю `id` из `table1` и полю `table2_id` из `table2`. Обратите внимание, что при объединении таблиц можно присваивать алиасы, чтобы не писать название таблиц. При этом допустимо не использовать инструкцию `AS`.

Основные виды JOIN

Существует несколько типов операций `JOIN`, каждая из которых имеет свои особенности и применяется в зависимости от задачи.



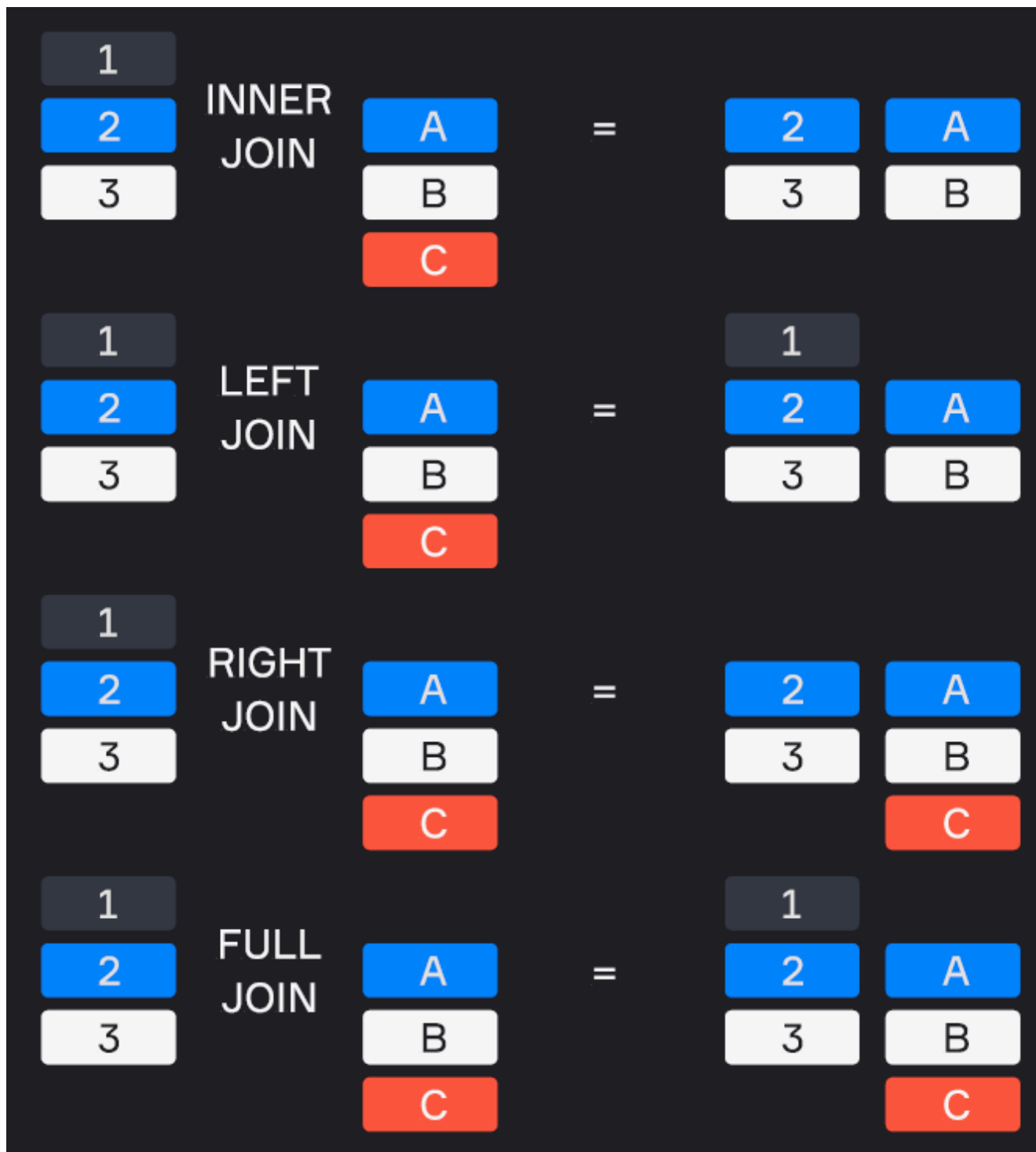
INNER JOIN возвращает только те строки, которые есть в обеих таблицах, то есть те, для которых найдено соответствие в обеих таблицах.

LEFT JOIN возвращает все строки из левой таблицы, и соответствующие строки из правой. Если в правой таблице нет соответствующих строк, будут возвращены `NULL` значения.

RIGHT JOIN возвращает все строки из правой таблицы, и соответствующие строки из левой. Если в левой таблице нет соответствующих строк, будут возвращены `NULL` значения.

FULL OUTER JOIN возвращает все строки из обеих таблиц, заполняя **NULL** там, где нет соответствующих строк.

В SQL при использовании оператора **JOIN** слово **INNER** можно опустить, поскольку по умолчанию выполняется именно внутреннее соединение (INNER JOIN). При использовании операторов **LEFT JOIN** и **RIGHT JOIN** слово **OUTER** также можно опустить. Это упрощает синтаксис и делает запросы более читаемыми, не изменяя их смысл.



В качестве примера возьмем таблицу сотрудников **employees** с полями **employee_id**, **name** и **department_id**. И таблицу департаментов **departments** с полями **department_id** и **department_name**.

employee_id	name	department_id
1	John	1
2	Jane	2
3	Mike	4
4	Anna	3
5	Steve	NULL
6	Bob	2

department_id	department_name
1	HR
2	Engineering
3	Sales

INNER JOIN (внутреннее соединение): возвращает строки, у которых есть соответствие в обеих таблицах.

Для того, чтобы найти только тех сотрудников, для которых в базе данных есть информация об их отделах, воспользуемся **INNER JOIN** :

▼

```
1 SELECT e.employee_id, e.name, d.department_name
2   FROM employees e
3  INNER JOIN departments d ON e.department_id = d.department_id;
```

В результат будут включены только те сотрудники, у которых есть соответствующий **department_id** в таблице **departments** . Сотрудники, у которых **department_id** равен **NULL** или не имеют соответствия в таблице **departments** , будут исключены из результата.

employee_id	name	department_name
1	John	HR

2	Jane	Engineering
4	Anna	Sales
6	Bob	Engineering

OUTER JOIN (внешнее соединение):

- **LEFT JOIN (LEFT OUTER JOIN):** возвращает все строки из левой таблицы и соответствующие строки из правой таблицы. Если соответствий нет, то вместо значений правой таблицы будут **NULL**.

Для того, чтобы обогатить таблицу сотрудников информацией о названиях отделов воспользуемся **LEFT JOIN**:

```

1 SELECT e.employee_id, e.name, d.department_name
2 FROM employees e
3 LEFT JOIN departments d ON e.department_id = d.department_id;
```

В результат будут включены все сотрудники из таблицы **employees**. Для сотрудников, у которых есть соответствие в таблице **departments**, будет проставлено соответствующее название департамента. Если такого соответствия нет, то **NULL**.

employee_id	name	department_name
1	John	HR
2	Jane	Engineering
3	Mike	NULL
4	Anna	Sales
5	Steve	NULL
6	Bob	Engineering

- **RIGHT JOIN (RIGHT OUTER JOIN):** возвращает все строки из правой таблицы и соответствующие строки из левой таблицы. Если соответствий нет, то вместо значений левой таблицы будут **NULL**.

Для того, чтобы обогатить таблицу отделов информацией о сотрудниках, воспользуемся **RIGHT**

JOIN :

```
1 SELECT e.employee_id, e.name, d.department_name
2 FROM employees e
3 RIGHT JOIN departments d ON e.department_id = d.department_id;
```

Запрос позволяет увидеть сотрудников, у которых указан департамент и он существует в таблице департаментов. Также он покажет, какие департаменты существуют, даже если в них нет сотрудников.

employee_id	name	department_name
1	John	HR
2	Jane	Engineering
4	Anna	Sales
6	Bob	Engineering

- **FULL JOIN (FULL OUTER JOIN)**: возвращает все строки из обеих таблиц, заполняя отсутствующие значения **NULL** .

```
1 SELECT e.employee_id, e.name, d.department_name
2 FROM employees e
3 FULL JOIN departments d ON e.department_id = d.department_id;
```

Запрос позволяет полностью объединить информацию о сотрудниках и департаментах. В данном случае результат запроса будет совпадать с результатом **LEFT JOIN** , так как все строки из левой таблицы (**employees**) присутствуют в объединении и все строки из правой таблицы (**departments**) имеют соответствия в левой таблице, так что дополнительные строки с **NULL** не добавляются в результат.

employee_id	name	department_name
1	John	HR

2	Jane	Engineering
3	Mike	NULL
4	Anna	Sales
5	Steve	NULL
6	Bob	Engineering

Еще один пример работы FULL JOIN:

id	name	description
1	Data Engineer	Обучение дата-инженеров
2	Data Analyst	Обучение дата-аналитиков
3	Start ML	Начальный курс по ML

id	course_id	number	start_date
1	1	31	2024-03-07
2	1	32	2024-04-01
3	1	33	2024-04-21



Соединение таблицы курсов и потоков:

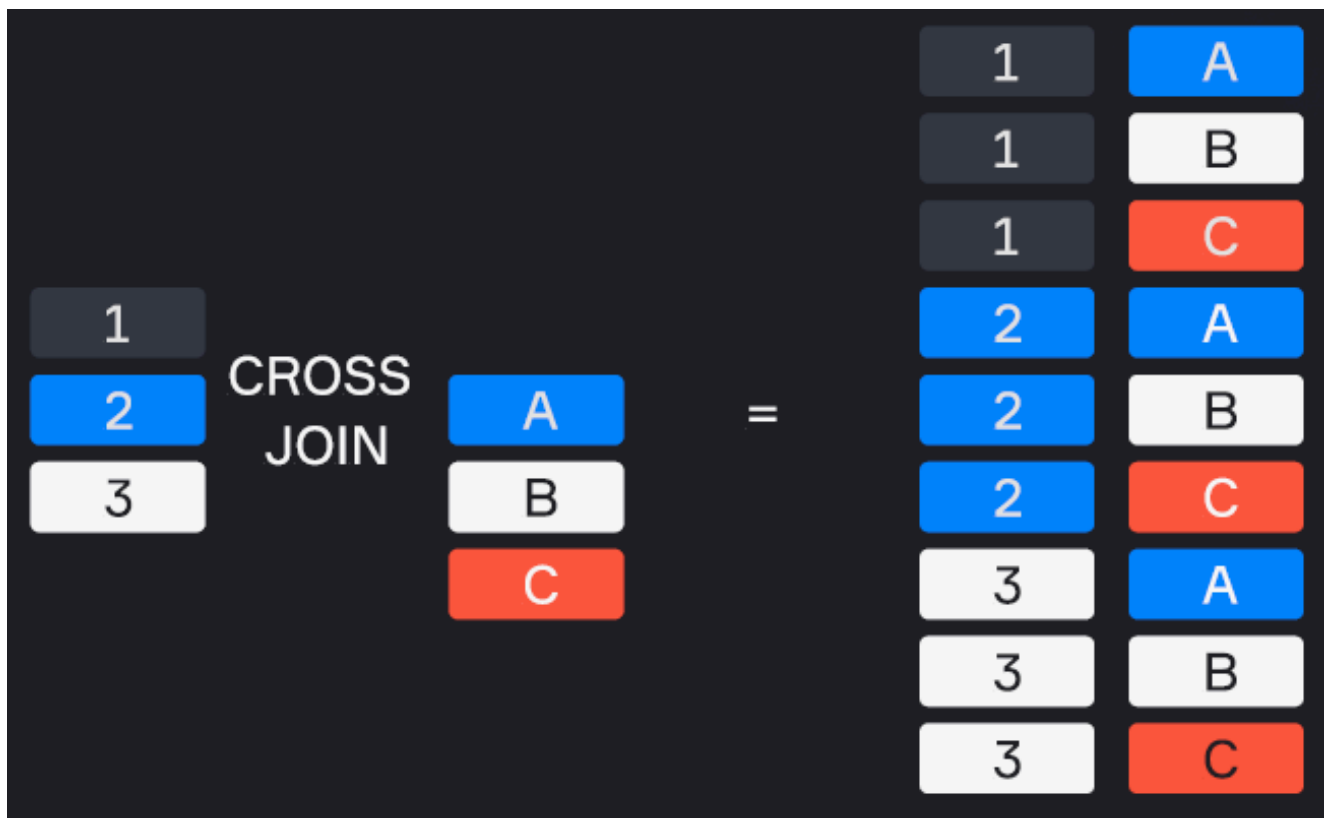

```
SELECT c.id AS course_id,  
       c.name AS course_name,  
       f.number AS flow_num,  
       f.start_date AS flow_start  
FROM courses AS c  
FULL JOIN flows AS f  
ON c.id = f.course_id
```

Таблица `courses` соединяется с таблицей `flows` с помощью `FULL JOIN`, при этом значения из таблицы `courses`, где отсутствуют соответствия, автоматически заполняются `NULL`.

course_id	course_name	flow_num	flow_start
1	Data Engineer	31	2024-03-07
1	Data Engineer	32	2024-04-01
1	Data Engineer	33	2024-04-21
2	Data Analyst	NULL	NULL
3	Start ML	NULL	NULL

Специальные виды JOIN

CROSS JOIN: соединение без условия, создающее декартово произведение двух таблиц. Он полезен в ситуациях, когда нужно сопоставить каждую строку одной таблицы с каждой строкой другой таблицы. Однако его следует использовать с осторожностью из-за возможного экспоненциального роста количества строк в результате, что может привести к значительному снижению производительности.



Пример работы CROSS JOIN:

```
1 SELECT c1.name AS name_1, c2.name AS name_2
2 FROM characters AS c1
3 CROSS JOIN characters AS c2
4 WHERE c1.name != c2.name;
```

```

1 SELECT c1.name AS name_1, c2.name AS name_2
2 FROM characters AS c1
3 CROSS JOIN characters AS c2
4 WHERE c1.name != c2.name;

```

{{ }}



LIMIT 1000

StartDE Theory ▾



Table



name_1

name_2

Rick Sanchez

Morty Smith

Rick Sanchez

Summer Smith

Rick Sanchez

Beth Smith

Rick Sanchez

Jerry Smith

В запросе выше мы выводим все возможные попарные вариации имен.

SELF JOIN: соединение таблицы с самой собой. Это полезно, когда необходимо сравнить строки одной и той же таблицы или получить данные, которые находятся в разных строках, но связаны между собой. Self Join часто используется для работы с иерархическими данными, например, для анализа отношений между сотрудниками и их начальниками в одной таблице.

Пример:



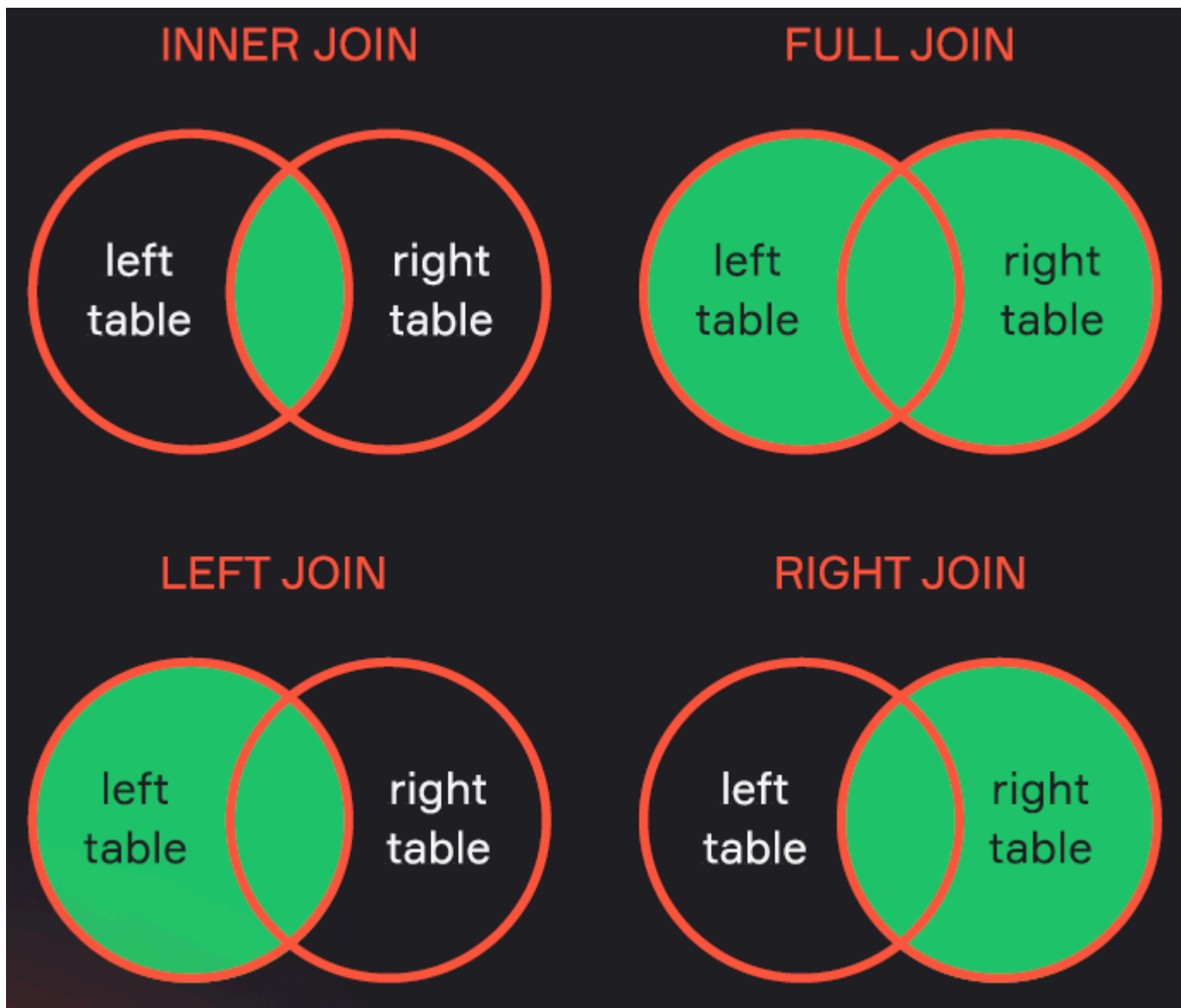
```

1 SELECT e1.name AS employee, e2.name AS manager
2 FROM employees e1
3 LEFT JOIN employees e2 ON e1.manager_id = e2.employee_id;

```

Этот запрос выполняет **SELF JOIN** на таблице `employees`, чтобы получить список сотрудников и их руководителей. Используется **LEFT JOIN**, чтобы для каждого сотрудника найти его руководителя по полям `manager_id` и `employee_id`. Если у сотрудника нет руководителя, в поле `manager` будет возвращено значение `NULL`.

Диаграммы Венна



Диаграммы Венна часто используются для объяснения различных типов JOINS в SQL, потому что они наглядно демонстрируют пересечения и объединения множеств.

Однако диаграммы Венна не всегда точно передают все аспекты и нюансы операций JOIN в контексте реляционных баз данных (например, если значения в таблице повторяются). Подробнее об этом можно почитать в статьях [Понимание джойнов сломано \(habr.com\)](https://habr.com/ru/articles/444444/) и [Понимание джойнов сломано. Продолжение \(habr.com\)](https://habr.com/ru/articles/444444/).

Вопросы на собеседовании

1. Как DISTINCT влияет на производительность запросов?

DISTINCT увеличивает время выполнения запроса, так как СУБД должна сравнивать каждое значение и запоминать уникальные строки, что приводит к дополнительным вычислениям и нагрузке на память.

2. Что будет, если выбрать DISTINCT * из таблицы?

Мы получим полную дедупликацию данных, то есть все строки, которые полностью идентичны по всем столбцам, будут исключены.

3. В чем разница между UNION и UNION ALL?

`UNION` удаляет дубликаты, а `UNION ALL` — нет. `UNION ALL` работает быстрее, так как не требует дедупликации, и его используют, если важно сохранить все строки, включая дубликаты.

4. Как можно объединить поля разных типов?

Можно использовать `CAST` или `CONVERT`, чтобы привести одно поле к нужному типу данных.

5. Какие виды JOIN вы знаете?

`INNER JOIN`, `LEFT JOIN`, `RIGHT JOIN`, `FULL JOIN`, `CROSS JOIN`, `SELF JOIN`. Эти типы соединений различаются по тому, какие строки из таблиц они включают в результат в зависимости от наличия соответствий.

6. В таблице t1 N-строк, в таблице t2 M-строк. Какой максимум строк мы можем получить при Join?

При `CROSS JOIN` мы можем получить $N * M$ строк, так как будет выполнено декартово произведение строк обеих таблиц. При других типах соединений, таких как `INNER JOIN`, количество строк зависит от того, сколько строк из обеих таблиц имеют совпадения.

7. Как мы можем выбрать все возможные пары сотрудников, работающих в одном отделе?

Нужно выполнить `SELF JOIN` на таблице сотрудников, используя условие, что два сотрудника работают в одном отделе, и затем исключить случаи, когда сотрудник соединяется сам с собой.

8. Как сдвоить таблицы по полям с разными типами данных?

Нужно привести одно из полей к нужному типу с помощью `CAST`, `::` или `CONVERT`, чтобы типы данных стали совместимыми для сравнения в запросе.

9. Что произойдет, если использовать условие WHERE в CROSS JOIN?

Сначала выполнится `CROSS JOIN`, который создаст все возможные комбинации строк из обеих таблиц, а затем фильтрация с условием `WHERE` ограничит результат. Это может занять больше времени и ресурсов, так как сначала создаётся декартово произведение всех строк.

Резюме

- Инструкция `DISTINCT` используется для удаления дублирующихся строк из результата запроса. Это помогает обеспечить уникальность данных в выводе.
- Операции `UNION` и `UNION ALL` используются для объединения результатов двух запросов:

- **UNION** объединяет результаты, исключая дублирующиеся строки. Применяется, когда важна уникальность данных.
- **UNION ALL** работает аналогично **UNION**, но не исключает дубликаты, что делает эту операцию быстрее, особенно при больших объёмах данных.
- Операция **JOIN** используется для обогащения данных из одной таблицы данными из другой. При этом каждой строке в одной таблице ищется соответствие во второй таблице по некоторому условию или без него. Различают следующие виды соединений:
 - **INNER JOIN** — возвращает строки, которые есть в обеих таблицах.
 - **LEFT JOIN** (или **LEFT OUTER JOIN**) — возвращает все строки из левой таблицы и соответствующие строки из правой, если таковые есть.
 - **RIGHT JOIN** (или **RIGHT OUTER JOIN**) — аналогично **LEFT JOIN**, но с приоритетом для правой таблицы.
 - **FULL JOIN** (или **FULL OUTER JOIN**) — возвращает все строки из обеих таблиц, с заполнением отсутствующих данных значением **NULL**.
 - **CROSS JOIN** — выполняет декартово произведение строк обеих таблиц.
- SELF JOIN** ****— соединение таблицы с самой собой. Это полезно, когда необходимо сравнить строки одной и той же таблицы или получить данные, которые находятся в разных строках, но связаны между собой.

К концу текущего урока вы можете написать SQL-запрос с использованием инструкций:

SELECT	Основной оператор для запроса данных из одной или нескольких таблиц. Позволяет выбрать конкретные столбцы и строки на основе заданных условий.
DISTINCT	Используется вместе с SELECT для удаления дублирующихся строк из результата запроса
FROM	Указывает на таблицу в базе данных, к которой обращается запрос
JOIN	Соединяет строки из двух или более таблиц на основе связанного столбца между ними. Различные типы соединений включают: INNER JOIN, LEFT JOIN (или LEFT OUTER JOIN), RIGHT JOIN (или RIGHT OUTER JOIN), FULL JOIN (или FULL OUTER JOIN), CROSS JOIN
WHERE	Указывает условия, по которым строки будут выбраны
ORDER BY	Упорядочивает строки в результате запроса по одному или нескольким столбцам. Может быть использован для сортировки по возрастанию (ASC) или убыванию (DESC)
LIMIT и OFFSET	LIMIT ограничивает количество возвращаемых строк. OFFSET пропускает указанное количество строк перед возвратом оставшихся строк

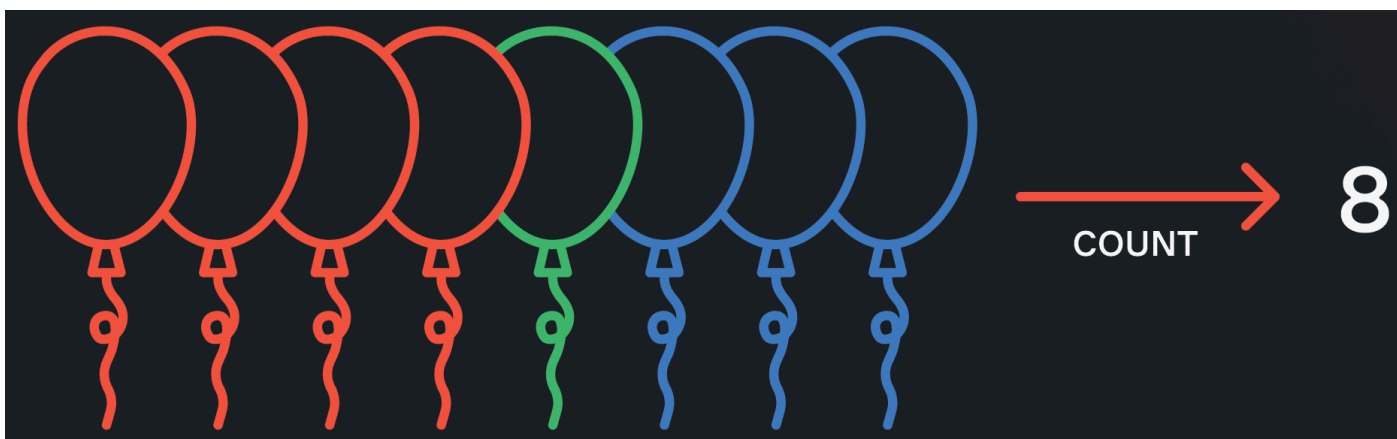
UNION и UNION ALL	Объединяют результаты двух или более SELECT запросов. UNION удаляет дубликаты, а UNION ALL сохраняет все строки, включая дубликаты
----------------------	--



Урок 5. Агрегация

1. Агрегатные функции
2. Группировка данных
3. Фильтрация по агрегатам
4. Вопросы на собеседовании
5. Резюме
6. Практика: Техническое собеседование

Агрегатные функции



Агрегатные функции выполняют вычисление над набором значений и возвращают одиночное значение, которое называют агрегатом. Они используются для анализа и обобщения данных, группируя их по определенным критериям и возвращая статистические значения. Например, мы можем посчитать сумму всех покупок за сегодня, количество человек в департаменте или средний рост населения.

Основные агрегатные функции включают:

Количество	COUNT(<поле>)	SELECT COUNT(*) FROM products
Сумма	SUM(<поле>)	SELECT SUM(final_price) FROM sales
Среднее	AVG(<поле>)	SELECT AVG(discount) FROM prices
Минимум	MIN(<поле>)	SELECT MIN(weight) FROM students
Максимум	MAX(<поле>)	SELECT MAX(weight) FROM students

Пример работы агрегатных функций:

▼

```

1 SELECT SUM(id), AVG(id), MIN(id), MAX(id)
2 FROM locations;

```

1 SELECT SUM(id), AVG(id), MIN(id), MAX(id)

2 FROM locations;

{{ }}

☰

⚡

☒ LIMIT 1000

StartDE Theory ▼

💾*

▶

Table +

sum	avg	min	max
8,001	63.50	1	126

Особенности функции COUNT

Функция COUNT имеет несколько вариаций:

- **COUNT(*)** возвращает количество строк в таблице (или в отфильтрованной таблице, если используется инструкция **WHERE**);
- **COUNT(1)** аналогично **COUNT(*)** , но СУБД не читает значения каждого поля, а использует константу;
- **COUNT(<поле>)** подсчитывает количество непустых значений в указанном поле;
- **COUNT(DISTINCT <поле>)** возвращает количество уникальных непустых значений в указанном поле.

Пример работы функции **COUNT** :

```
1 SELECT COUNT(*) AS count_,
2     COUNT(1) AS count_1,
3     COUNT(dimension) AS count_dimension,
4     COUNT(distinct type) AS count_dist_type
5 FROM locations;
```

```
1 SELECT COUNT(*) AS count_,
2     COUNT(1) AS count_1,
3     COUNT(dimension) AS count_dimension,
4     COUNT(distinct type) AS count_dist_type
5 FROM locations;
```

{{ }}



LIMIT 1000

StartDE Theory



Table



count_

count_1

count_dimension

count_dist_type

126

126

95

44

Пример подсчета количества значений в колонке с использованием **COUNT** , **SUM** и **CASE** :

```
1 SELECT COUNT(1) AS row_cnt,
2     SUM(CASE WHEN gender = 'Male' THEN 1 ELSE 0 END) AS male_cnt,
3     SUM(CASE WHEN gender = 'Female' THEN 1 ELSE 0 END) AS female_cnt
4 FROM characters
5 WHERE status = 'Alive';
```

```

1 SELECT COUNT(1) AS row_cnt,
2       SUM(CASE WHEN gender = 'Male' THEN 1 ELSE 0 END) AS male_cnt,
3       SUM(CASE WHEN gender = 'Female' THEN 1 ELSE 0 END) AS female_cnt
4 FROM characters
5 WHERE status = 'Alive';

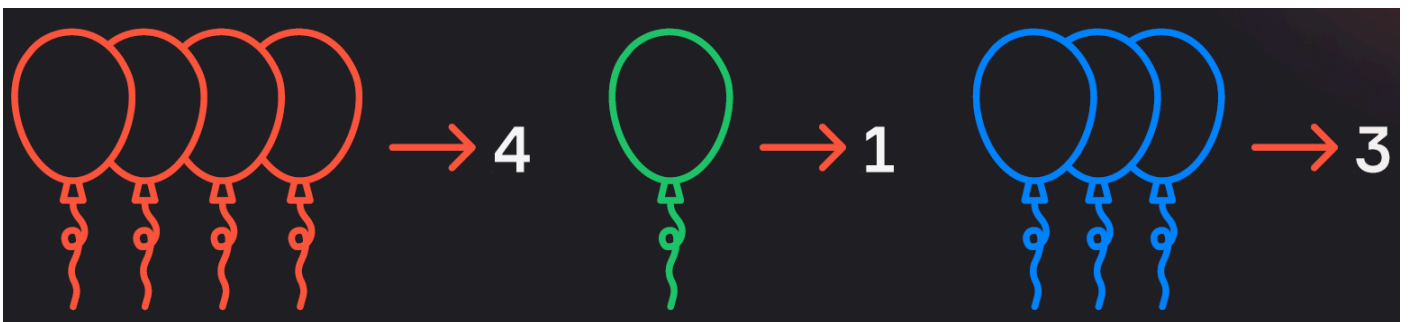
```

{{ }}
☰
⚡
☒ LIMIT 1000
 StartDE Theory
 ▼
💾*
▶

Table	+		
row_cnt	male_cnt	female_cnt	
439	309	95	

В запросе выше поле с количеством строк получаем с помощью `COUNT(1)`, а подсчет по гендерному признаку мы осуществляем с помощью суммирования результата работы функций `CASE`. В случае соответствия нужному нам гендеру мы проставляем 1, а в противном случае — 0. Если просуммировать эти единички, мы получим количество мужских и женских персонажей.

Группировка данных



Для вычисления агрегатов по наборам данных используется инструкция `GROUP BY`. Она позволяет задать поле группировки и выполнять агрегирующие функции в рамках каждой группы.

Общий вид запроса с группировкой:

```

1 SELECT <поле группировки>, <агрегат>
2 FROM table
3 GROUP BY <поле группировки>;

```

Пример использования группировки и агрегации

Нам необходимо посчитать количество потоков для каждого курса:

id	name	description
1	Data Engineer	Обучение дата-инженеров
2	Data Analyst	Обучение дата-аналитиков
3	Start ML	Начальный курс по ML



id	course_id	number	start_date
1	1	31	2024-03-07
2	1	32	2024-04-01
3	1	33	2024-04-21

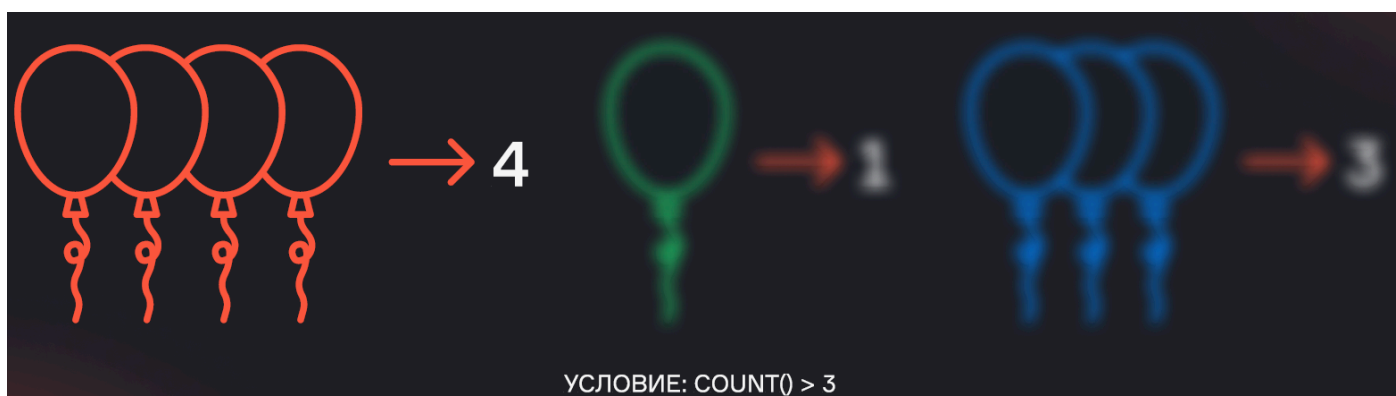
Для этого необходимо написать SQL-запрос, соединяющий таблицы курсов и потоков, а также агрегирующий поле «потоки» для каждого из курсов:

```
SELECT c.id AS course_id,  
       c.name AS course_name,  
       COUNT(f.id) AS flow_cnt  
FROM courses AS c  
FULL JOIN flows AS f  
  ON c.id = f.course_id  
GROUP BY course_id, course_name
```

В запросе мы объединяем две таблицы, группируем данные по `course_id` и `course_name` и считаем количество потоков для каждого курса. В результате получаем нужную нам таблицу:

course_id	course_name	flow_cnt
1	Data Engineer	3
2	Data Analyst	0
3	Start ML	0

Фильтрация по агрегатам



Иногда требуется отфильтровать данные после их агрегации. Для этого используется инструкция **HAVING**. Она позволяет задать условия для агрегированных данных.

Общий вид запроса с группировкой и фильтрацией по агрегату:

```

1 SELECT <поле группировки>, <агрегат>
2   FROM table
3  GROUP BY <поле группировки>
4  HAVING <условие на агрегат>

```

Пример запроса с фильтрацией по агрегату

Нам необходимо найти курсы, для которых есть хотя бы один поток:

id	name	description
1	Data Engineer	Обучение дата-инженеров
2	Data Analyst	Обучение дата-аналитиков
3	Start ML	Начальный курс по ML



id	course_id	number	start_date
1	1	31	2024-03-07
2	1	32	2024-04-01
3	1	33	2024-04-21

Для этого необходимо написать SQL-запрос, соединяющий таблицы курсов и потоков, а также агрегирующий поле «потоки» для каждого из курсов, применив условие фильтрации по агрегированному значению:

```
SELECT c.id AS course_id,
       c.name AS course_name,
       COUNT(f.id) AS flow_cnt
FROM courses AS c
FULL JOIN flows AS f
  ON c.id = f.course_id
GROUP BY course_id, course_name
HAVING COUNT(f.id) > 0
```

В запросе мы объединяем две таблицы, группируем данные по `course_id` и `course_name`, считаем количество потоков для каждого курса и затем фильтруем это количество. В результате получаем нужную нам таблицу:

course_id	course_name	flow_cnt
1	Data Engineer	3

Этот запрос возвращает только те курсы, для которых есть хотя бы один поток.

Пример запроса с фильтрацией по агрегату:

```

1 SELECT char.name, COUNT(ep.name) AS ep_cnt
2 FROM characters AS char
3 FULL JOIN char_ep AS c_e
4     ON char.id = c_e.character_id
5 FULL JOIN episodes AS ep
6     ON ep.id = c_e.episode_id
7 GROUP BY char.name
8 HAVING COUNT(ep.name) > 10
9 ORDER BY ep_cnt DESC;

```

```

1 SELECT char.name, COUNT(ep.name) AS ep_cnt
2 FROM characters AS char
3 FULL JOIN char_ep AS c_e
4     ON char.id = c_e.character_id
5 FULL JOIN episodes AS ep
6     ON ep.id = c_e.episode_id
7 GROUP BY char.name
8 HAVING COUNT(ep.name) > 10
9 ORDER BY ep_cnt DESC;

```

{{ }}
☰
⚡
☒ LIMIT 1000
StartDE Theory ▾
💾*
▶

Table +

name	ep_cnt
Morty Smith	54
Rick Sanchez	54
Beth Smith	52
Summer Smith	51

Фильтрацию данных по неагрегированным значениям можно делать как в блоке WHERE, так и в блоке HAVING. Например, результат следующих запросов будет одинаковый:

```
1 SELECT sex, COUNT(user_id)
2 FROM users
3 WHERE sex != 'male'
4 GROUP BY sex
```

```
1 SELECT sex, COUNT(user_id)
2 FROM users
3 GROUP BY sex
4 HAVING sex != 'male'
```

Однако делать фильтрацию по неагрегированным данным рекомендуется именно в блоке WHERE, т.е. заранее. В таком случае ненужные данные убираются из расчётов ещё до группировки, и не расходуются вычислительные ресурсы на подсчёт значений, которые всё равно будут отфильтрованы вами позже.

Вопросы на собеседовании

- **Как посчитать количество сотрудников, работающих в конкретном департаменте?**

Мы считаем `COUNT(1)` из таблицы, фильтруя с помощью `WHERE` департамент по названию или ID.

- **Как одним запросом посчитать общее количество строк, количество мужчин и количество женщин в таблице?**

Используя `COUNT(1)`, а также подменяя признак гендера мужчин единицей в одном поле и признак гендера женщин единицей в другом поле, а потом суммируя эти строки.

- **Как посмотреть дубли с помощью группировки?**

Мы можем сгруппировать данные по нужным нам полям, посчитав количество строк по группам, затем отфильтровав в `HAVING` группы, содержащие более одной строки, то есть дубликаты.

- **Чем отличается `WHERE` от `HAVING` ?**

- `WHERE` фильтрует изначальные строки до их группировки
- `HAVING` фильтрует агрегированные данные после выполнения группировки

Последовательность выполнения: сначала **WHERE** , затем **GROUP BY** , и в конце **HAVING** .

Резюме

- Для вычислений над набором значений используются **агрегатные функции** (**SUM** , **MIN** , **MAX** , **AVG** , **COUNT**). С помощью функции **COUNT** можно подсчитать количество строк в таблице, количество непустых значений в поле, а также количество уникальных непустых значений поля.
- Для применения агрегатных функций к группам строк, а не ко всему полю, используется инструкция **GROUP BY** . Она позволяет задать поля группировки, объединив строки, имеющие одинаковые значения в группы, и выполнять агрегирующие функции в рамках каждой группы.
- Для фильтрации результата вычисления агрегатных функций после группировки используется инструкция **HAVING** . При этом сначала выполняется фильтрация в **WHERE** , затем **GROUP BY** , а в конце фильтрация в **HAVING** .

К концу текущего урока вы можете написать SQL-запрос с использованием инструкций:

SELECT	Основной оператор для запроса данных из одной или нескольких таблиц. Позволяет выбрать конкретные столбцы и строки на основе заданных условий
DISTINCT	Используется вместе с SELECT для удаления дублирующихся строк из результата запроса
FROM	Указывает на таблицу (таблицы) в базе данных, к которой обращается запрос
JOIN	Соединяет строки из двух или более таблиц на основе связанного столбца между ними. Различные типы соединений включают: INNER JOIN, LEFT JOIN (или LEFT OUTER JOIN), RIGHT JOIN (или RIGHT OUTER JOIN), FULL JOIN (или FULL OUTER JOIN), CROSS JOIN
WHERE	Указывает условия, по которым строки будут выбраны
GROUP BY	Группирует строки с одинаковыми значениями в указанных столбцах и позволяет выполнять агрегатные функции на каждой группе
HAVING	Используется для фильтрации групп, созданных с помощью GROUP BY, на основе заданных условий
ORDER BY	Упорядочивает строки в результате запроса по одному или нескольким столбцам. Может быть использован для сортировки по возрастанию (ASC) или убыванию (DESC)
LIMIT и OFFSET	LIMIT ограничивает количество возвращаемых строк. OFFSET пропускает указанное количество строк перед возвратом оставшихся строк

UNION и UNION ALL	Объединяют результаты двух или более SELECT запросов. UNION удаляет дубликаты, а UNION ALL сохраняет все строки, включая дубликаты
-------------------	--

Практика: Техническое собеседование

В рамках этого задания вам предстоит выполнить несколько SQL-запросов, используя базу данных **Project**, которая содержит две таблицы:

1. **Employees** — таблица сотрудников с тремя полями:

- `employee_id` — ID сотрудника,
- `employee_name` — имя сотрудника,
- `manager_id` — ID менеджера сотрудника.

2. **Projects** — таблица проектов с четырьмя полями:

- `project_id` — ID проекта,
- `employee_id` — ID сотрудника, работающего над проектом,
- `project_name` — название проекта,
- `hours_spent` — количество часов, которые сотрудник потратил на проект.

Задача 1: Иерархия сотрудников

Напишите запрос, который выведет всех сотрудников вместе с их непосредственными руководителями. Также нужно учесть самого верхнего руководителя, у которого нет руководителя в структуре.

Результат запроса:

- `employee_id` — ID сотрудника,
- `employee_name` — имя сотрудника,
- `manager_name` — имя руководителя сотрудника.

Требования:

- Используйте `JOIN` для соединения таблицы `Employees` с самой собой.
- Учтите, что у самого верхнего руководителя (кто не имеет менеджера) поле `manager_name` должно быть `NULL`.

Задача 2: Общая информация о проектах

Напишите запрос, который выводит общую информацию о каждом проекте. Для каждого проекта выведите:

- название проекта,
- количество часов, которые каждый сотрудник работал над этим проектом,
- сумму часов, которые все сотрудники потратили на проект,
- количество уникальных сотрудников, которые работали над проектом.

Результат запроса:

- `project_name` — название проекта,
- `employee_name` — имя сотрудника,
- `hours_spent` — количество часов, потраченных сотрудником,
- `total_hours` — сумма всех часов, потраченных на проект,
- `unique_employees` — количество уникальных сотрудников, работающих над проектом.

Требования:

- Используйте `JOIN` для объединения таблиц `Employees` и `Projects`.
- Для получения количества уникальных сотрудников используйте агрегатную функцию `COUNT(DISTINCT)`.

Задача 3: Сотрудники с наибольшей нагрузкой

Напишите запрос, который выводит имена трех сотрудников с наибольшим количеством часов работы над различными проектами. Сотрудники должны быть отсортированы по убыванию времени, которое они потратили, и нужно выбрать только трех первых.

Результат запроса:

- `employee_name` — имя сотрудника,
- `total_hours` — общее количество часов, которое сотрудник потратил на проекты.

Требования:

- Используйте `JOIN` для объединения таблиц `Employees` и `Projects`.
- Для подсчёта часов используйте функцию `SUM()`.

- Используйте **LIMIT 3**, чтобы выбрать только трех сотрудников с наибольшим количеством часов.
-

Задача 4: Сотрудники, работающие только над одним проектом

Напишите запрос, который выводит имена сотрудников, которые работали только над одним проектом.

Результат запроса:

- **employee_name** — имя сотрудника.

Требования:

- **Не используйте JOIN**. Вам нужно найти решение без использования соединений таблиц.
 - Используйте подзапрос, чтобы подсчитать количество проектов для каждого сотрудника.
-

Задача 5: Вовлеченность сотрудника в проект

Напишите запрос, который для каждого проекта выводит три поля:

- название проекта,
- имя сотрудника,
- процент времени, который сотрудник провёл на проекте относительно всего времени, потраченного всеми сотрудниками над проектом.

Результат запроса:

- **project_name** — название проекта,
- **employee_name** — имя сотрудника,
- **participation_percentage** — процент времени, который сотрудник потратил на проект.

Требования:

- Используйте **JOIN** для объединения таблиц **Employees** и **Projects**.
 - Для подсчета процентов используйте арифметические операции.
 - Не забудьте вычислить общее количество часов, потраченных всеми сотрудниками над проектом.
-

Инструкции для выполнения:

1. Все запросы необходимо выполнить в базе данных **Project**.

2. Используйте SQL-операторы, такие как `SELECT` , `JOIN` , `GROUP BY` , `HAVING` , `SUM()` , `COUNT(DISTINCT)` , `ORDER BY` , и `LIMIT` .
3. Каждый запрос должен быть написан с учётом оптимизации производительности, особенно при работе с большим количеством данных.
4. После выполнения задания протестируйте запросы на небольшом объеме данных, чтобы убедиться в корректности результатов.



Урок 6. Подзапросы

1. Что такое подзапрос?
2. Подзапросы в FROM
3. Подзапросы в JOIN
4. Подзапросы в SELECT
5. Подзапросы в WHERE
6. Подзапросы в HAVING
7. Подзапросы в ORDER BY
8. Common Table Expression (CTE)
9. Вопросы на собеседованиях
10. Резюме

Урок 6. Подзапросы

Что такое подзапрос?

Подзапрос — это запрос внутри другого запроса. Например, мы можем посчитать агрегат по одной таблице, а затем использовать полученные данные в другом запросе к другой таблице. Подзапросы могут использоваться в различных частях основного запроса, таких как `SELECT` , `FROM` , `JOIN` , `WHERE` , `HAVING` и `ORDER BY` .

Типы подзапросов

1. **Некоррелированные подзапросы** — это подзапросы, которые могут быть выполнены независимо от внешнего запроса. Такой подзапрос выполняется один раз, и его результат используется во внешнем запросе.

Предположим, у нас есть таблица `orders` (заказы) и таблица `customers` (клиенты). Таблица `orders` содержит информацию о заказах, включая ID заказа, ID клиента и сумму заказа. Таблица `customers` содержит информацию о клиентах, включая ID клиента и имя клиента.

```
1 SELECT name
2   FROM customers
3  WHERE customer_id IN (
4      SELECT customer_id
5        FROM orders
6      WHERE order_amount > 1000
7 );
```

Приведенный выше запрос выбирает имена клиентов, которые сделали заказы на сумму более 1000. Подзапрос `SELECT customer_id FROM orders WHERE order_amount > 1000` выполняется один раз и возвращает список ID клиентов, которые затем используются во внешнем запросе.

2. **Коррелированные подзапросы** — это подзапросы, которые обращаются к внешней таблице, то есть они зависят от внешнего запроса. Для каждой строки во внешнем запросе подзапрос возвращает результат, зависящий от значения этой строки. Таким образом, коррелированный подзапрос выполняется многократно, по одному разу для каждой строки внешнего запроса, что влияет на производительность.

Предположим, у нас есть две таблицы: `employees` (сотрудники) и `departments` (отделы). Таблица `employees` содержит информацию о сотрудниках, включая их ID, имя, зарплату и ID отдела. Таблица `departments` содержит информацию об отделах, включая ID отдела и имя отдела.

```
1 SELECT e1.name, e1.salary
2   FROM employees e1
3  WHERE e1.salary > (
4      SELECT AVG(e2.salary)
5        FROM employees e2
6      WHERE e2.department_id = e1.department_id
7 );
```

Приведенный выше запрос выбирает сотрудников, чья зарплата выше средней зарплаты в их отделе. Подзапрос внутри `WHERE` выполняется для каждой строки из внешнего запроса

```
employees e1 .
```

Также в отдельную группу выделяют **скалярные запросы**. Это запросы, которые возвращают единственное значение. Например, `SELECT AVG(column) FROM table;` .

Подзапросы в FROM

```
1 SELECT *
2 FROM (SELECT * FROM table) AS t1;
```

Это некоррелированный подзапрос, в котором внешний `SELECT` получает данные из внутреннего подзапроса.



Некоторые СУБД требуют, чтобы у подзапросов, которые используются во `FROM` , были алиасы. Использование алиасов — полезная привычка!

Пример подзапроса в `FROM` :

```
1 SELECT substr(episode_id, 1, 3) AS season, COUNT(DISTINCT name) AS char_cnt
2 FROM
3   (SELECT e.episode_id, c.name
4     FROM episodes AS e
5     JOIN char_ep AS ce
6       ON e.id = ce.episode_id
7     JOIN characters AS c
8       ON ce.character_id = c.id
9     ) AS epis_char
10 GROUP BY substr(episode_id, 1, 3)
11 ORDER BY 1
```


Table + Add Visualization	
season	char_cnt
S01	188
S02	167
S03	195
S04	177
S05	162

Подзапросы в JOIN

Подзапросы в SQL могут быть использованы в различных частях запроса, включая секцию **JOIN**. Это позволяет объединить таблицу с результатами подзапроса, что полезно в случаях, когда необходимо выполнить агрегацию или фильтрацию данных до того, как они будут объединены с основной таблицей. Подзапросы в **JOIN** — один из самых часто встречающихся способов оптимизации запросов, особенно когда требуется работать с большими объемами данных или объединять таблицы с результатами агрегации.

```
1 SELECT *
2 FROM table1 AS t1
3 JOIN (SELECT * FROM table2) AS t2
4 ON t1.id = t2.id;
```

В этом примере подзапрос внутри **JOIN** выбирает все данные из таблицы **table2**, а затем объединяет их с таблицей **table1** по полю **id**. Это полезно, когда необходимо ограничить результаты из второй таблицы перед объединением.

Усложним предыдущий запрос и добавим к нему подзапрос в **JOIN**:

```
1 SELECT epis_char.season,
2        COUNT(DISTINCT ep.episode_id) AS ep_cnt,
3        COUNT(DISTINCT epis_char.name) AS char_cnt
4 FROM
5     (SELECT substr(e.episode_id, 1, 3) AS season, c.name
6      FROM episodes AS e
```

```

7     JOIN char_ep AS ce
8         ON e.id = ce.episode_id
9     JOIN characters AS c
10        ON ce.character_id = c.id
11 ) AS epis_char
12 JOIN episodes AS ep ON substr(ep.episode_id, 1, 3) = epis_char.season
13 GROUP BY epis_char.season
14 ORDER BY 1

```

Этот запрос сложнее, так как подзапрос сначала выполняет объединение таблиц `episodes`, `char_ep` и `characters` для получения данных о персонажах и эпизодах, а затем результат этого подзапроса объединяется с основной таблицей `episodes` по сезону (первым трем символам из `episode_id`). В результате мы получаем количество эпизодов и количество уникальных персонажей для каждого сезона.

Table
+ Add Visualization

season	ep_cnt	char_cnt
S01	11	188
S02	10	167
S03	10	195
S04	10	177
S05	10	162

Когда и зачем использовать подзапросы в `JOIN` ?

- Оптимизация больших данных:** Когда в одной из таблиц слишком много данных, и необходимо сначала выполнить фильтрацию, чтобы уменьшить объем данных, прежде чем делать объединение с основной таблицей. Например, если вам нужно выбрать 5% данных из огромной таблицы и затем объединить их с другой таблицей, подзапрос позволяет сделать это эффективнее.
- Объединение с агрегацией:** Часто подзапросы используются для предварительной агрегации данных. Например, если вы хотите объединить таблицу с результатами подсчета, подзапрос может сначала выполнить агрегирование, а затем результат объединяется с основной таблицей.
- Упрощение сложных запросов:** Подзапросы могут упрощать запросы, делая их более читаемыми и удобными для отладки. Это особенно важно при работе с комплексными операциями с несколькими объединениями или агрегированиями.

Типичные ошибки при использовании подзапросов в `JOIN` :

1. **Не присвоено имя подзапросу:** Это может привести к ошибкам в запросах, так как без имени подзапроса невозможно к нему обратиться.
2. **Неиспользование алиасов для полей:** Когда в подзапросах и основной таблице имеются одинаковые имена столбцов, нужно использовать алиасы для избегания путаницы.
3. **Ошибка в логике соединения:** Если неправильно указать условие соединения, это может привести к неправильному объединению и получению лишних данных, например, дубликатов.

Пример с оптимизацией:

Предположим, у нас есть таблица `locations` с тысячами записей. Мы можем использовать подзапрос, чтобы сначала отфильтровать данные по нужному измерению (например, `dimension = 'C137'`), а затем объединить результат с таблицей `characters`, чтобы получить информацию о персонажах в этом измерении.

```
1 SELECT c.name, l.name AS location_name
2   FROM characters AS c
3  JOIN (SELECT id, name FROM locations WHERE dimension = 'C137') AS l
4      ON c.origin_id = l.id;
```

Здесь подзапрос в JOIN сначала фильтрует локации по измерению C137, а затем объединяет их с персонажами, для которых это измерение является родным.

Подзапросы в SELECT

Некоррелированный подзапрос:

```
1 SELECT id, name, (SELECT avg(price) FROM table2) AS new_column
2   FROM table;
```

Выше мы используем подзапрос, чтобы добавить еще одно поле в основной запрос. Подзапрос должен быть скалярным, иначе СУБД не поймет, какое из значений, возвращенных подзапросом, необходимо использовать в конкретной строке, и вернет ошибку.



Некоррелированный подзапрос в `SELECT` должен быть скалярным.

Коррелированный подзапрос:

```
▼
1 SELECT name,
2       (SELECT COUNT(1)
3        FROM orders
4        WHERE orders.employee_id = employees.employee_id) AS order_cnt
5 FROM employees;
```

В запросе выше подзапрос использует данные из внешнего запроса для фильтрации, которая объединяет внутреннюю и внешнюю таблицы.

Пример подзапроса в **SELECT** :

```
▼
1 SELECT epis_char.season,
2       (SELECT COUNT(ep.episode_id)
3        FROM episodes AS ep
4        WHERE substr(ep.episode_id, 1, 3) = epis_char.season) AS ep_cnt,
5       (SELECT COUNT(1) FROM episodes) AS all_ep_cnt,
6       COUNT(DISTINCT epis_char.name) AS char_cnt
7 FROM
8       (SELECT substr(e.episode_id, 1, 3) AS season, c.name
9        FROM episodes AS e
10       JOIN char_ep AS ce
11        ON e.id = ce.episode_id
12       JOIN characters AS c
13        ON ce.character_id = c.id
14       ) AS epis_char
15 GROUP BY epis_char.season
16
```

Table

[+ Add Visualization](#)

season	ep_cnt	all_ep_cnt	char_cnt
S01	11	51	188
S02	10	51	167
S03	10	51	195
S04	10	51	177
S05	10	51	162

Подзапросы в WHERE

Некоррелированный подзапрос

```
1 SELECT *
2 FROM table1
3 WHERE price > (SELECT AVG(price) FROM table2);
```

В запросе выше производится фильтрация по значению-константе, возвращаемому скалярным подзапросом (выбираем из таблицы все товары, у которых цена выше средней).

Коррелированный подзапрос

```
1 SELECT *
2 FROM product_price AS pp
3 WHERE
4     pp.price = (
5         SELECT min(ppm.price)
6         FROM product_price AS ppm
7         WHERE ppm.product_id = pp.product_id
8     )
9 ORDER BY pp.product_id, pp.price, pp.store_id;
```

В запросе выше отбираются только записи с минимальной ценой для каждого продукта. Фильтрация осуществляется таким образом, чтобы цена во внешней таблице равнялась минимальной цене во

внутренней таблице при равенстве `product_id` .

Многоколоночный подзапрос

```
1 SELECT employee_id, name, department_id
2   FROM employees
3  WHERE (department_id, location) =
4         (SELECT department_id, location
5          FROM departments
6          WHERE department_id = 101);
```

Многоколоночный подзапрос в `WHERE` используется для сравнения двух и более полей с подзапросом. В примере выше мы сравниваем ID департамента и его местоположение с тем, что возвращает нам отфильтрованный подзапрос. Это пример некоррелированного подзапроса.

Инструкции в `WHERE` для подзапросов с одним полем и несколькими записями:

- **IN** — проверяет, содержится ли значение в результатах подзапроса. Возвращает `TRUE` , если значение поля входит в результат подзапроса, иначе `FALSE` .

```
1 SELECT *
2   FROM table1
3  WHERE id IN (SELECT id FROM table2);
```

Выше пример некоррелированного подзапроса. Во внешнем запросе производится фильтрация, которая оставляет только записи с `id` , входящими в результат подзапроса.

- **EXISTS** — проверяет наличие строк в подзапросе. Возвращает `TRUE` , если подзапрос возвращает хотя бы одну строку, иначе `FALSE` .

```
1 SELECT *
2   FROM table1
3  WHERE EXISTS (SELECT id
4                FROM table2
5                WHERE table2.name = table1.name);
```

Из таблицы 1 выбираются данные, для которых в таблице 2 есть `id` , у которых имя в таблице 2 равно имени в таблице 1. Другими словами, мы джойним таблицы и смотрим, есть ли во второй таблице хотя бы один нужный нам ID, и если есть — условие считается выполненным. `EXISTS` имеет смысл, когда запрос коррелированный.

- **ANY/SOME** — используется для сравнения значения с любым значением из подзапроса.

Возвращает **TRUE**, если условие выполняется хотя бы для одного значения в наборе, иначе **FALSE**.

```
1 SELECT employee_id, name, salary
2   FROM employees
3  WHERE salary > ANY (SELECT salary FROM employees WHERE department_id = 10);
```

В запросе выше мы фильтруем записи таким образом, чтобы зарплата была больше, чем хотя бы у одного из сотрудников департамента 10.

ANY и **SOME** — это совершенно одинаковые инструкции, их работа ничем не отличаются. Некоторые СУБД обрабатывают обе инструкции, а некоторые обрабатывают только одну из них — либо только **ANY**, либо только **SOME**.

- **ALL** — используется для сравнения значения со всеми значениями в подзапросе.

Возвращает **TRUE**, если условие выполняется для всех значений в наборе, иначе **FALSE**.

```
1 SELECT employee_id, name, salary
2   FROM employees
3  WHERE salary > ALL (SELECT salary FROM employees WHERE department_id = 10);
```

В запросе выше мы фильтруем записи таким образом, чтобы зарплата была больше, чем у каждого из сотрудников департамента 10.

Пример подзапроса в **WHERE**:

```
1 SELECT *
2   FROM characters
3  WHERE EXISTS (SELECT 1
4                  FROM char_loc
5                 WHERE location_id = (SELECT id FROM locations WHERE name =
   'Earth (C-137)')
6                  AND char_loc.character_id = characters.id);
```

id	name	status	species	type	gender	origin_id
38	Beth Smith	Alive	Human		Female	1
45	Bill	Alive	Human		Male	1
71	Conroy	Dead	Robot			20
82	Cronenberg Rick		Cronenberg		Male	12
83	Cronenberg Morty		Cronenberg		Male	12
92	Davin	Dead	Human		Male	1

Подзапросы в HAVING

При использовании подзапроса в блоке **HAVING** выбираются только те записи, агрегаты которых есть в подзапросе, или результат сравнения с подзапросом возвращает **TRUE** (подзапрос при этом должен возвращать только одно значение). Пример некоррелированного подзапроса в блоке **HAVING** :

```
1 SELECT id, SUM(price) AS sm
2   FROM table1
3  GROUP BY id
4 HAVING SUM(price) > (SELECT AVG(price) FROM table1);
```

Пример подзапроса в **HAVING** :

```
1 SELECT substr(episode_id, 1, 3) AS season,
2        COUNT(1) AS cnt
3   FROM episodes
4  GROUP BY substr(episode_id, 1, 3)
5 HAVING COUNT(1) < (SELECT COUNT(1) FROM episodes WHERE episode_id like
   'S01%');
```


Table

+ Add Visualization

season	cnt
S03	10
S04	10
S02	10
S05	10

Подзапросы в ORDER BY

Использование коррелированного подзапроса в блоке **ORDER BY** позволяет для каждой строки вычислить значение для сортировки.

```
1 SELECT employee_id, name, salary, department_id
2 FROM employees
3 ORDER BY (SELECT AVG(salary)
4           FROM employees AS e2
5           WHERE e2.department_id = employees.Department_id);
```

Запрос выше извлекает информацию о сотрудниках и сортирует результаты в зависимости от среднего значения зарплат по отделам, к которым эти сотрудники принадлежат. Подзапрос выполняется для каждой строки основного запроса и вычисляет среднюю зарплату (**AVG(salary)**) для всех сотрудников, работающих в том же отделе, что и текущий сотрудник в строке основного запроса.



Использование некоррелированных подзапросов в блоке **ORDER BY** не имеет практического смысла, так как сортировка по константе не изменит порядок строк, итоговый порядок будет определяться другими факторами, такими как порядок появления строк в исходной таблице, если он определен, или же порядок, в котором строки возвращает сервер базы данных.

Пример подзапроса в **ORDER BY** :

```
1 SELECT *
2 FROM characters
```

```

3 ORDER BY (
4     SELECT COUNT(1)
5     FROM char_ep
6     WHERE char_ep.character_id = characters.id
7 ) DESC;

```

Table

+ Add Visualization

id	name	status	species	type	gender	origin_id
1	Rick Sanchez	Alive	Human		Male	1
2	Morty Smith	Alive	Human		Male	
4	Beth Smith	Alive	Human		Female	20
3	Summer Smith	Alive	Human		Female	20
5	Jerry Smith	Alive	Human		Male	20
180	Jessica	Alive	Human	Time God	Female	20

Common Table Expression (CTE)

CTE (Common Table Expression) позволяет вынести подзапрос за пределы основного запроса. Он объявляется с помощью ключевого слова **WITH** и может использоваться несколько раз, но только в рамках одного общего запроса.

```

1 WITH cte_name AS (
2     SELECT id, name, count FROM table
3 )
4 SELECT *
5 FROM table1
6 JOIN cte ON table1.id = cte.id;

```

В примере выше после инструкции **WITH** задается название CTE, которое должно быть осмысленным для понимания содержимого запроса. Внутри CTE помещается запрос, результаты которого в дальнейшем могут быть использованы во **FROM**, **JOIN** и других операторах.

Преимущества CTE

1. **Улучшение читаемости:** Разделение сложного запроса на более понятные части.

2. **Оптимизация:** Возможность использовать результаты CTE несколько раз без повторного выполнения подзапроса.

Пример оптимизации с CTE

```
1 WITH cte AS (  
2     SELECT city, country, date, weather FROM table  
3 )  
4 SELECT weather  
5     FROM cte  
6 WHERE country = 'Russia'  
7 UNION  
8 SELECT weather  
9     FROM cte  
10 WHERE country = 'Peru';
```

В примере выше подзапрос выполняется СУБД один раз, а используется дважды. Таким образом уменьшается количество ресурсов для выполнения запроса.

CTE может быть несколько в одном запросе, в таком случае инструкция **WITH** пишется только один раз, а CTE перечисляются через запятую.

```
1 WITH cte AS (  
2     SELECT id, name, count FROM table  
3 ),  
4 cte2 AS (  
5     SELECT id, name, SUM(count) FROM cte GROUP BY id, name  
6 )  
7 SELECT *  
8     FROM table1  
9     JOIN cte2 ON table1.id = cte2.id;
```

В примере выше второй CTE обращается к первому. Это достаточно лаконично и позволяет понять, что происходит в первом CTE, что происходит во втором CTE (который использует результаты первого) и что происходит в самом **SELECT**, который построен на основе второго CTE.

Разница между CTE и временными таблицами

- **Временная таблица** создается пользователем и существует до завершения сессии (завершения соединения с СУБД). Пользователь может пользоваться временной таблицей все время, которое он проводит в сессии.

- **СТЕ** существует только в пределах выполнения запроса. Пользователь может использовать его внутри запроса много раз, но как только запрос вернул нам данные – СТЕ удаляется из памяти СУБД.

Пример с использованием СТЕ:

```
1 WITH epis_char AS
2   (SELECT substr(e.episode_id, 1, 3) AS season, c.name
3     FROM episodes AS e
4     JOIN char_ep AS ce
5       ON e.id = ce.episode_id
6     JOIN characters AS c
7       ON ce.character_id = c.id
8   )
9   SELECT epis_char.season,
10  (SELECT COUNT(ep.episode_id)
11   FROM episodes AS ep
12   WHERE substr(ep.episode_id, 1, 3) = epis_char.season) AS ep_cnt,
13  (SELECT COUNT(1) FROM episodes) AS all_ep_cnt,
14  COUNT(DISTINCT epis_char.name) AS char_cnt
15 FROM epis_char
16 GROUP BY epis_char.season
17 ORDER BY 1;
```

Table

+ Add Visualization

season	ep_cnt	all_ep_cnt	char_cnt
S01	11	51	188
S02	10	51	167
S03	10	51	195
S04	10	51	177
S05	10	51	162

Вопросы на собеседованиях

1. Чем отличаются коррелированные и некоррелированные подзапросы?

- Некоррелированные подзапросы могут выполняться независимо от внешнего запроса
- Коррелированные подзапросы зависят от данных из внешнего запроса

2. Как работают EXISTS, SOME, ALL и ANY?

- EXISTS проверяет наличие хотя бы одной строки в подзапросе
- SOME и ANY сравнивают значение поля с любым значением из подзапроса
- ALL сравнивает значение поля со всеми значениями в подзапросе

3. Что такое скалярный подзапрос?

- Подзапрос, который возвращает единственное значение

4. Что будет, если подзапрос в WHERE вернет NULL?

- Условие не будет выполнено, так как NULL не является ни true, ни false. Таким образом, во внешнем подзапросе не вернется ни одной строки

5. Может ли подзапрос в ORDER BY быть некоррелированным?

- Да, технически может, но он не повлияет на сортировку

6. Зачем используется CTE?

- Для улучшения читаемости и для оптимизации запросов

7. Чем CTE отличается от временной таблицы?

- Временная таблица существует до завершения сессии пользователем, а CTE существует только в пределах выполняемого запроса

8. Может ли CTE ссылаться на другой CTE внутри запроса?

- Да, может, это улучшает читаемость

Резюме

- **Подзапрос** — это запрос внутри другого запроса
- **Типы подзапросов:**
 - **Некоррелированные подзапросы** — подзапросы, которые могут быть выполнены независимо от внешнего запроса
 - **Коррелированные подзапросы** — подзапросы, которые обращаются к внешней таблице, то есть они зависят от внешнего запроса
 - **Скалярные запросы** — запросы, которые возвращают единственное значение

- Подзапросы могут использоваться в блоках `SELECT` , `FROM` , `JOIN` , `WHERE` , `HAVING` и `ORDER BY`
- Некоторые СУБД требуют, чтобы у подзапросов, которые используются во `FROM` , были алиасы
- **Инструкции в WHERE с подзапросами:**
 - `IN` — проверяет, содержится ли значение в результатах подзапроса
 - `EXISTS` — проверяет наличие строк в подзапросе
 - `ANY/SOME` — используется для сравнения значения с любым значением из подзапроса
 - `ALL` — используется для сравнения значения со всеми значениями в подзапросе
- **CTE (Common Table Expression)** позволяет вынести подзапрос за пределы основного запроса. Он объявляется с помощью ключевого слова `WITH` и может использоваться несколько раз, но только в рамках одного общего запроса
- **Преимущества CTE:**
 - **Улучшение читаемости:** Разделение сложного запроса на более понятные части
 - **Оптимизация:** Возможность использовать результаты CTE несколько раз без повторного выполнения подзапроса
- **Разница между CTE и временными таблицами:**
 - **Временная таблица** создается пользователем и существует до завершения сессии (завершения соединения с СУБД)
 - **CTE** существует только в пределах выполнения запроса



Урок 7 Аналитические функции

1. Аналитические функции
2. Три группы аналитических функций
3. Агрегирующие функции
4. Функции смещения
5. Функции ранжирования
6. Фреймы
7. Window Frame
8. Вопросы на собеседовании
9. Резюме

Аналитические функции

Аналитические функции SQL, также известные как оконные функции, играют ключевую роль в современном анализе данных. Они позволяют выполнять сложные вычисления на множестве строк, сохраняя при этом каждую строку в результирующем наборе данных. Эти функции расширяют возможности традиционных агрегатных функций, таких как `SUM`, `AVG`, и `COUNT`, позволяя выполнять операции над подмножествами данных. Благодаря своей мощности и гибкости аналитические функции стали незаменимым инструментом для

аналитиков и разработчиков, стремящихся к более глубокому пониманию данных и оптимизации производительности запросов.

Аналитическая (оконная) функция — это функция, выполняющая вычисления на основе определенного набора записей, которые являются окном (партицией). Оконная функция применяется к данным внутри окна и возвращает одиночное значение в качестве результата вычисления.



Одно из главных преимуществ оконных функций заключается в том, что они возвращают ровно то же количество записей, которое получили на вход. Представьте, что вы хотите рассчитать некоторое значение для группы строк, объединённых общим признаком (например, id пользователя). Если бы вы воспользовались оператором GROUP BY, то на выходе вместо исходного количества строк в группе получили бы одну строку с результатом. При группировке так происходит всегда — число строк в результирующей таблице всегда равно количеству групп в исходной таблице. В то же время оконная функция позволяет проводить те же расчёты с агрегацией по группам, но при этом сохраняет структуру исходной таблицы: для каждой записи, принадлежащей определённой группе, в отдельном столбце просто указывается результат агрегации.

Аналитические (оконные) функции позволяют реализовать сложную логику, такую как расчет нарастающей суммы, определение дат первых и последних визитов клиента, и многое другое.

Данные делятся на **партиции (окна)** по определённому полю или полям. Внутри каждой партиции данные сортируются, после чего выполняется их анализ.

Пример

Рассмотрим пример. У нас есть пользователи, совершающие покупки. Для каждого пользователя в таблице фиксируется дата и время покупки, а также сумма покупки.

user_id	payment_sum	payment_dttm
1	5	2025-05-06 18:00:34
1	12	2025-05-06 16:17:43
1	50	2025-05-06 15:58:02
2	33	2025-05-06 09:13:22
3	2	2025-05-08 14:28:59
3	34	2025-05-05 11:39:05
4	6	2025-05-09 14:18:02

Пример запроса с оконной функцией, вычисляющий номер покупки для каждого пользователя в хронологическом порядке, и сумму с нарастающим итогом всех покупок для каждого пользователя:

example.sql ×

```
1 SELECT user_id,  
2        payment_dttm,  
3        ROW_NUMBER() OVER  
4          (PARTITION BY user_id  
5            ORDER BY payment_dttm) AS num,  
6        SUM(payment_sum) OVER  
7          (PARTITION BY user_id  
8            ORDER BY payment_dttm) AS cume_sum  
9 FROM payments
```

Рассмотрим подробнее синтаксис оконной функции. Оконная функция используется с ключевым словом **OVER**. После **OVER** в скобках указывается описание окна. **PARTITION BY** определяет поля для разбиения данных на группы. **ORDER BY** задаёт порядок сортировки данных внутри каждой партии.

1 <функция>(<поле>) OVER (PARTITION BY <партия> ORDER BY <сортировка> <фрейм>)

- **функция** — оконная функция над выбранным полем;
- **партия** — поле или набор полей, по которым определяется группа;
 - в нашем случае разбиваем на группы по user_id

user_id	payment_sum	payment_dttm
1	5	2025-05-06 18:00:34
1	12	2025-05-06 16:17:43
1	50	2025-05-06 15:58:02
2	33	2025-05-06 09:13:22
3	2	2025-05-08 14:28:59
3	34	2025-05-05 11:39:05
4	6	2025-05-09 14:18:02

- **сортировка** — поле или набор полей, по которым строки сортируются внутри партии;
 - в нашем примере сортируем по дате совершения покупки

user_id	payment_sum	payment_dttm
1	50	2025-05-06 15:58:02
1	12	2025-05-06 16:17:43
1	5	2025-05-06 18:00:34
2	33	2025-05-06 09:13:22
3	34	2025-05-05 11:39:05
3	2	2025-05-08 14:28:59
4	6	2025-05-09 14:18:02

- **фрейм** — набор строк внутри окна, которыми мы оперируем.

- необязательная часть запроса

Сначала производится разбиение на партии, определенные в инструкции **OVER**, при необходимости записи сортируются, а затем уже к каждой записи внутри партии применяется функция. **ROW_NUMBER()** возвращает номер строки внутри партии в поле **num**, а **SUM()** вычисляет кумулятивную сумму значений внутри партии в поле **cum_sum** (с каждой новой строкой наша сумма нарастает, таким образом получается сумма с нарастающим итогом).

user_id	payment_sum	payment_dttm	user_id	num	cume_sum	payment_dttm
1	50	2025-05-06 18:00:34	1	1	50	2025-05-06 18:00:34
1	12	2025-05-06 16:17:43	1	2	62	2025-05-06 16:17:43
1	5	2025-05-06 15:58:02	1	3	67	2025-05-06 15:58:02
2	33	2025-05-06 09:13:22	2	1	33	2025-05-06 09:13:22
3	34	2025-05-08 14:28:59	3	1	34	2025-05-08 14:28:59
3	2	2025-05-05 11:39:05	3	2	36	2025-05-05 11:39:05
4	6	2025-05-09 14:18:02	4	1	6	2025-05-09 14:18:02



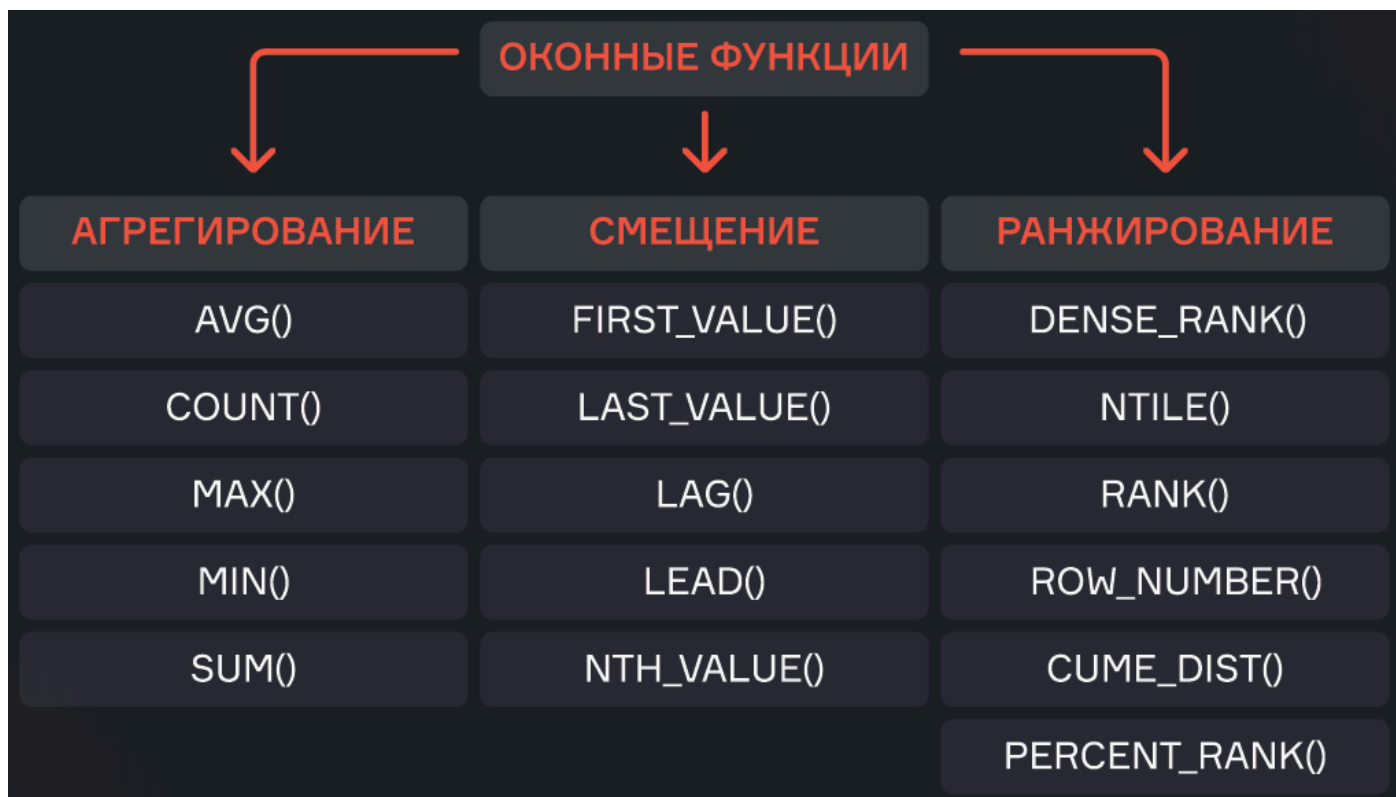
Почему же считается сумма именно текущей и всех предыдущих, а не вообще всех покупок пользователя? Дело в том, что при использовании оконных функций в паре с агрегирующими для каждой строки

определяется так называемая **рамка окна** — набор строк в её партии. Если в OVER указать ORDER BY, то по умолчанию рамка будет состоять из всех строк от начала партии до текущей строки (также в рамку будут включены строки, равные текущей строке по значению, указанному в ORDER BY). Именно поэтому в нашем примере сумма считается по каждому пользователю нарастающим итогом. Если же ORDER BY не указывать, то рамка по умолчанию будет состоять из всех строк партии, т.е. будет посчитана сумма всех покупок каждого пользователя. Также можно не указывать и PARTITION BY, тогда рамкой окна станет вся таблица, и мы просто посчитаем сумму покупок всех пользователей.

Три группы аналитических функций

Аналитические функции делятся на три основные группы:

1. Агрегирующие функции (SUM, AVG, MAX, MIN).
2. Функции смещения (LAG, LEAD, FIRST_VALUE, LAST_VALUE, NTH_VALUE).
3. Функции ранжирования (ROW_NUMBER, RANK, DENSE_RANK, NTILE, CUM_DIST, PERCENT_RANK).



Агрегирующие функции

Агрегирующие функции позволяют считать накопительные (кумулятивные) значения внутри окна:

- среднее (`AVG`)
- количество (`COUNT`)
- максимум (`MAX`)
- минимум (`MIN`)
- сумма (`SUM`)

Для рассмотрения оконных функций рассмотрим следующий запрос:

```
1 SELECT c.name,  
2        substr(ep.episode_id, 1, 3) AS season,  
3        COUNT(DISTINCT ep.episode_id) AS ep_cnt  
4 FROM characters AS c  
5 LEFT JOIN char_ep AS ce  
6     ON c.id = ce.character_id  
7 LEFT JOIN episodes AS ep  
8     ON ce.episode_id = ep.id  
9 GROUP BY 1, 2  
10 HAVING COUNT(ep.episode_id) > 5
```

В этом запросе мы объединили таблицу персонажей с таблицей эпизодов, получив на выходе всех персонажей и эпизоды, в которых они задействованы. Далее мы сгруппировали выборку по имени персонажа и по сезону, посчитав количество уникальных эпизодов, в которых наши персонажи участвовали, и ограничив выборку теми персонажами, которые участвовали в сезоне более пяти раз.

Результат запроса:

name	season	ep_cnt
Beth Smith	S01	11
Beth Smith	S02	8
Beth Smith	S03	10
Beth Smith	S04	10
Beth Smith	S05	9
Jerry Smith	S01	11
Jerry Smith	S02	9
Jerry Smith	S03	6
Jerry Smith	S04	8
Jerry Smith	S05	10
Morty Smith	S01	11
Morty Smith	S02	10

Обернув этот запрос в CTE, можно показать работу аналитических агрегирующих функций:

```

1 WITH char_in_episodes AS (
2     SELECT c.name,
3           substr(ep.episode_id, 1, 3) AS season,
4           COUNT(DISTINCT ep.episode_id) AS ep_cnt
5     FROM characters AS c
6     LEFT JOIN char_ep AS ce
7           ON c.id = ce.character_id
8     LEFT JOIN episodes AS ep
9           ON ce.episode_id = ep.id
10    GROUP BY 1, 2
11    HAVING COUNT(ep.episode_id) > 5
12 )
13 SELECT name,
14        season,
15        ep_cnt,
16        SUM(ep_cnt) OVER (PARTITION BY name ORDER BY season)::int AS ep_sum,
17        AVG(ep_cnt) OVER (PARTITION BY name ORDER BY season) AS ep_avg,
18        MIN(ep_cnt) OVER (PARTITION BY name ORDER BY season)::int AS ep_min,
19        MAX(ep_cnt) OVER (PARTITION BY name ORDER BY season)::int AS ep_max

```

```
20 FROM char_in_episodes;
```

name	season	ep_cnt	ep_sum	ep_avg	ep_min	ep_max
Beth Smith	S01	11	11	11.00	11	11
Beth Smith	S02	8	19	9.50	8	11
Beth Smith	S03	10	29	9.67	8	11
Beth Smith	S04	10	39	9.75	8	11
Beth Smith	S05	9	48	9.60	8	11
Jerry Smith	S01	11	11	11.00	11	11
Jerry Smith	S02	9	20	10.00	9	11
Jerry Smith	S03	6	26	8.67	6	11
Jerry Smith	S04	8	34	8.50	6	11
Jerry Smith	S05	10	44	8.80	6	11
Morty Smith	S01	11	11	11.00	11	11
Morty Smith	S02	10	21	10.50	10	11
Morty Smith	S03	10	31	10.33	10	11
Morty Smith	S04	10	41	10.25	10	11
Morty Smith	S05	10	51	10.20	10	11

Этот запрос показывает кумулятивные сумму (`ep_sum`), среднее (`ep_avg`), минимум (`ep_min`) и максимум (`ep_max`) по эпизодам для каждого персонажа по сезонам.

Функции смещения

Функции смещения позволяют обращаться к значениям соседних строк:

- `FIRST_VALUE` — первая строка в партии
- `LAST_VALUE` — последняя строка в партии
- `LAG` — значение в предыдущей строке партии
- `LEAD` — значение в следующей строке партии

- **NTH_VALUE** — значение в строке партии под номером N (переданным в качестве второго аргумента функции)

Пример работы функций смещения:

```
1 WITH char_in_episodes AS (  
2     SELECT c.name,  
3           substr(ep.episode_id, 1, 3) AS season,  
4           COUNT(DISTINCT ep.episode_id) AS ep_cnt  
5     FROM characters AS c  
6     LEFT JOIN char_ep AS ce  
7     ON c.id = ce.character_id  
8     LEFT JOIN episodes AS ep  
9     ON ce.episode_id = ep.id  
10    GROUP BY 1, 2  
11    HAVING COUNT(ep.episode_id) > 5  
12 )  
13 SELECT name,  
14        season,  
15        ep_cnt,  
16        FIRST_VALUE(ep_cnt) OVER (PARTITION BY name ORDER BY season) AS first,  
17        LAST_VALUE(ep_cnt) OVER (PARTITION BY name ORDER BY season) AS last,  
18        LAG(ep_cnt) OVER (PARTITION BY name ORDER BY season) AS  
19        prev_season_cnt,  
20        LEAD(ep_cnt) OVER (PARTITION BY name ORDER BY season) AS  
21        next_season_cnt,  
22        NTH_VALUE(ep_cnt, 2) OVER (PARTITION BY name ORDER BY season) AS  
23        second_season_cnt  
24 FROM char_in_episodes;
```


name	season	ep_cnt	first	last	prev_season_cnt	next_season_cnt	second_season_cnt
Beth Smith	S01	11	11	11		8	
Beth Smith	S02	8	11	8	11	10	8
Beth Smith	S03	10	11	10	8	10	8
Beth Smith	S04	10	11	10	10	9	8
Beth Smith	S05	9	11	9	10		8
Jerry Smith	S01	11	11	11		9	
Jerry Smith	S02	9	11	9	11	6	9
Jerry Smith	S03	6	11	6	9	8	9
Jerry Smith	S04	8	11	8	6	10	9

В этом примере используются функции `FIRST_VALUE`, `LAST_VALUE`, `LAG`, `LEAD` и `NTH_VALUE` для получения первого (`first`), последнего (`last`), предыдущего (`prev_season_cnt`), следующего (`next_season_cnt`) и второго (`second_season_cn`) значений в рамке каждого окна.

Функции ранжирования

Функции ранжирования назначают порядковые номера строкам или ранжируют их внутри партиций:

- `ROW_NUMBER` — возвращает номер строки внутри партиции
- `RANK` — вычисляет ранг записи в рамках партиции, отбрасывая следующий ранг при повторяющихся значениях
- `DENSE_RANK` — вычисляет ранг записи в рамках партиции без пропусков при повторяющихся значениях
- `NTILE` — производит разбиение окна на заданное количество групп, которое передается в функцию в качестве аргумента, и возвращает номер группы

- **CUME_DIST** — возвращает относительный номер строки (кумулятивное распределение RANK/COUNT)
- **PERCENT_RANK** — возвращает относительный ранг $(RANK-1)/(COUNT-1)$

Пример работы функций ранжирования:

```
1 WITH char_in_episodes AS (  
2     SELECT c.name,  
3           substr(ep.episode_id, 1, 3) AS season,  
4           COUNT(DISTINCT ep.episode_id) AS ep_cnt  
5     FROM characters AS c  
6     LEFT JOIN char_ep AS ce  
7     ON c.id = ce.character_id  
8     LEFT JOIN episodes AS ep  
9     ON ce.episode_id = ep.id  
10    GROUP BY 1, 2  
11    HAVING COUNT(ep.episode_id) > 5  
12 )  
13 SELECT name,  
14        season,  
15        ep_cnt,  
16        ROW_NUMBER() OVER (PARTITION BY name ORDER BY ep_cnt) AS rn,  
17        RANK() OVER (PARTITION BY name ORDER BY ep_cnt) AS rank,  
18        DENSE_RANK() OVER (PARTITION BY name ORDER BY ep_cnt) AS dense_rank,  
19        NTILE(2) OVER (PARTITION BY name ORDER BY ep_cnt) AS group_num,  
20        CUME_DIST() OVER (PARTITION BY name ORDER BY ep_cnt) AS cume_dist,  
21        PERCENT_RANK() OVER (PARTITION BY name ORDER BY ep_cnt) AS percent_rank  
22 FROM char_in_episodes;
```

name	season	ep_cnt	rn	rank	dense_rank	group_num	cume_dist	percent_rank
Beth Smith	S02	8	1	1	1	1	0.20	0.00
Beth Smith	S05	9	2	2	2	1	0.40	0.25
Beth Smith	S03	10	3	3	3	1	0.80	0.50
Beth Smith	S04	10	4	3	3	2	0.80	0.50
Beth Smith	S01	11	5	5	4	2	1.00	1.00
Jerry Smith	S03	6	1	1	1	1	0.20	0.00
Jerry Smith	S04	8	2	2	2	1	0.40	0.25
Jerry Smith	S02	9	3	3	3	1	0.60	0.50
Jerry Smith	S05	10	4	4	4	2	0.80	0.75
Jerry Smith	S01	11	5	5	5	2	1.00	1.00
Morty Smith	S03	10	1	1	1	1	0.80	0.00
Morty Smith	S02	10	2	1	1	1	0.80	0.00
Morty Smith	S04	10	3	1	1	1	0.80	0.00
Morty Smith	S05	10	4	1	1	2	0.80	0.00

Этот запрос показывает различия между `ROW_NUMBER` (столбец `rn`), `RANK`, `DENSE_RANK`, `NTILE` (столбец `group_num`), `CUME_DIST` и `PERCENT_RANK`:

- Функция `ROW_NUMBER` пронумеровала строки в рамках каждого персонажа. А переданное в качестве аргумента в функцию `NTILE` число 2 позволило разделить строки внутри каждой партии на две примерно равные части (влияет нечетность количества строк внутри партий) и пронумеровать каждую строку в соответствии со своей группой.
- Мы видим, что в третьем и в четвертом сезоне Бэт Смит играла по 10 раз и обе строки пронумерованы цифрой 3 как в `RANK`, так и в `DENSE_RANK`. Но следующая строка в `DENSE_RANK` будет иметь номер 4, а в `RANK` номер 5.
- Функция `CUME_DIST` показывает долю строк, которые меньше либо равны текущей строке. А функция `PERCENT_RANK` показывает долю строк, которые меньше текущей строки. Соответственно для последнего значения обеих функций это всегда единица. Для первого значения в `PERCENT_RANK` нет ни одной строки, значение которых меньше ее, а для кумулятивного распределения `CUME_DIST` первое значение — ненулевое.

Фреймы

Фреймы позволяют детализировать диапазон строк для агрегирования.

Существует два способа задания фреймов:

1. **ROWS** – работает со строками внутри партиции.
2. **RANGE** – работает с диапазоном значений внутри партиции.

ROWS

Указывает нижнюю и верхнюю границу окна по номерам строк.

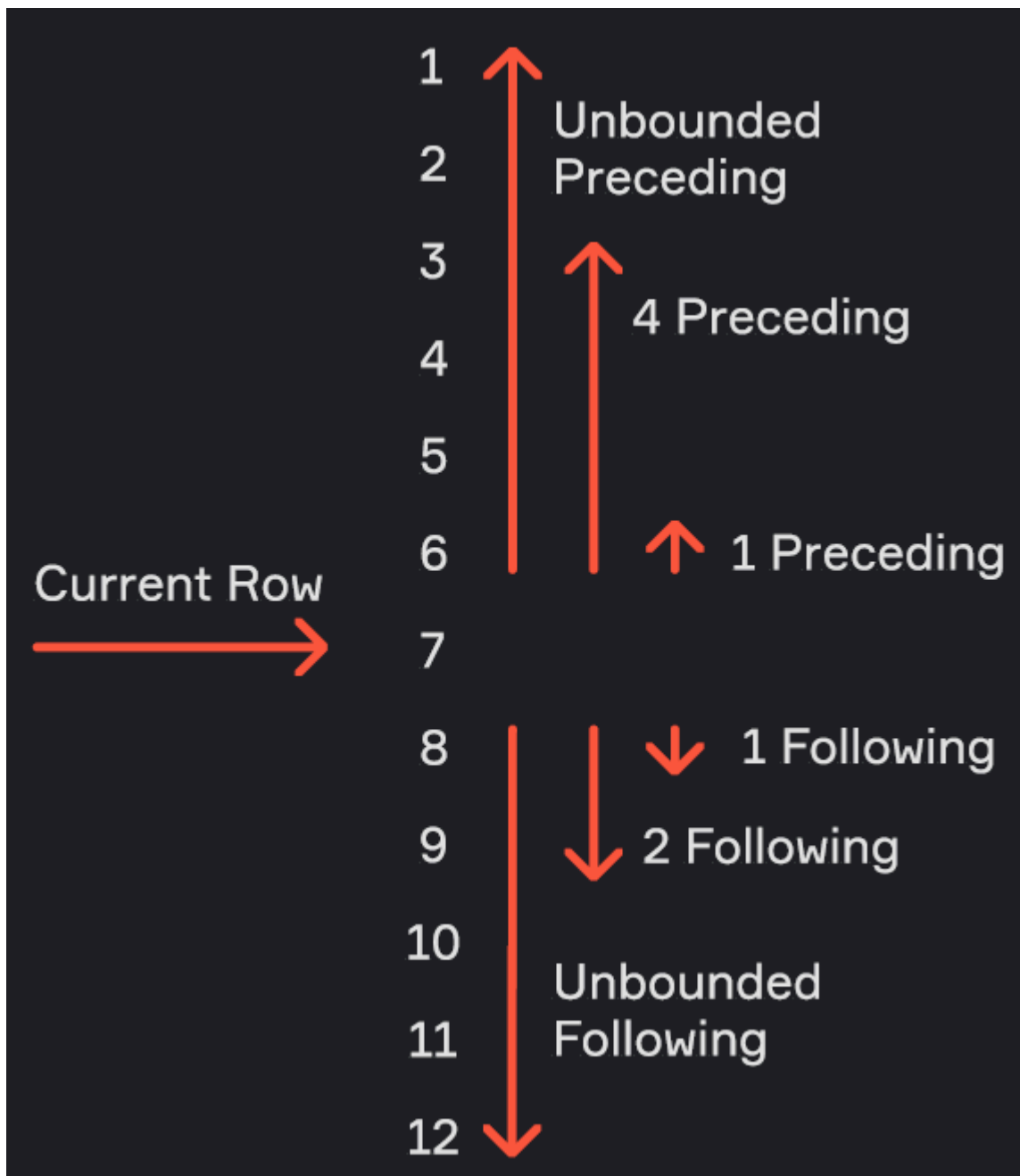
Синтаксис:

ROWS BETWEEN <нижняя граница> AND <верхняя граница>

- **UNBOUNDED PRECEDING** — окно начинается с первой строки группы
- **UNBOUNDED FOLLOWING** – окно заканчивается на последней строке группы
- **CURRENT ROW** – окно начинается или заканчивается на текущей строке

Также можно указать номера строк:

- **<число строк> PRECEDING** – определяет число строк перед текущей строкой
- **<число строк> FOLLOWING** — определяет число строк после текущей строки



Например, у нас есть партиция из 12 строк. **ROWS** позволяет нам выбрать те строки, значения которых нам нужно агрегировать. Например, текущая строка номер 7, по умолчанию мы агрегируем для нее все строки, начиная с первой и до нее включительно. Но можно выбрать и другие строки, которые мы будем агрегировать. Например, начиная с третьей строки до текущей. Начиная с одной строки назад до текущей. Начиная с текущей до последней строки в окне. Начиная с текущей и еще две строки. Начиная с текущей и еще одну строку. Для этого мы используем нотацию **ROWS BETWEEN**, затем указываем нижнюю границу, **AND** и верхнюю границу.

Пример запроса с использованием фрейма ROWS:

```
1 WITH char_in_episodes AS (
```

```

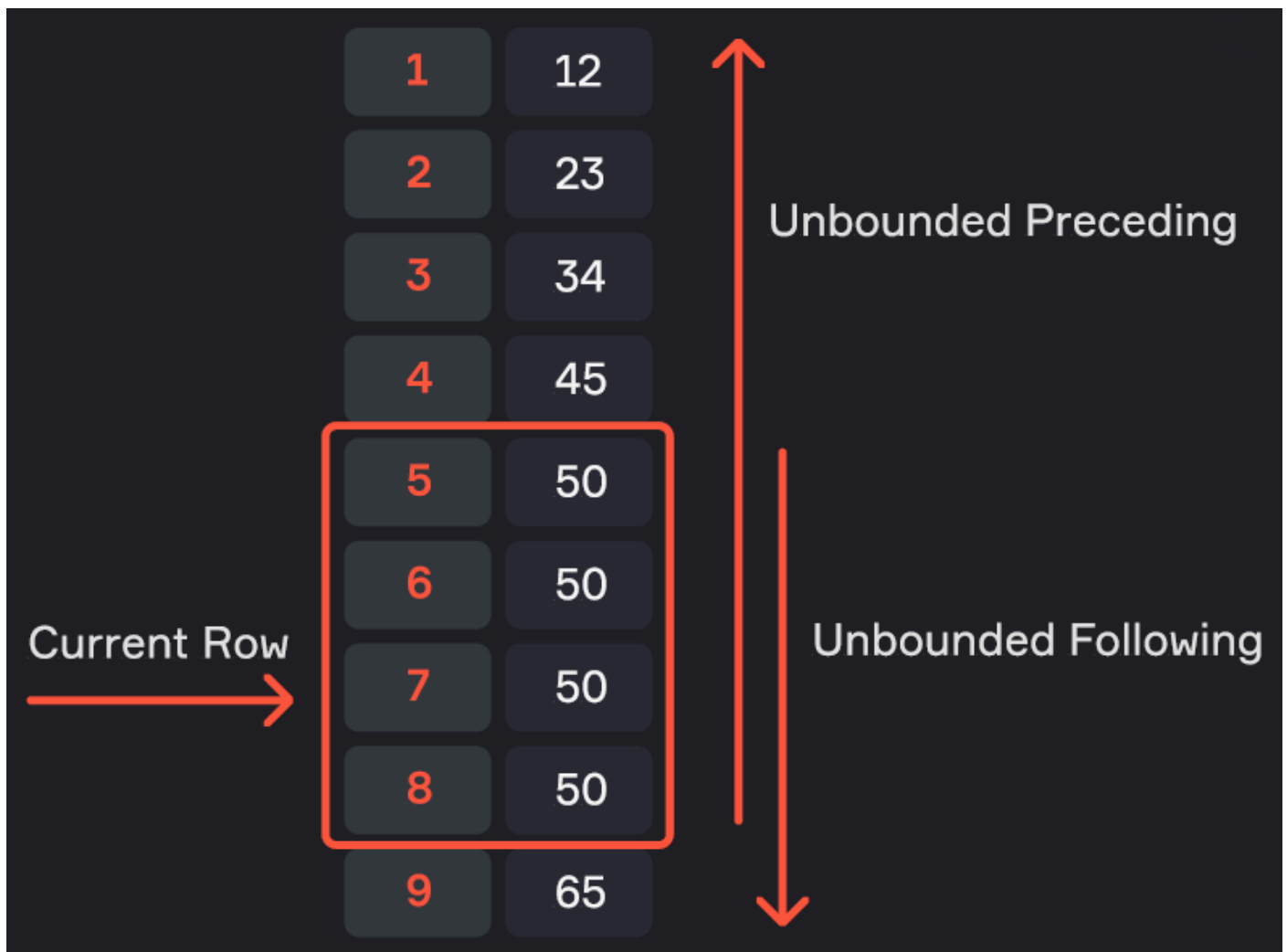
2  SELECT c.name,
3         substr(ep.episode_id, 1, 3) AS season,
4         COUNT(DISTINCT ep.episode_id) AS ep_cnt
5  FROM characters AS c
6  LEFT JOIN char_ep AS ce
7         ON c.id = ce.character_id
8  LEFT JOIN episodes AS ep
9         ON ce.episode_id = ep.id
10 GROUP BY 1, 2
11 HAVING COUNT(ep.episode_id) > 5
12 )
13 SELECT name,
14        season,
15        ep_cnt,
16        SUM(ep_cnt) OVER (PARTITION BY name ORDER BY season ROWS BETWEEN UNBOUNDED
17        PRECEDING AND CURRENT ROW)::int AS v1,
18        SUM (ep_cnt) OVER (PARTITION BY name ORDER BY season ROWS BETWEEN CURRENT
19        ROW AND UNBOUNDED FOLLOWING) AS v2,
20        SUM (ep_cnt) OVER (PARTITION BY name ORDER BY season ROWS BETWEEN 3
21        PRECEDING AND CURRENT ROW)::int AS v3,
22        SUM (ep_cnt) OVER (PARTITION BY name ORDER BY season ROWS BETWEEN CURRENT
23        ROW AND 2 FOLLOWING)::int AS v4
24 FROM char_in_episodes

```

name	season	ep_cnt	v1	v2	v3	v4
Beth Smith	S01	11	11	48.00	11	29
Beth Smith	S02	8	19	37.00	19	28
Beth Smith	S03	10	29	29.00	29	29
Beth Smith	S04	10	39	19.00	39	19
Beth Smith	S05	9	48	9.00	37	9
Jerry Smith	S01	11	11	44.00	11	26
Jerry Smith	S02	9	20	33.00	20	23
Jerry Smith	S03	6	26	24.00	26	24
Jerry Smith	S04	8	34	18.00	34	18
Jerry Smith	S05	10	44	10.00	33	10

RANGE

Указывает нижнюю и верхнюю границу окна по значениям строк.



RANGE BETWEEN <нижняя граница> AND <верхняя граница>

- **UNBOUNDED PRECEDING** — окно начинается с первого значения группы
- **UNBOUNDED FOLLOWING** – окно заканчивается на последнем значении группы
- **CURRENT ROW** – окно начинается или заканчивается на текущем значении



По умолчанию, если мы не указали фрейм, применяется **RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW** : все записи, начиная с первой строки окна до текущей строки.

Всё, что до текущей строки
и само значение текущей строки

**RANGE BETWEEN UNBOUNDED
PRECEDING AND CURRENT ROW**

Текущая строка и всё, что после неё

**RANGE BETWEEN CURRENT ROW
AND UNBOUNDED FOLLOWING**

Значение по умолчанию

Пример

Все, что до текущего диапазона и само текущее значение:

```
1 RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
```

Текущий диапазон и все, что после него:

```
1 RANGE BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING
```

Пример запроса с использованием фрейма RANGE:

```
1 WITH char_in_episodes AS (  
2     SELECT c.name,  
3         substr(ep.episode_id, 1, 3) AS season,  
4         COUNT(DISTINCT ep.episode_id) AS ep_cnt  
5     FROM characters AS c  
6     LEFT JOIN char_ep AS ce  
7         ON c.id = ce.character_id  
8     LEFT JOIN episodes AS ep  
9         ON ce.episode_id = ep.id  
10    GROUP BY 1, 2  
11    HAVING COUNT(ep.episode_id) > 5  
12 )
```



```

13 SELECT name,
14     season,
15     ep_cnt,
16     SUM(ep_cnt) OVER (PARTITION BY name ORDER BY ep_cnt RANGE BETWEEN
    UNBOUNDED PRECEDING AND CURRENT ROW)::int AS v1,
17     SUM(ep_cnt) OVER (PARTITION BY name ORDER BY ep_cnt RANGE BETWEEN
    UNBOUNDED PRECEDING AND CURRENT ROW)::int AS v2,
18     SUM (ep_cnt) OVER (PARTITION BY name ORDER BY ep_cnt RANGE BETWEEN CURRENT
    ROW AND UNBOUNDED FOLLOWING) AS v3
19 FROM char_in_episodes

```

name	season	ep_cnt	v1	v2	v3
Beth Smith	S02	8	8	8	48.00
Beth Smith	S05	9	17	17	40.00
Beth Smith	S03	10	37	37	31.00
Beth Smith	S04	10	37	37	31.00
Beth Smith	S01	11	48	48	11.00
Jerry Smith	S03	6	6	6	44.00
Jerry Smith	S04	8	14	14	38.00
Jerry Smith	S02	9	23	23	30.00
Jerry Smith	S05	10	33	33	21.00
Jerry Smith	S01	11	44	44	11.00

Window Frame

Window Frame — именованная партиция, которую можно использовать в запросе больше одного раза. Window Frame улучшает читаемость запроса и в некоторых СУБД увеличивает скорость запросов.

Синтаксис Window Frame:

```

1 **WINDOW <alias>** AS (PARTITION BY <партиция> ORDER BY <сортировка> <фрейм>)

```

Определение именованной партиции производится после запроса.

Пример:

```
1 WITH char_in_episodes AS (  
2     SELECT c.name,  
3         substr(ep.episode_id, 1, 3) AS season,  
4         COUNT(DISTINCT ep.episode_id) AS ep_cnt  
5     FROM characters AS c  
6     LEFT JOIN char_ep AS ce  
7         ON c.id = ce.character_id  
8     LEFT JOIN episodes AS ep  
9         ON ce.episode_id = ep.id  
10    GROUP BY 1, 2  
11    HAVING COUNT(ep.episode_id) > 5  
12 )  
13 SELECT name,  
14     season,  
15     ep_cnt,  
16     SUM(ep_cnt) OVER w::int AS an_sum,  
17     AVG(ep_cnt) OVER w AS an_avg,  
18     MIN(ep_cnt) OVER w::int AS an_min,  
19     MAX(ep_cnt) OVER w::int AS an_max  
20 FROM char_in_episodes  
21 WINDOW w AS (PARTITION BY name ORDER BY season)
```

name	season	ep_cnt	an_sum	an_avg	an_min	an_max
Beth Smith	S01	11	11	11.00	11	11
Beth Smith	S02	8	19	9.50	8	11
Beth Smith	S03	10	29	9.67	8	11
Beth Smith	S04	10	39	9.75	8	11
Beth Smith	S05	9	48	9.60	8	11
Jerry Smith	S01	11	11	11.00	11	11
Jerry Smith	S02	9	20	10.00	9	11
Jerry Smith	S03	6	26	8.67	6	11
Jerry Smith	S04	8	34	8.50	6	11
Jerry Smith	S05	10	44	8.80	6	11
Morty Smith	S01	11	11	11.00	11	11
Morty Smith	S02	10	21	10.50	10	11
Morty Smith	S03	10	31	10.33	10	11
Morty Smith	S04	10	41	10.25	10	11
Morty Smith	S05	10	51	10.20	10	11

Вопросы на собеседовании

- **Как посчитать кумулятивную сумму?**

С помощью аналитической функции `SUM`. Мы делим на партии наши данные, сортируем их и применяем к нему агрегирующую функцию.

- **Что будет, если в оконных функциях не указать фрейм?**

Мы агрегируем данные, начиная с самой первой строки окна по текущую включительно.

- **Какие варианты указания фрейма вы знаете?**

- по строкам (`ROWS`)
- по значениям строк (`RANGE`).

- **Чем отличается RANK и DENSE_RANK?**

RANK при ранжировании делает пропуски, если значение нескольких строк одинаково. А DENSE_RANK после одинаковых строк присваивает последующее значение без пропусков.

Резюме

- **Аналитическая (оконная) функция** — это функция, выполняющая вычисления на основе определенного набора записей, которые являются окном (партицией). Оконная функция применяется к отсортированным данным и возвращает одиночное значение в качестве результата вычисления.
- Одно из главных преимуществ оконных функций заключается в том, что они возвращают ровно то количество записей, которое получили на вход.
- Синтаксис оконной функции: **<функция>(<поле>) OVER (PARTITION BY <партиция> ORDER BY <сортировка> <фрейм>)**, где:
 - **функция** — оконная функция над выбранным полем;
 - **партиция** — поле или набор полей, по которым определяется группа;
 - **сортировка** — поле или набор полей, по которым строки отсортировываются внутри партии;
 - **фрейм** — набор строк внутри окна, которыми мы оперируем.
- Аналитические функции делятся на три основные группы:
 - Агрегирующие функции (SUM, AVG, MAX, MIN) позволяют считать накопительные (кумулятивные) значения внутри окна.
 - Функции смещения (LAG, LEAD, FIRST_VALUE, LAST_VALUE, NTH_VALUE) позволяют обращаться к значениям соседних строк.
 - Функции ранжирования (ROW_NUMBER, RANK, DENSE_RANK, NTILE, CUM_DIST, PERCENT_RANK) назначают порядковые номера строкам или ранжируют их внутри партий.
- Фреймы позволяют детализировать диапазон строк для агрегирования. Существует два способа задания фреймов:
 - **ROWS** — работает со строками внутри партии;
 - **RANGE** — работает с диапазоном значений внутри партии.
- Синтаксис задания фрейма:

ROWS BETWEEN <нижняя граница> AND <верхняя граница>

- **UNBOUNDED PRECEDING** — окно начинается с первой строки группы
- **UNBOUNDED FOLLOWING** — окно заканчивается на последней строке группы
- **CURRENT ROW** — окно начинается или заканчивается на текущей строке
- **<число строк> PRECEDING** — определяет число строк перед текущей строкой
- **<число строк> FOLLOWING** — определяет число строк после текущей строки

Для **RANGE** указание числа строк не актуально.

- **Window Frame** — именованная партиция, которую можно использовать в запросе больше одного раза. Window Frame улучшает читаемость запроса и в некоторых СУБД увеличивает скорость запросов.

Синтаксис Window Frame:

WINDOW <alias> AS (PARTITION BY <партиция> ORDER BY <сортировка>
<фрейм>)



Урок 8 DML&DDL

1. DDL (Data Definition Language)
2. DML (Data Manipulation Language)
3. Резюме

Урок 8 DML&DDL

DDL (Data Definition Language)

DDL (Data Definition Language) описывает структуру данных, то есть таблицы, индексы, ограничения и последовательности. Основные инструкции DDL включают `CREATE`, `ALTER`, `DROP` и `TRUNCATE`.

CREATE

Создание таблицы:

```
1 CREATE TABLE my_table (  
2     id                int,  
3     name              text,  
4     birth_date        date,  
5     sex               text  
6 );
```

my_table			
id	name	birth_date	sex
1	Иван	2000-01-01	male
2	Марья	1997-03-18	female

Для PostgreSQL и ClickHouse синтаксис различается:

Мы рассматриваем синтаксис DDL достаточно упрощенно, поэтому при работе с конкретной СУБД необходимо обязательно ознакомиться с документацией. Ниже приведены примеры различий синтаксиса для разных СУБД.

PostgreSQL:

```

1 CREATE TABLE my_table (
2     id                SERIAL PRIMARY KEY,
3     name              VARCHAR(255),
4     birth_date        DATE,
5     sex               VARCHAR(6) NOT NULL,
6     CONSTRAINT sex_check CHECK (sex IN ('male', 'female'))
7 );

```

Обратите внимание, что для Postgres мы указываем, что поле `id` это не `INT`, а `SERIAL PRIMARY KEY`. В `name` указываем не `TEXT`, а `VARCHAR` с ограничением 255 символов. Пол имеет ограничение `NOT NULL`. И в конце еще можно добавить ограничение, чтобы пол был 'male' или 'female'.

ClickHouse:

```

1 CREATE TABLE my_table
2     ON CLUSTER cluster_name
3 (
4     id                UInt64,
5     name              Nullable(String),
6     birth_date        Nullable(Date),
7     sex               String

```

```
8 ) ENGINE = ReplicatedMergeTree('/clickhouse/tables/{shard}/my_table',  
  '{replica}')  
9 ORDER BY id;
```

Для ClickHouse другие нюансы. Во-первых, мы раскатываем таблицу на все ноды нашего кластера в ClickHouse. Указывается движок Replicated Merge Tree. Данные внутри движка сортируются по полю `id`. Для тех полей, которые могут быть пустыми, прописывается `Nullable`.

ALTER

`ALTER TABLE` позволяет переименовывать таблицы, добавлять, изменять и удалять поля.

Переименование таблицы:

```
1 ALTER TABLE my_table  
2 RENAME TO people;
```

people			
id	name	birth_date	sex
1	Иван	2000-01-01	male
2	Марья	1997-03-18	female

Чаще всего переименование не занимает много времени, поскольку в данном случае не создается новая таблица и копируются в нее данные, а просто заменяется одно название другим внутренними средствами СУБД.

Добавление нового поля:

```
1 ALTER TABLE people  
2 ADD COLUMN is_pet boolean;
```


people				
id	name	birth_date	sex	is_pet
1	Иван	2000-01-01	male	NULL
2	Марья	1997-03-18	female	NULL

Во всех строках, которые мы добавили для этого поля, будет принято значение `NULL`.

Переименование поля:

```
1 ALTER TABLE people
2 ALTER COLUMN is_pet
3 RENAME TO with_pet;
```

people				
id	name	birth_date	sex	with_pet
1	Иван	2000-01-01	male	NULL
2	Марья	1997-03-18	female	NULL

Задание значения по умолчанию для поля:

```
1 ALTER TABLE people
2 ALTER COLUMN with_pet
3 SET DEFAULT False;
```

people				
id	name	birth_date	sex	with_pet
1	Иван	2000-01-01	male	False
2	Марья	1997-03-18	female	False



Значение по умолчанию будет работать только для новых строк. Все строки, которые были добавлены ранее и имеют значение `NULL`, останутся без изменений.

TRUNCATE

Очищение таблицы от записей:



```
1 TRUNCATE TABLE people;
```

people			
id	name	birth_date	sex

`TRUNCATE TABLE` очень быстро удаляет все строки из таблицы. Гораздо быстрее, чем операция `DELETE FROM table`.

DROP

Удаление колонки:



```
1 ALTER TABLE people
```

```
2 DROP COLUMN with_pet;
```

people			
id	name	birth_date	sex
1	Иван	2000-01-01	male
2	Марья	1997-03-18	female

Удаление таблицы:

```
1 DROP TABLE people;
```



Будьте аккуратны с удалением таблицы, потому что ее будет очень сложно восстановить из бэкапа.

DML (Data Manipulation Language)

DML (Data Manipulation Language) манипулирует данными в таблицах. Основные операции включают **INSERT**, **UPDATE** и **DELETE**.

INSERT

Добавление записей:

- **Значений:**

```
1 INSERT INTO people (id, name, birth_date, sex)
2 VALUES (1, 'Иванов Иван Иванович', date '2000-01-01', 'male'),
3         (2, 'Петрова Инна Николаевна', date '1998-03-18', 'female');
```

people			
id	name	birth_date	sex
1	Иванов Иван Иванович	2000-01-01	male
2	Петрова Инна Николаевна	1998-03-18	female

- **Строк из запроса:**

```

1 INSERT INTO people
2 SELECT id, name, date, NULL
3 FROM employees
4 WHERE species = 'Human';

```

Обратите внимание, что поля, которые выбираются в `SELECT`, должны строго соответствовать тем полям, которые будут в таблице, в которую затем добавляются. Если нет какого-либо поля, то на его месте необходимо поставить, например, `NULL`.

UPDATE

Изменение данных:

- Во всех записях столбца:

```

1 UPDATE people
2 SET salary = salary * 1.1;

```

- В отфильтрованных записях:

```

1 UPDATE people
2 SET name = 'Тюленев Петр Алексеевич'
3 WHERE id = 1;

```

people			
id	name	birth_date	sex
1	Тюленев Петр Алексеевич	2000-01-01	male
2	Петрова Инна Николаевна	1998-03-18	female

DELETE

Удаление данных:

```
1 DELETE FROM people
2 WHERE id > 200;
```



Важно помнить, что **DELETE** всегда должен быть с условием, иначе целесообразнее использовать **TRUNCATE**



DELETE FROM people;



Добавить условие

Использовать **TRUNCATE**

Резюме

- **DDL (Data Definition Language)** описывает структуру данных, то есть таблицы, индексы, ограничения и последовательности. Основные инструкции DDL включают:
 - **CREATE TABLE** — создание таблицы
 - **ALTER TABLE** — переименование таблицы, добавление, изменение и удаление полей
 - **DROP COLUMN** — удаление поля
 - **TRUNCATE TABLE** — удаление всех записей из таблицы
- **DML (Data Manipulation Language)** манипулирует данными в таблицах. Основные инструкции DML включают:
 - **INSERT INTO** — добавление записей

- **UPDATE** — изменение данных
- **DELETE** — удаление данных



Урок 9 Дополнительные возможности SQL

1. Представления
2. Оптимизация запросов
3. Регулярные выражения
4. Рекурсивные запросы
5. Вопросы для собеседования
6. Резюме

Урок 9 Дополнительные возможности SQL

Представления

Когда у нас есть несколько таблиц-источников, которые мы объединяем, фильтруем и агрегируем, результат такого запроса может использоваться многократно, а многократное написание запроса может стать трудозатратным. В этом случае можно создать представление (или `VIEW`) на основе этого запроса, то есть фактически дать запросу свое имя. Затем мы можем обращаться к представлению так, как будто это таблица. Но на самом деле представление не является таблицей, а данные хранятся в исходных таблицах-источниках. Каждый раз при обращении к представлению выполняется тот же самый запрос: объединение, фильтрация, агрегация данных. Таким образом,

ресурсы не экономятся, но это удобный способ сделать запрос более читаемым или предоставить заказчикам удобный вид результата.

Несмотря на то, что представления, как и CTE используются в схожих целях, между ними есть различия:

- Представление можно использовать в других запросах без повторного его определения (в отличие от CTE, который живет в рамках одного запроса). Это объясняется тем, что представление является физическим объектом в базе данных и хранится на диске (не забываем, что хранится только запрос, а не данные, возвращаемые этим запросом).
- CTE может быть частью представления. Это помогает, например, когда нам необходимо выполнить рекурсивный запрос, который не возможен без использования CTE.

Пример создания представления:

```
1 CREATE VIEW only_human AS
2 SELECT id, name, status, type, gender, origin_id
3 FROM characters
4 WHERE species = 'Human';
```

Мы создаем представление с помощью инструкции `CREATE VIEW`, указываем название представления, и затем добавляем `SELECT`-запрос, который хотим именовать. Теперь мы можем обращаться к этому представлению как к таблице согласно синтаксису SQL:

```
1 SELECT * FROM only_human;
```

Преимущества:

- Упрощение работы с часто используемыми запросами, так как с представлением можно работать в разных запросах.
- Улучшение читаемости и удобства при предоставлении данных заказчикам.
- Представление может использоваться для ограничения доступа определенных пользователей к базе данных (мы можем предоставить пользователям доступ к определенным представлениям, которые запрашивают данные, которые им разрешено видеть, не раскрывая всю базу данных).



Данные в представлении не хранятся. Каждый раз при обращении к представлению исходные таблицы объединяются, фильтруются и агрегируются заново. Таким образом, при использовании представления не происходит экономии вычислительных ресурсов.

Материализованные представления

Материализованные представления выглядят как обычные, но результаты запроса сохраняются в физическую таблицу (которая хранится на диске). Они поддерживаются не во всех СУБД, а работа с ними может существенно различаться. Например, в некоторых системах данные обновляются в реальном времени, в других — по расписанию или по триггеру, а в некоторых такого функционала вообще нет. Поэтому, работая с материализованными представлениями, всегда нужно обращаться к документации, чтобы понять, как они реализованы в конкретной СУБД.

Оптимизация запросов

Когда мы пишем запрос, внутри движка СУБД он попадает в оптимизатор. Оптимизатор стремится сделать так, чтобы запрос выполнялся быстрее и с минимальными затратами ресурсов. Он анализирует статистику таблиц, выбирает наиболее подходящие индексы и иногда переписывает запрос, чтобы улучшить его работу. Уровень «умности» оптимизатора зависит от конкретной СУБД: в одних случаях он более продвинут, в других менее, а в некоторых мы можем подсказать оптимизатору, какой индекс использовать.

Оптимальные запросы важны по нескольким причинам:

1. **Производительность:** в production-системах каждая миллисекунда важна, чтобы данные быстро возвращались пользователю.
2. **Совместное использование ресурсов:** в аналитических базах, где работает много пользователей, необходимо минимизировать использование ресурсов, чтобы не мешать другим пользователям.
3. **Экономия:** в облачных базах данных вы в том числе платите за использованные вычислительные ресурсы, поэтому оптимизация запросов позволяет сэкономить деньги.

Есть множество способов оптимизации запросов, но они различаются в разных СУБД. Однако есть несколько универсальных подходов, которые мы затронем.

Основные способы оптимизации

Индексы

Для лучшего понимания можно выстроить аналогию. Представим библиотеку, где нужно найти конкретную книгу. Перебор полок займет много времени, поэтому мы обращаемся к библиотекарю, который использует картотеку. В одной картотеке книги отсортированы по фамилии автора, в другой — по названию, в третьей — по году издания. Напротив каждого названия указана полка, где можно найти нужную книгу. Индексы работают аналогично картотеке: они ускоряют поиск данных, но требуют хранения информации и обновления при изменениях.

Создание индексов на всех полях таблицы нецелесообразно. Во-первых, это увеличивает занимаемое место на диске. Во-вторых, добавление, изменение или удаление записей приводит к необходимости обновления индексов. Лучше создавать индексы на тех полях, которые часто используются в фильтрации или объединении таблиц. Оптимизатор использует индексы, чтобы быстрее вернуть данные. В разных СУБД индексы работают по-разному, и их использование зависит от статистики, подсказок пользователя в запросах и других факторов.

Анализ плана запроса

Анализ плана запроса позволяет понять, как именно данные собираются, стоимость (cost) запроса, какие индексы и соединения (`JOIN`) используются. Для этого в запрос добавляют ключевое слово `EXPLAIN` :

▼

```
1 EXPLAIN
2 SELECT ...      -- запрос
```

Примеры оптимизации

Пример — фильтрация по дате:

▼

```
1 SELECT *
2 FROM big_table
3 WHERE year(date) = '2021';
```

Здесь используется функция `year()`, из-за которой индекс на поле `date` не работает. Индексы зависят от конкретного поля, и функции могут их блокировать. Лучше использовать конструкцию `BETWEEN`:

```
1 SELECT *
2 FROM big_table
3 WHERE date BETWEEN DATE '2021-01-01' AND DATE '2021-12-31';
```

Этот запрос менее читаемый, но более оптимальный.

Еще один пример — оптимизация соединений:

```
1 SELECT *
2 FROM first_table AS f
3 JOIN second_table AS s
4 ON s.id = f.id;
```

Если нужно фильтровать данные из второй таблицы, например, по условию соответствия года, лучше добавить условие в `ON`, чтобы избежать лишней фильтрации после соединения:

```
1 SELECT *
2 FROM first_table AS f
3 JOIN second_table AS s
4 ON s.id = f.id AND s.year = '2021';
```

Регулярные выражения

Регулярные выражения позволяют находить строки, соответствующие заданным условиям. Они сложны для чтения, но полезны.

Примеры использования в SQL

- Поиск строк с регистрозависимым совпадением:

```
▼
```

```
1 SELECT *
2 FROM employees
3 WHERE name ~ '[0-9]';
```

- Регистронезависимое совпадение:

```
1 SELECT *
2 FROM employees
3 WHERE name ~* 'alice';
```

- Замена вхождений:

```
1 SELECT regexp_replace(name, 'Ваня', 'Иван', 'g')
2 FROM employees;
```

Флаг `'g'` указывает на замену всех вхождений.

- Извлечение совпадений:

```
1 SELECT regexp_matches(name, '\b[Aa]\w+', 'g')
2 FROM employees;
```

В разных СУБД регулярные выражения работают немного по-разному, но основной принцип одинаков — поиск, замена и извлечение подстрок.

Пример регулярного выражения, проверяющий соответствие пароля указанным в выражении правилам:

```
1 ^(?=[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$
```

В данном выражении проверяется, что во введённом пароле присутствует:

- Как минимум одна заглавная буква `(?=.*[A-Z])`
- Как минимум одна строчная буква `(?=.*[a-z])`
- Как минимум одна цифра `(?=.*\d)`
- Как минимум один специальный символ `(?=.*[@$!%*?&])`

- Не менее восьми символов `[A-Za-z\d@$!%*?&]{8,}`

Тема регулярных выражений стоит немного в стороне, но при этом стоит того, чтобы в нее погрузиться. Предлагаем ознакомиться на [Хабре](#) со статьей от коллег из Росбанка.

Рекурсивные запросы

Рекурсия — это вызов функции самой себя. В SQL рекурсивные запросы позволяют собирать данные в цикле. Они полезны для построения иерархий, таких как организационные структуры или получение упорядоченных списков (например, всех дат за период).

Пример рекурсивного запроса в PostgreSQL:

```
1 WITH RECURSIVE numbers AS (  
2   SELECT 1 AS num  
3   UNION ALL  
4   SELECT num + 1  
5   FROM numbers  
6   WHERE num < 20  
7 )  
8 SELECT num FROM numbers;
```

Этот запрос создает список чисел от 1 до 20. Начальное значение — 1, затем в цикле добавляется 1, пока не достигнем 20.

Другой пример — получение всех дат за определенный период:

```
1 WITH RECURSIVE dates AS (  
2   SELECT '2027-01-01'::DATE AS date  
3   UNION ALL  
4   SELECT date + INTERVAL '1 day'  
5   FROM dates  
6   WHERE date < '2027-03-31'::DATE  
7 )  
8 SELECT date FROM dates;
```

Этот запрос возвращает все даты с 1 января по 31 марта 2027 года.

Вопросы для собеседования

1. Зачем используются представления?

Представления упрощают сбор данных из одних и тех же таблиц-источников, уменьшая количество одинакового кода.

2. Чем отличается материализованное представление от обычного?

Материализованное представление — это физическая таблица, которая обновляется по определённому событию, в отличие от обычного представления, которое является просто именованным запросом.

3. Что произойдёт, если изменится схема таблицы, на основе которой создано представление?

Представление сломается, и его нужно будет пересоздать, используя `DROP VIEW` и `CREATE OR REPLACE VIEW`.

Резюме

- Представление** — это таблица, которая не хранит данные, а извлекает их из других таблиц при необходимости. Таким образом, при использовании представления не происходит экономии вычислительных ресурсов. Представления используются для удобства и улучшения читаемости запросов.
- Материализованные представления** выглядят как обычные, но результаты запроса сохраняются в физическую таблицу (которая хранится на диске). Они поддерживаются не во всех СУБД, и работа с ними может существенно различаться.
- Оптимизация запросов** позволяет улучшить производительность систем, оптимизировать их совместное использование, а также сэкономить вычислительные ресурсы.

Основные способы оптимизации запросов:

- Индексы** — структуры данных, которые используются для ускорения поиска и сортировки данных в таблицах. Они создаются на одном или нескольких столбцах таблицы и позволяют быстро находить строки без необходимости сканировать всю таблицу.
- Анализ плана запроса** — позволяет понять, как выполняется запрос под капотом СУБД и ориентировочные затрачиваемые ресурсы (стоимость

запроса).

4. **Регулярные выражения** позволяют находить строки, соответствующие заданным условиям.
5. **Рекурсивные запросы** в SQL позволяют собирать данные в цикле. Они полезны для построения иерархий, таких как организационные структуры или получение упорядоченных списков (например, всех дат за период).



Урок 10 Типичные вопросы на собеседовании

1. Паттерны решения задач
2. Вопросы на собеседовании

Паттерны решения задач

В этом уроке разберем основные паттерны написания SQL-запросов, которые пригодятся как на собеседованиях, так и в работе.

Количество записей по дням

Первый паттерн позволяет определить, сколько записей поступило к нам в каждый из дней. Для этого используем следующий запрос:

```
1 SELECT date,  
2         COUNT(1)  
3   FROM orders  
4  GROUP BY date  
5  ORDER BY date DESC;
```

Этот паттерн позволяет заметить критическое падение количества записей в таблице. Мы группируем данные по дням, подсчитываем количество записей за каждый день и сортируем результат в обратном порядке, чтобы актуальные даты отображались первыми. Это помогает визуально определить дни, когда произошло уменьшение записей, и понять причины: может, к нам пришло больше клиентов или случился отток, или возникла проблема с интеграцией. Таким образом, мы сможем точнее определить, в какой день это произошло.

Обнаружение отсутствующих дат в последовательности

Более сложным вариантом паттерна выше является обнаружение дыр в последовательности.

```
1 WITH RECURSIVE dates AS (  
2     SELECT '2027-01-01'::DATE AS date  
3     UNION ALL  
4     SELECT date + INTERVAL '1 day'  
5     FROM dates  
6     WHERE date < '2027-03-31'  
7 )  
8 SELECT o.date,  
9        COUNT(1)  
10 FROM dates  
11 LEFT JOIN orders AS o  
12     ON dates.date = o.date  
13 GROUP BY o.date  
14 ORDER BY date DESC;
```

В этом запросе мы рекурсивно создаем список всех дат и накладываем на него таблицу с записями по конкретным дням. Разница с предыдущим запросом в том, что здесь мы увидим не только падение или увеличение количества строк, но и отсутствие записей за определенные дни. Это позволяет определить даты, в которые нет записей.

Посмотрим на наших данных, как это можно сделать.

Возьмем таблицу эпизодов. Если бы эпизоды выходили каждый день или по несколько раз в день, запрос мог бы выглядеть так:

```
1 SELECT date_trunc('month', air_date)::date AS month, COUNT(1)  
2 FROM episodes  
3 GROUP BY date_trunc('month', air_date)  
4 ORDER BY 1 DESC;
```

Этот запрос показывает все даты. Обратите внимание на форматирование даты — Redash интерпретирует дату как 01/09/21, хотя хранится она как 2021-09-01. Здесь мы видим, отсортированные в обратном порядке, месяцы и количество эпизодов, вышедших в каждый месяц.

Теперь создадим рекурсивный запрос:

```
1 WITH RECURSIVE date_series AS (  
2     SELECT DATE '2013-12-01' AS first_day
```

```

3  UNION ALL
4  SELECT (first_day + INTERVAL '1 month')::date
5  FROM date_series
6  WHERE first_day < DATE '2021-09-01'
7 )
8 SELECT first_day, COUNT(e.id)
9  FROM date_series
10 LEFT JOIN episodes AS e
11     ON first_day = date_trunc('month', air_date)::date
12 GROUP BY first_day
13 ORDER BY first_day DESC;

```

Первая дата выхода сериала — декабрь 2013 года, поэтому мы начинаем с 2013-12-01 и приводим к типу `date` для удобства. Затем мы добавляем к дате один месяц и выбираем дату последнего эпизода в базе, это сентябрь 2021 года. Таким образом, мы получаем список дат. Далее мы присоединяем к этим датам таблицу эпизодов с помощью `LEFT JOIN`, чтобы не потерять ни одной даты. Приведение даты эпизода к типу `date` возьмем из предыдущего запроса. После этого мы увидим не только месяцы, в которые выходили эпизоды, но и месяцы, в которые не было ни одного эпизода, что позволяет выявить дыры в данных.

Количество типов записей в поле

Следующий паттерн пригодится, если нам нужно одним запросом выяснить, сколько записей разных типов содержится в таблице. Например, у нас есть таблица пользователей, и мы хотим узнать количество пользователей с базовой и расширенной подпиской.

В запросе ниже мы хотим узнать, сколько мужчин и женщин воспользовались вашим продуктом:

```

1 SELECT COUNT(1) AS all_cnt,
2        SUM(CASE WHEN sex = 'male' THEN 1 ELSE 0 END) AS male_cnt,
3        SUM(CASE WHEN sex = 'female' THEN 1 ELSE 0 END) AS female_cnt
4  FROM people;

```

В первой строке мы возвращаем общее количество записей в таблице, чтобы проверить наличие пустых полей или полей с другими значениями. Далее мы используем конструкцию `CASE`: если пол пользователя равен `male`, мы ставим единицу, иначе — ноль; в третьем поле делаем наоборот. В итоге суммируем количество единиц. Так мы получаем общее количество пользователей, мужчин и женщин одним запросом, без `UNION` и группировок.

Антиджоин

Следующий паттерн часто называют антиджойн. При использовании левого или правого джойна мы соединяем одну таблицу с другой, но иногда нас интересуют строки, для которых нет соответствий во второй таблице. Это может понадобиться, например, для получения списка клиентов, которые не сделали ни одной покупки. В таком случае мы используем `LEFT JOIN` и исключаем строки, имеющие соответствия во второй таблице.

```
1 SELECT a.*
2   FROM table_a AS a
3  LEFT JOIN table_b AS b
4    ON a.id = b.id
5 WHERE b.id IS NULL;
```

Это и называют антиджойн. Его логику можно реализовать через `NOT EXISTS`. Хотя антиджойн не является официальным типом джойна, вы можете упомянуть его на собеседовании, если понадобится.

Дубли в таблице

Следующий паттерн позволяет найти дубли в таблице. Например, вы даете большую скидку на первую покупку, пользователь регистрируется и получает скидку. Но вдруг вы забыли сделать проверку на существующий e-mail в базе. Что делать? Сначала добавляем проверку при регистрации, а затем запускаем следующий запрос для выявления дублей:

```
1 SELECT email, COUNT(1)
2   FROM customers
3  GROUP BY email
4 HAVING COUNT(1) > 1;
```

Мы группируем строки по e-mail и оставляем только те, у которых количество больше единицы. Так вы получите список e-mail с дублями. Что делать с ними дальше, решать вам: можно объединить или удалить лишние записи.

Потерянные данные

Следующий паттерн разберем на примере. Представим, что у нас есть корзина пользователя, но кто-то случайно удалил ее. Мы хотим восстановить корзину из таблицы логов действий пользователей. Пример структуры такой таблицы:

date	user_id	item_id	in_out
------	---------	---------	--------

2024-03-02	1	2	in
2024-03-03	1	1	in
2024-03-05	1	1	out
2024-03-05	1	2	out
2024-03-08	1	4	in
2024-03-09	1	3	in
2024-03-11	1	2	in

У нас есть дата, `user_id`, `item_id`, и действие `in_out`, обозначающее добавление или удаление товара из корзины. В итоге мы хотим получить следующую таблицу:

user_id	item_id	in_out
1	2	in
1	1	out
1	4	in
1	3	in

Для каждого товара нас интересует последнее действие. Например, товар 2 был удален из корзины, а товары 1, 4 и 3 остались. Как это можно сделать проще всего? Есть несколько способов, включая использование аналитических функций, но наиболее простой подход следующий:

```

1 SELECT user_id, item_id,
2         SUBSTR(MAX(CONCAT(date::text, in_out)), 11)
3 FROM basket
4 GROUP BY user_id, item_id;
```

Мы берем корзину, группируем ее по `user_id` и `item_id`. Затем объединяем дату и поле `in_out` в текстовую строку. После этого ищем максимум среди этих значений. Очевидно, максимум определяется по дате. Когда мы нашли максимум, `SUBSTR` берет с десятого символа значение `in_out`, то есть нам не нужно искать, например, первую или последнюю запись, — мы просто сравниваем конкатенированные строки и берем максимальное значение. Так мы получаем информацию о последнем действии для каждого товара в корзине.

Вопросы на собеседовании

В каком порядке выполняются инструкции в SQL?

Внутри СУБД операторы в запросе выполняются в определенном порядке. Это происходит в следующем порядке:

1. **FROM:** движок СУБД сначала определяет, из каких таблиц будут извлекаться данные.
2. **ON:** затем обрабатываются условия соединения, заданные в части **ON**, где происходит фильтрация до выполнения самого соединения.
3. **JOIN:** выполняется операция соединения таблиц, объединяющая данные на основе указанных условий.
4. **WHERE:** на этом этапе происходит фильтрация строк, удовлетворяющих условиям запроса.
5. **GROUP BY:** строки группируются по заданным столбцам, чтобы впоследствии можно было применять агрегатные функции.
6. **HAVING:** применяется фильтрация к агрегированным данным, отсекаются группы, не удовлетворяющие условиям.
7. **SELECT:** извлекаются конкретные столбцы, указанные в запросе, для формирования итогового набора данных, при необходимости выполняются операции с данными.
8. **DISTINCT:** убираются дубликаты из результатов, если это необходимо.
9. **ORDER BY:** сортировка данных по указанным столбцам в нужном порядке.
10. **LIMIT/OFFSET:** окончательно определяются строки для вывода, ограничивается количество возвращаемых результатов и осуществляется выборка с указанием смещения.

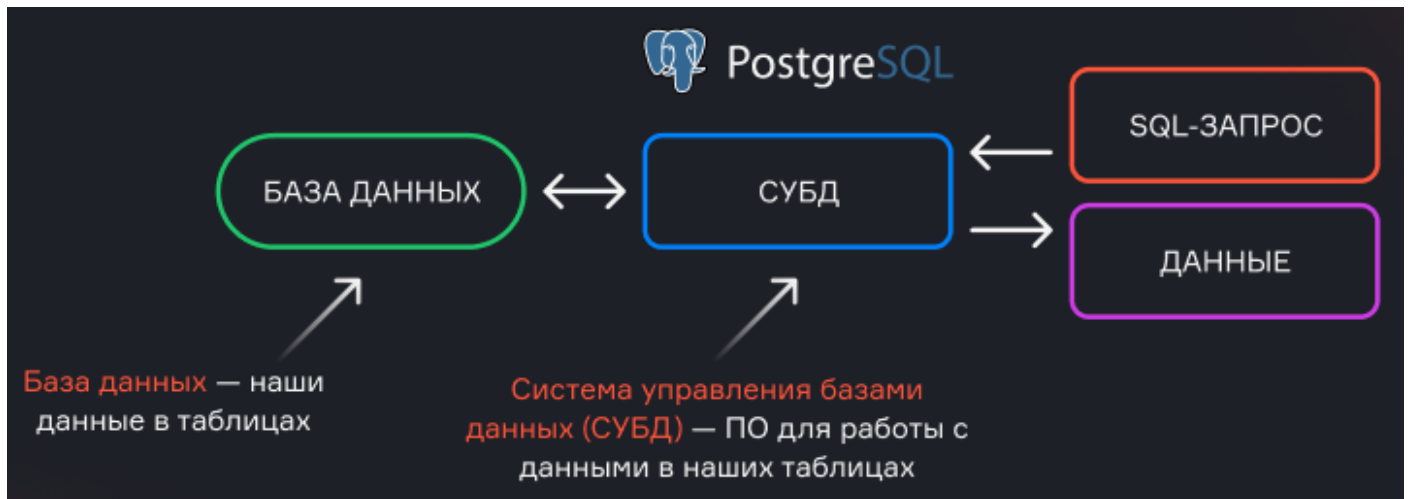
Эта последовательность объясняет, как движок СУБД обрабатывает запросы. Знание порядка выполнения инструкций позволяет оптимально структурировать запросы и эффективно обрабатывать данные.

Для упрощения понимания последовательности выполнения операторов в SQL-запросе представим, что мы работаем с двумя большими стопками документов. Сначала мы фильтруем и убираем ненужные страницы, затем объединяем их, группируем по определённым признакам, отмечаем важные элементы, и в итоге упорядочиваем и сокращаем по необходимости.

В чем отличие между базой данных и системой управления базами данных (СУБД)?

База данных и система управления базами данных (СУБД) — это два взаимосвязанных, но различных понятия:

- **База данных** — это организованная коллекция данных, которая хранится и управляется в цифровом формате. Это сами данные, с которыми ведётся работа.
- **СУБД** — это программное обеспечение, которое управляет базой данных и обеспечивает доступ к ней. СУБД предоставляет интерфейс для выполнения операций над данными, таких как создание, чтение, обновление и удаление. Она также заботится о таких аспектах, как целостность данных, управление транзакциями, обеспечение безопасности, управление доступом и выполнение запросов.



Примером СУБД является PostgreSQL, с которой мы уже работали. СУБД освобождает пользователя от необходимости заниматься низкоуровневыми задачами по управлению данными, позволяя сосредоточиться на их анализе и обработке.

Как реализуется связь «многие ко многим» в SQL?

Связь «многие ко многим» между двумя таблицами в реляционной базе данных реализуется с помощью третьей таблицы, называемой **таблицей связей** или **соединительной таблицей**.

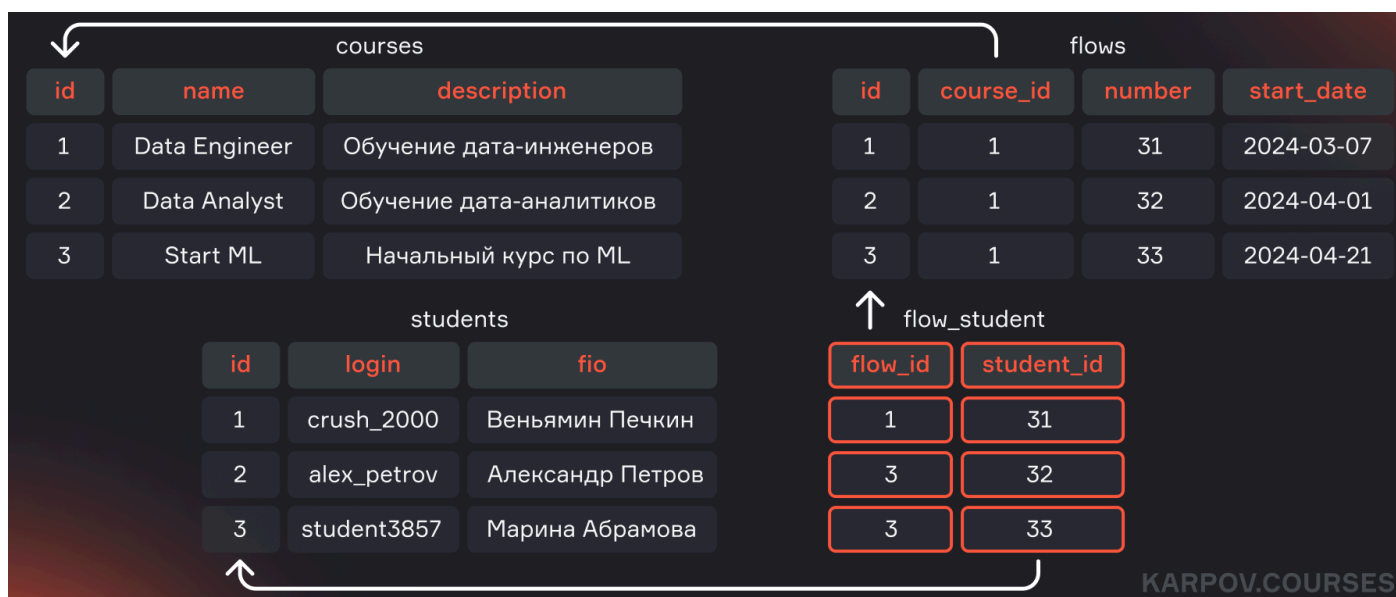
Вот как выглядят различные типы связей:

- **Один к одному:** каждая запись в одной таблице соответствует одной записи в другой таблице. В большинстве случаев такие таблицы можно объединить в одну без изменения количества строк.
- **Один ко многим:** каждая запись в родительской таблице может иметь множество связанных записей в дочерней таблице. Это классическая связь между таблицами, где, например, один заказчик может иметь несколько заказов.
- **Многие ко многим:** любая запись из одной таблицы может соответствовать множеству записей в другой таблице и наоборот. Для реализации этой связи добавляется третья таблица, которая содержит внешние ключи обеих таблиц, связывая их друг с другом.

Пример:

Предположим, у нас есть таблицы `students` (студенты), `courses` (курсы) и `flows` (потoki).

Каждый студент может записаться на несколько курсов и состоять в разных потоках. Для реализации связи «многие ко многим» создаётся таблица `flow_student`, которая содержит столбцы `student_id` и `flow_id`, представляющие собой внешние ключи, связывающие студентов и потоки.



Таким образом, связь «многие ко многим» организуется с помощью промежуточной таблицы, хранящей все возможные комбинации записей между двумя основными таблицами.

Зачем нужны алиасы?

Алиасы (или псевдонимы) используются для временного присвоения имен таблицам или столбцам в SQL-запросах для упрощения работы с данными и улучшения читаемости запросов. Алиасы бывают двух типов:

1. **Алиасы для столбцов** — позволяют переименовать столбцы в результирующем наборе данных, что облегчает восприятие и понимание результатов запроса. Например, вместо вывода столбца с именем `total_sales` вы можете использовать более понятное имя `Total Sales`.

```
1 SELECT total_sales AS "Total Sales"
2 FROM Sales;
```

2. **Алиасы для таблиц** — используются для сокращения имен таблиц, особенно в запросах с соединениями, где может быть несколько таблиц с одинаковыми именами столбцов. Алиасы облегчают написание запросов и понимание того, из какой таблицы берутся данные.

```
1 SELECT s.name, c.course_name
2 FROM Students AS s
```

```
3 JOIN StudentCourses AS sc ON s.student_id = sc.student_id
4 JOIN Courses AS c ON sc.course_id = c.course_id;
```

Использование алиасов в запросах повышает их читаемость и ясность, особенно при работе с большими и сложными наборами данных.

Как найти вторую по величине зарплату в таблице?

Для поиска второй по величине зарплаты в таблице можно использовать комбинацию операторов **ORDER BY**, **OFFSET** и **LIMIT**. Пример SQL-запроса:

```
1 SELECT salary
2 FROM Employees
3 ORDER BY salary DESC
4 OFFSET 1
5 LIMIT 1;
```

Этот запрос сортирует зарплаты в порядке убывания и пропускает первую строку (**OFFSET 1**), возвращая затем одну строку (**LIMIT 1**), то есть вторую по величине зарплату.

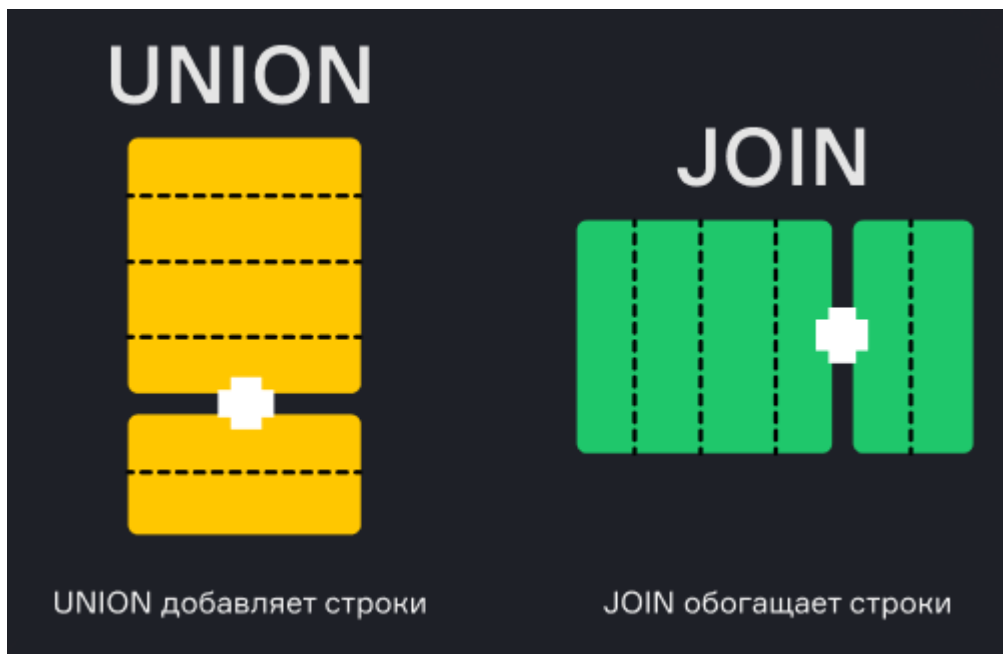
Однако важно уточнить у заказчика, как обрабатывать случаи, когда несколько сотрудников имеют одинаковую самую высокую или вторую по величине зарплату. Если таких сотрудников несколько, возможно, потребуется использовать аналитические функции, такие как **DENSE_RANK()** или **ROW_NUMBER()**, чтобы более точно идентифицировать вторую по величине зарплату:

```
1 SELECT DISTINCT salary
2 FROM (
3     SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rank
4     FROM Employees
5 ) ranked_salaries
6 WHERE rank = 2;
```

Этот запрос использует функцию **DENSE_RANK()**, которая присваивает ранги зарплатам, и выбирает те, которые имеют второй ранг.

Чем отличается **JOIN** от **UNION** ?

JOIN и **UNION** — это две различные операции, используемые в SQL для работы с несколькими таблицами, однако их применение и результаты отличаются.



- **JOIN** : объединяет строки из двух или более таблиц на основе логического условия, определенного через оператор **ON** . **JOIN** работает с горизонтальным объединением данных, обогащая строки из одной таблицы информацией из другой.

- **Пример использования JOIN** : объединение таблиц **Employees** и **Departments** для получения информации о сотрудниках и их отделах:

```
1 SELECT e.name, d.department_name
2 FROM Employees e
3 JOIN Departments d ON e.department_id = d.id;
```

В этом примере каждая строка в таблице **Employees** объединяется с соответствующей строкой из таблицы **Departments** на основе идентификатора отдела (**department_id**).

- **UNION** : объединяет результаты двух или более SQL-запросов в один результирующий набор. **UNION** работает с вертикальным объединением данных, складывая строки из разных запросов. При этом автоматически удаляются дубликаты строк, если они есть. Для сохранения всех строк, включая дубликаты, используется **UNION ALL** .

- **Пример использования UNION** : объединение списков клиентов из двух разных таблиц **Customers_2023** и **Customers_2024** :

```
1 SELECT customer_name, email FROM Customers_2023
2 UNION
```

```
3 SELECT customer_name, email FROM Customers_2024;
```

В этом случае данные из обоих запросов объединяются в один набор результатов, исключая дублирующиеся строки. **UNION** требует, чтобы количество и порядок столбцов в объединяемых запросах совпадали.

Какие виды **JOIN** кроме **OUTER** и **INNER** вы знаете?

В SQL помимо **OUTER** и **INNER** существует несколько видов операций соединения (**JOIN**), каждый из которых имеет свое предназначение и особенности:

- **CROSS JOIN** : создает декартово произведение двух таблиц, то есть каждая строка из первой таблицы соединяется с каждой строкой из второй таблицы. Результат содержит все возможные комбинации строк.

```
1 SELECT e.name, d.department_name
2 FROM Employees e
3 CROSS JOIN Departments d;
```

- **SELF JOIN** : это соединение таблицы с самой собой. Используется, например, для нахождения отношений между записями в одной таблице, таких как иерархии.

```
1 SELECT e1.name AS Employee, e2.name AS Manager
2 FROM Employees e1
3 JOIN Employees e2 ON e1.manager_id = e2.id;
```

- **Anti-Join**: этот тип соединения позволяет найти строки в одной таблице, для которых нет соответствующих строк в другой таблице. В SQL Anti-Join часто реализуется с использованием **LEFT JOIN** в комбинации с **WHERE** для отбора **NULL** значений.

```
1 SELECT Employees.name
2 FROM Employees
3 LEFT JOIN Departments ON Employees.department_id = Departments.id
4 WHERE Departments.id IS NULL;
```

Антиджоин официально не выделяют в качестве одного из типов соединений, однако это понятие бывает используется и следует его знать.

Чем **COUNT(1)** отличается от **COUNT(*)** и **COUNT(Name)** ?

В SQL функции `COUNT()` используются для подсчета количества строк или значений в наборе данных, но они имеют различные особенности в зависимости от параметров:

- `COUNT(*)` : подсчитывает количество строк в результирующем наборе, включая строки с `NULL` значениями. При использовании `COUNT(*)` движок обрабатывает все столбцы в таблице, смотря их значения.



```
1 SELECT COUNT(*) FROM Employees;
```

- `COUNT(1)` : работает аналогично `COUNT(*)` , подсчитывая количество строк в таблице. Однако движок СУБД уже проверять все столбцы, поэтому использование константы в качестве аргумента как правило приводит уменьшению времени выполнения запроса.



```
1 SELECT COUNT(1) FROM Employees;
```

- `COUNT(Name)` : подсчитывает количество непустых (`NOT NULL`) значений в указанном столбце `Name` . Это полезно, когда необходимо узнать количество заполненных записей в конкретном столбце.



```
1 SELECT COUNT(Name) FROM Employees;
```

Таким образом, `COUNT(*)` и `COUNT(1)` подсчитывают все строки, а `COUNT(Name)` подсчитывает только строки с непустыми значениями в столбце `Name` .

Чем отличается `WHERE` от `HAVING` ?

`WHERE` и `HAVING` — это операторы SQL, которые используются для фильтрации данных, но применяются на разных этапах обработки запроса:

- `WHERE` : используется для фильтрации строк до выполнения группировки данных и применения агрегатных функций. `WHERE` задает условия, которым должны удовлетворять строки из исходного набора данных.



```
1 SELECT * FROM Employees
```

```
2 WHERE department_id = 1;
```

- **HAVING** : применяется для фильтрации данных после выполнения группировки, то есть для отбора групп, удовлетворяющих определенным условиям. Используется в сочетании с **GROUP BY**.

```
1 SELECT department_id, COUNT(*) as num_employees
2 FROM Employees
3 GROUP BY department_id
4 HAVING COUNT(*) > 10;
```

Оба оператора могут быть использованы одновременно в одном запросе, если необходимо фильтровать данные как до, так и после группировки:

```
1 SELECT department_id, COUNT(*) as num_employees
2 FROM Employees
3 WHERE salary > 50000
4 GROUP BY department_id
5 HAVING COUNT(*) > 10;
```

Обратите внимание, что фильтрацию данных по неагрегированным значениям можно делать как в блоке WHERE, так и в блоке HAVING. Например, результат следующих запросов будет одинаковый:

```
1 SELECT sex, COUNT(user_id)
2 FROM users
3 WHERE sex != 'male'
4 GROUP BY sex
```

```
1 SELECT sex, COUNT(user_id)
2 FROM users
3 GROUP BY sex
4 HAVING sex != 'male'
```

Однако делать фильтрацию по неагрегированным данным рекомендуется именно в блоке WHERE, т.е. заранее. В таком случае ненужные данные убираются из расчётов ещё до группировки и не расходуются вычислительные ресурсы на подсчёт значений, которые всё равно будут отфильтрованы вами позже. Это важный момент, касающийся оптимизации SQL-запросов.

Что такое подзапрос и когда его следует использовать?

Подзапрос — это запрос, который выполняется внутри другого запроса. Он используется для работы с данными, которые были получены или вычислены в другом запросе. Подзапросы могут быть полезны в ситуациях, когда необходимо выполнить дополнительную обработку или фильтрацию данных, которые уже были извлечены или агрегированы. Хотя большинство подзапросов можно заменить на объединения (join), иногда это приводит к таким сложным и запутанным конструкциям, что их становится трудно читать и понимать.

Пример — получение имён сотрудников с самой высокой зарплатой:

```
1 SELECT name
2 FROM Employees
3 WHERE salary = (SELECT MAX(salary) FROM Employees);
```

Подзапросы являются мощным инструментом, так как они позволяют легко использовать результаты одного запроса в другом, и их можно применять практически в любой части SQL-запроса.

Чем отличаются агрегатные, аналитические и оконные функции?

Агрегатные функции служат для вычисления агрегированных значений, таких как сумма, количество, среднее значение или максимум. Для этого данные группируются по одному или нескольким полям, и затем над каждой группой выполняется вычисление агрегата. Примером может быть нахождение общей суммы продаж по каждому региону.

Аналитические и оконные функции — это на самом деле один и тот же тип функций в SQL, и между ними нет различий. Эти функции позволяют выполнять сложные вычисления и добавлять результаты к каждой строке в наборе данных без потери информации о самих строках. В отличие от агрегатных функций, которые сокращают число строк в результате запроса, аналитические и оконные функции возвращают все строки исходной таблицы, добавляя к каждой строке результаты вычислений. Например, они могут быть использованы для расчета скользящего среднего, ранжирования или кумулятивной суммы.

Чем отличаются DELETE и TRUNCATE?

Операции **DELETE** и **TRUNCATE** используются для удаления данных из таблиц, но функционируют они по-разному.

- **DELETE** удаляет строки из таблицы построчно и обычно используется с условиями фильтрации для удаления определённых записей. Вы можете удалить все записи из таблицы с помощью **DELETE**, но этот процесс будет медленнее, поскольку каждая строка удаляется отдельно, что требует больше времени и ресурсов.

- **TRUNCATE**, с другой стороны, очищает всю таблицу целиком и сразу, фактически «выбрасывая» все её содержимое. Это более быстрый и эффективный метод удаления всех данных из таблицы, так как не происходит построчного удаления. **TRUNCATE** следует использовать, когда необходимо полностью очистить таблицу от данных без сохранения информации о конкретных удалённых строках.

На этом теоретическая часть подошла к концу. Нужно помнить, что SQL, как и другие инструменты дата-инженера, требует практики. Поэтому настоятельно рекомендуем к прохождению наш бесплатный [симулятор SQL](#), который позволит повторить пройденный материал и закрепить знания. Пусть освоенные в этом блоке навыки помогут успешно пройти любые собеседования и придадут уверенности на новом рабочем месте.



> Конспект > 1 урок > Реляционные и МРР Базы данных. Что и как в них хранить

> Где и как хранить много однотипной информации?

Задачи:

- Сохранить много данных
- Быстро добавлять новые данные
- Быстро выполнять расчеты над данными и получать результаты
- Гарантировать целостность и непротиворечивость данных
- Иметь возможность восстановиться после сбоя
- Построить КХД

Способы решения:

- Записать все на бумагу и создать несколько копий
- Сохранить все в файлы на диске
- Записать данные в библиотеку магнитных лент
- Использовать Excel

- Использовать Базы Данных и Системы Управления Базами Данных

> Базы Данных и Системы Управления Базами Данных

> Базы Данных (БД)

База данных - это организованная коллекция данных хранящаяся и доступная в электронном виде при помощи вычислительных машин.

Элементы базы данных:

- Модель
- Логическая структура
- Объекты БД
- Язык определения данных DDL (Data Definition Language)
- Язык изменения данных DML (Data Manipulation Language)

> Система Управления Базами Данных (СУБД)

Система Управления Базами Данных - комплекс программ, позволяющих создать БД и манипулировать данными.

СУБД обеспечивает:

- Безопасность
- Надежность хранения
- Целостность данных

Основные функции:

- Управление данными на внешних дисках
- Управление данными в оперативной памяти
- Журнализация изменений
- Резервное копирование
- Восстановление БД после сбоев

- Поддержание языков БД DDL и DML

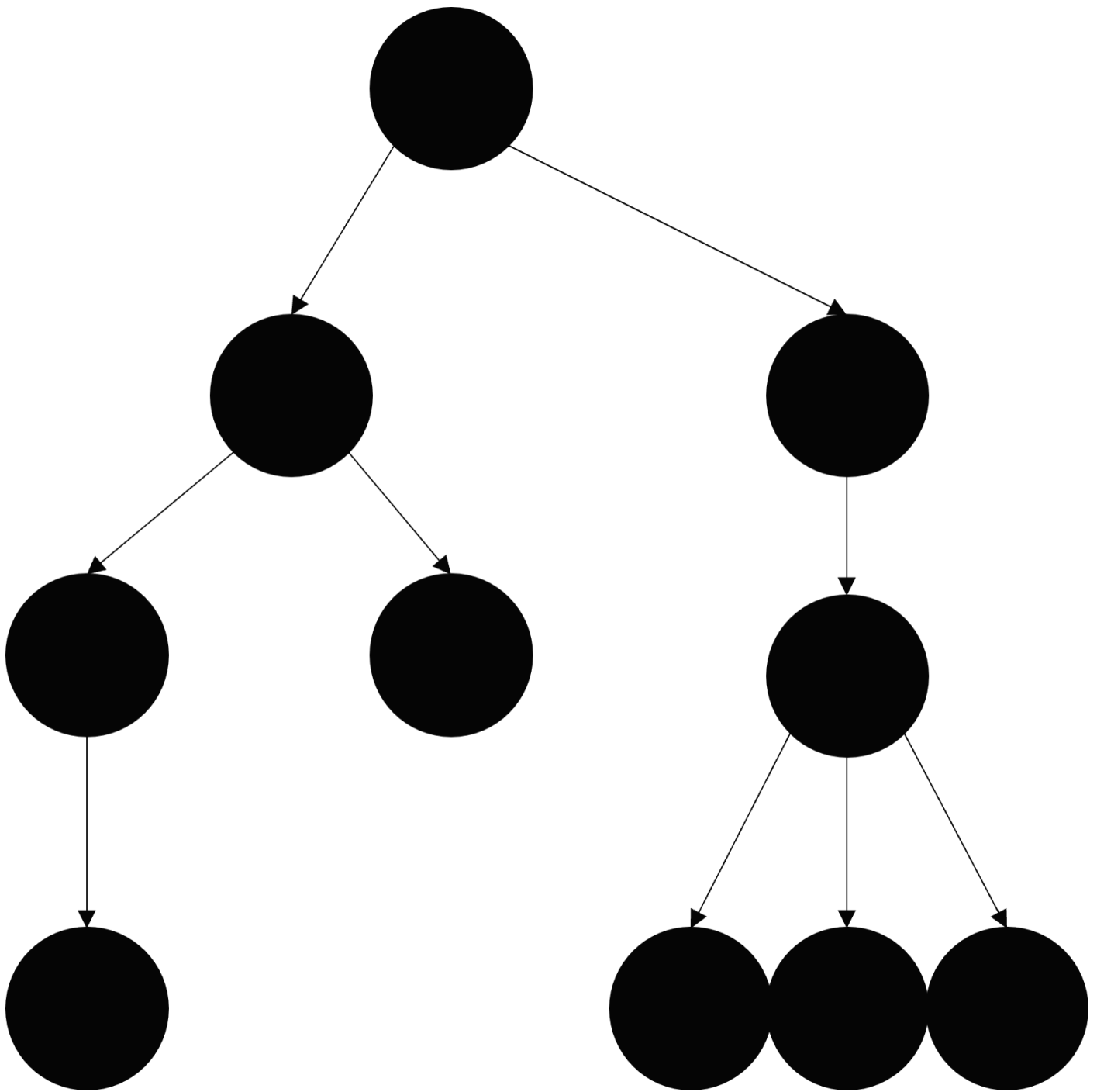
Компоненты СУБД:

- Ядро. Управляет всеми процессами находящимися в БД и управляет всеми приложениями входящими в ее состав.
- Процессор языка БД (оптимизатор). Интерпретирует язык запроса пользователя к БД. Также строит план выполнения запроса и возвращает его результат.
- Подсистема поддержки времени выполнения. Позволяет выполнять все пользовательские запросы и отдавать результаты. Кроме этого позволяет пользователям работать параллельно.
- Сервисные программы. Набор дополнительных компонентов, которые расширяют функционал БД. Например, функция резервного копирования.

> Модели Баз Данных

> Иерархическая

Элементы организованы в структуры, связанные между собой иерархическими или древовидными связями. Родительский элемент может иметь несколько дочерних элементов. Но у дочернего элемента может быть только один предок.



Особенности:

- Простая для понимания структура
- "Дешевая" навигация по узлам и связям
- Жесткая структура

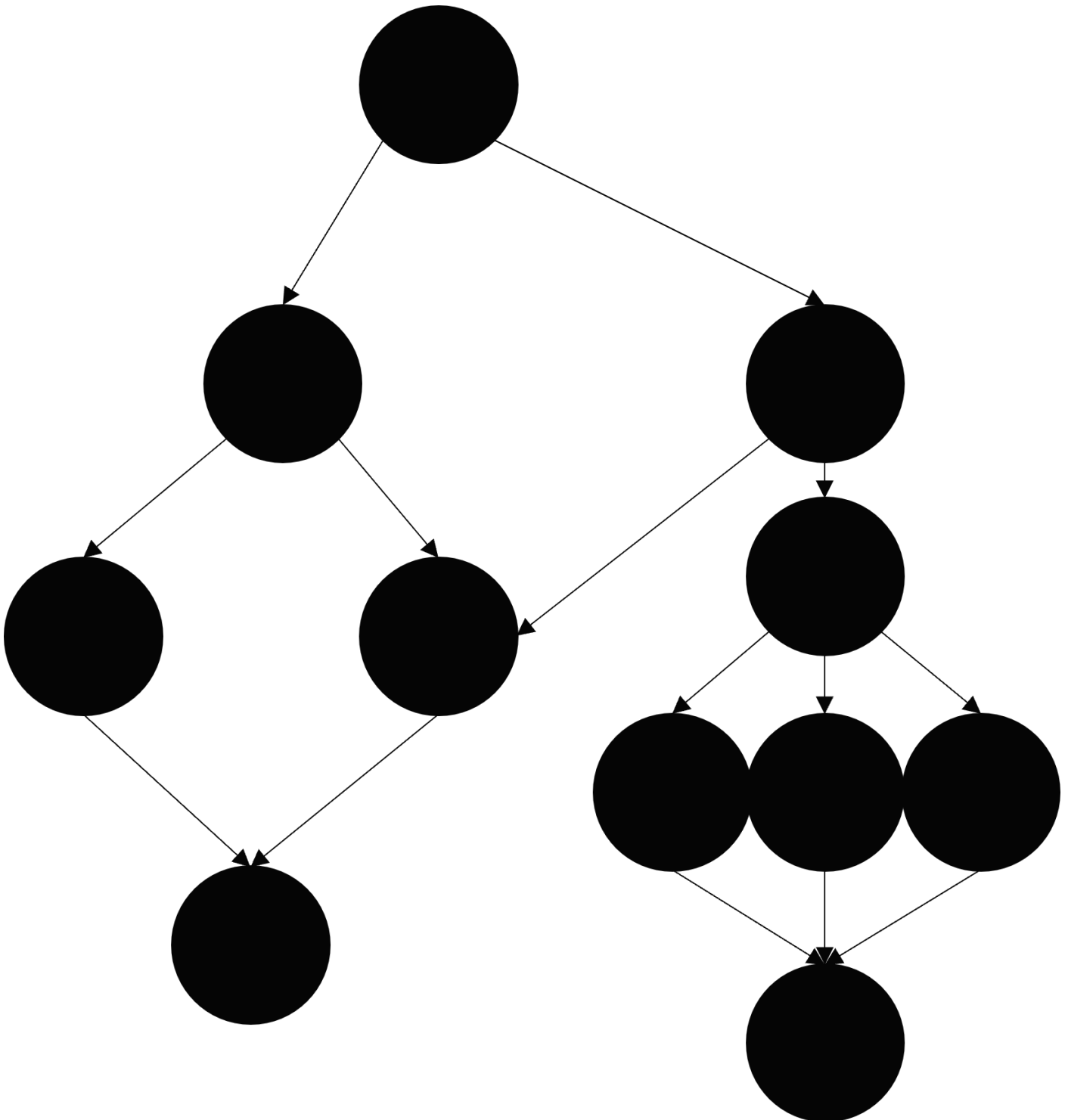
Примеры:

- IMS
- Windows Registry
- TDMS

- Cache
- Файловые системы

> Сетевая

Модель сетевой базы данных позволяет каждой записи иметь несколько родителей и несколько дочерних записей, которые, когда они визуализируются, принимают форму сетевой структуры сетевых записей.



- Простая для понимания структура

- Эффективные вычисления
- Сложна для изменений

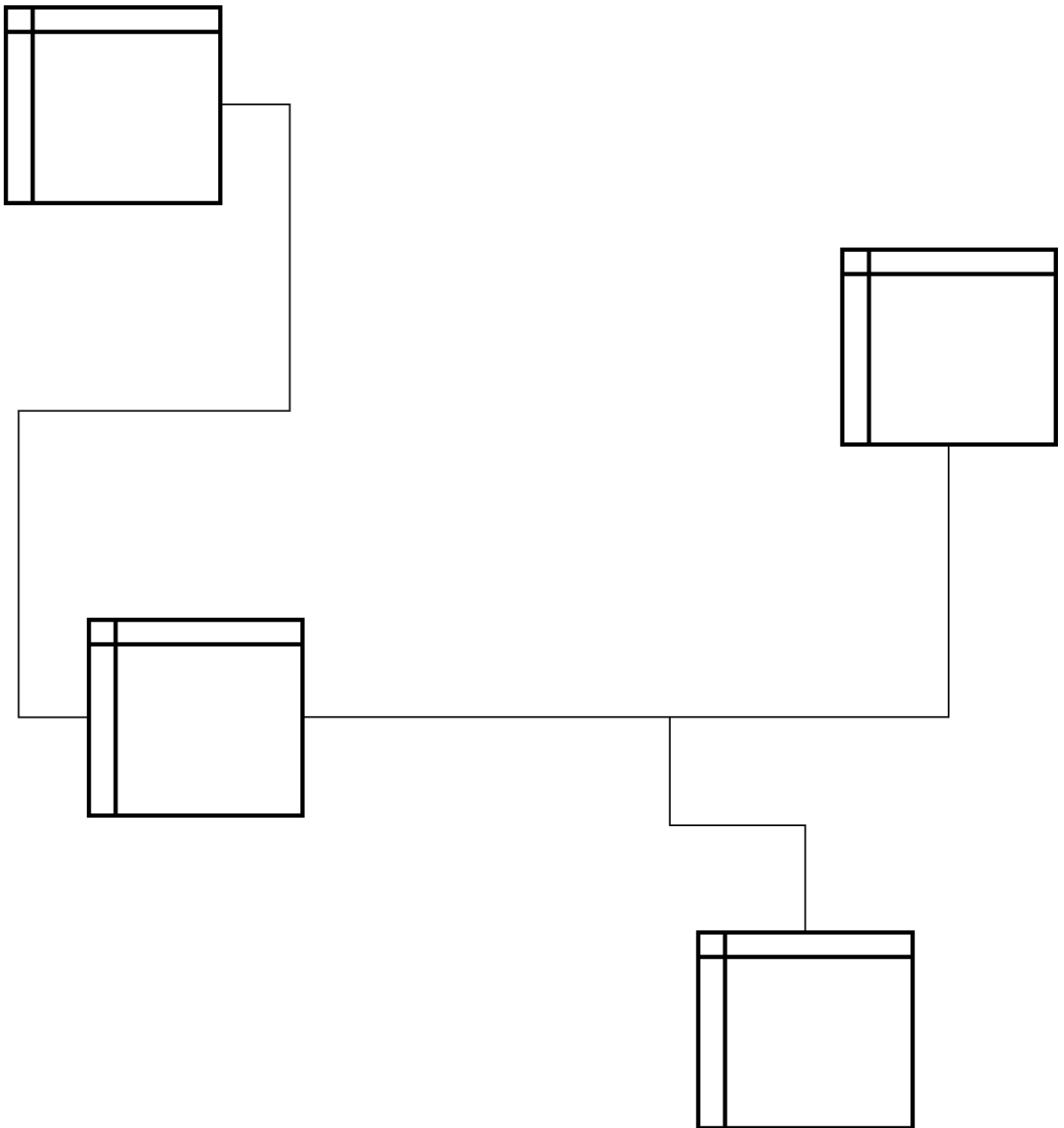
Примеры:

- IDS, IDS/2
- IDMS (Cullinet)
- DMS-1100, DMS-90 (UNIVAC)
- DBMS-10 (DEC)
- IMAGE/3000 (HP)
- UDS (Siemens)

> Реляционная

В реляционной модели, как объекты, так и их отношения представлены только таблицами. Основана на реляционной алгебре и включает в себя:

- Отношения/Relations
- Поля/Колонки/Атрибуты/Columns
- Строки/Rows/Tuples



Свойства ACID:

Atomicity - Атомарность. Если мы что-то запустили (транзакции), то они либо все выполняются, либо все не выполняются.

Consistency - Согласованность. Все изменения в БД приводят ее к согласованности, т.е. все измененные данные в одной таблице и связанные с другими таблицами будут меняться вместе и связно. Зафиксируются только допустимые результаты.

Isolation - Изолированность. Все транзакции изолированы друг от друга. В момент когда выполняется какая-то транзакция последовательно меняющая данные в нескольких объектах таблиц, никакая

другая транзакция не может увидеть изменения пока они не завершились. Все транзакции видят согласованное состояние БД.

Durability - Устойчивость. Позволяет БД оставаться согласованной в момент сбоя.

Достоинства:

- Простота и доступность для понимания конечным пользователям
- Применение математического аппарата реляционной алгебры
- Полная независимость данных
- Изоляция физической структуры от логической
- SQL
- ACID
- Идеально для КХД

Недостатки:

- "Дорогой" доступ к данным
- Большое кол-во отношений
- Не все данные "красиво" зайдут в реляционную модель
- Высокая стоимость владения для Корпоративных решений

> NoSQL

- Графовые - вершины, ребра и их свойства.
- Объектно-ориентированные
- Ключ-значение - являются по сути словарем, позволяющим извлечь однозначное значение по ключу.
- Семейство столбцов - данные хранятся по столбцам, а не по строкам.
- Документные - похожи на ключ-значение, только значения с разметкой (XML, JSON), которая и образует "документ".

> PostgreSQL

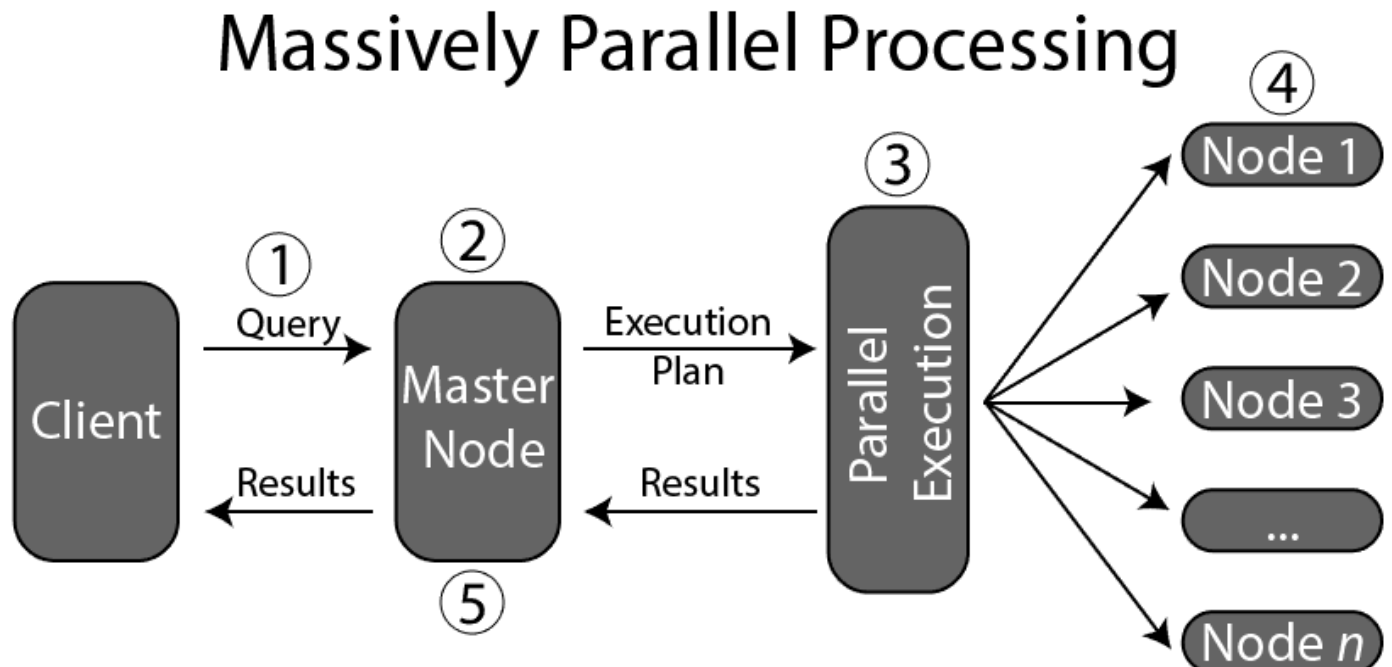
- OpenSource проект
- Динамически развивается и исправляется

- Поддержка любого размера БД
- Высокая совместимость с SQL2016
- Доступны несколько вариантов кластеров и реплик
- Поддержка дополнительных языков
- Встроенная поддержка SSL

> MPP

MPP (Massive Parallel Processing) или Массово-параллельная архитектура - это параллельные вычисления на нескольких серверах объединенных в один кластер.

- Данные распределены по нескольким row / column store сегментам кластера
- Обработка данных как можно ближе к месту их хранения
- Мощный интерконнект для обмена порциями данных
- Shared Nothing архитектура
- Идеально для OLAP / DWH



Достоинства:

- Быстрота обработки больших объемов данных
- SQL Conformance
- Легкое горизонтальное масштабирование

- Отказоустойчивость за счет создания релик/зеркал
- Стандартное Hardware для сегментов
- Row / Column store
- Сжатие хранимых данных
- Параллельная загрузка частей данных напрямую в шарды
- Cloud Compatibility. Совместимость с облачными решениями.

Недостатки:

- Высокие требования к элементам инфраструктуры
- Низкая производительность на OLTP профиле нагрузки
- Ограничения в поддержке всех функций SQL
- Возможности возникновения перекосов данных
- Специфическая поддержка и обслуживание

> GreenPlum

- OpenSource проект
- Динамически развивается и исправляется
- Высокая совместимость с PostgreSQL
- Поддержка разных языков (Python, R, C)
- Поддержка SSL
- Легкая масштабируемость

> Глоссарий

Шардинг — метод разделения и хранения единого логического набора данных в виде множества баз данных. Другое определение шардинга — горизонтальное разделение данных.

Нода - объект в базе данных, узел графа (например, сервер).

Кластер - это группа серверов (именуемых "нодами"), которые работают вместе, выполняют общие задачи и клиенты видят их как одну систему.

Репликация - поддержания двух (или более) наборов данных в согласованном состоянии.

DDL (Data Definition Language) - это группа операторов определения данных. Другими словами, с помощью операторов, входящих в эту группы, мы определяем структуру базы данных и работаем с объектами этой базы, т.е. создаем, изменяем и удаляем их.

DML (Data Manipulation Language) - это группа операторов для манипуляции данными. С помощью этих операторов мы можем добавлять, изменять, удалять и выгружать данные из базы, т.е. манипулировать ими.



> Конспект > 2 урок > Объекты баз данных. Зачем и что используется

> Таблица

Таблица - основной объект хранения данных в базе данных. Она состоит из:

- столбцов определенных типов;
- строк;
- данные в ячейках.

Столбцы в БД должны иметь уникальные имена.

Виды таблиц.

Постоянная таблица.

Хранится на диске, журналируется, её свойства - атомарность, конститетность, изолированность и устойчивость. Особенности - в PostgreSQL это одна таблица, GreenPlum представляет собой набор небольших таблиц, и умеет хранить данные построчно и по колонночно. Каждый столбец и каждая колонка будет сохранена в свой отдельный файл, это позволяет получить выигрыш при чтении данных с диска.

Временная таблица - применяется в процессе вычислений, создается “на лету” и после окончания вычислений может быть уничтожена. Временная таблица может существовать во время одной транзакции. Особенность - операции по сбору статистики или сжатия должны выполняться разработчиками самостоятельно. В GreenPlum для удаления временных таблиц нужно выполнять отдельный процесс.

Нежурналируемая таблица - это таблица, изменения в которой не записываются в лог базы данных. Это ускоряет взаимодействие пользователя с данными, но при перезагрузке или сбоях данные в этой таблице будут потеряны, останутся только атрибуты. Таблица существует в СУБД PostgreSQL.

Внешняя таблица - это специальный объект, для которого в базе данных хранится только описание или метаданные (метаинформация). Доступ к таким данным осуществляется с помощью специальных драйверов. Для пользователей внешняя таблица выглядит как обычная таблица, но для СУБД это отдельный объект. Поэтому доступ к такой таблице может занимать гораздо больше времени.

Представления - это виртуальная (логическая) таблица, представляющая собой поименованный запрос (синоним к запросу), который будет подставлен как подзапрос при использовании представления. Таблица позволяет получать результаты действий над другими таблицами и пользователям результат представляется “на лету”. Они (представления) обычно хранятся в системе управления базами данных в виде SQL-запроса.

В отличие от обычных таблиц реляционных баз данных, представление не является самостоятельной частью и хранится в виде своего определения. Содержимое представления динамически вычисляется на основании данных, находящихся в реальных таблицах. Изменение данных в реальной таблице базы данных немедленно отражается в содержимом всех представлений, построенных на основании этой таблицы.

Отличие **GreenPlum** - это ключ дистрибуции. При создании таблиц или представлений необходимо его указывать, чтобы данные были разделены между сегментами.

Справка: ключ дистрибуции

Distribution Key, он же ключ распределения - важный элемент в обеспечении равномерности распределения данных по сегментам MPP СУБД и как следствие обеспечения эффективности ее работы.

Ключ дистрибуции указывается при создании таблицы и может быть изменен при необходимости.

GreenPlum (вы еще можете встретить сокращение GP) может использовать Hash distribution алгоритм при указании ключа (1 или несколько полей) или при автоматическом выборе ключа (первая колонка таблицы), если это явно не указано в DDL. Также может быть указан вариант распределения Randomly, когда строки распределяются по сегментам по кругу по порядку - обычно самое равномерное распределение.

В качестве ключа дистрибуции лучше выбирать поле, по которому по большим объектам будет происходить JOIN, желательно целочисленного типа, желательно без NULLS. Если у таблицы есть Primary Key - это лучший кандидат.

В случае возникновения сильного перекоса лучше сделать Distributed Randomly.

> Типы данных

Типы данных - доступные способы представления данных в БД. Каждый тип данных - это и набор операторов над этим типом данных, позволяющие осуществлять разные действия с данными.

Какие существуют типы данных:

- числовые
- символьные
- двоичные
- логические (истина или ложь)
- дата и время
- перечисляемый
- массивы
- составные
- диапазоны
- геометрические
- сложносоставные
- пользовательские

Массив - тип данных , который сохраняет несколько значений одного и того же подтипа.

Составной тип данных - в одном элементе сочетаются разнотипные данные.

Диапазоны - тип данных, при помощи которого можно хранить и обрабатывать диапазон дат, значений.

Сложносоставные типы данных, например xml, json - при помощи данных типов PostgreSQL и GreenPlum реализуют возможности noSQL БД, расширяя зону своего присутствия.

> Ограничения целостности

Ограничения целостности - это набор инструментов, позволяющих БД выполнять требования консистентности.

Какие бывают ограничения целостности:

Первичный ключ (Primary Key) - позволяет отслеживать отсутствие одинаковых значений в таблице по первичному ключу. Он может состоять из нескольких колонок таблицы, которые не должны повторяться.

Внешний ключ (Foreign Key) - он позволяет связывать таблицы между собой. Он не позволяет удалить запись из родительской таблицы, если хоть одна запись из дочерней таблицы ссылается на эту запись.

Уникальность (Unique) - это свойство требует уникальные значения в каком-либо столбце таблицы.

Обязательность (Not NULL) - в таблицу не может быть записана строка, в которой ограничение целостности Not NULL пустое.

Проверка (check) - в проверку могут быть заложены любые пользовательские условия, например ограничение по размеру информации или по типу данных.

Исключение (exclude) - позволяет проверять, что вставляемые в таблицу значения, уже не попадают в какой-либо диапазон. Это ограничения удобно применять со специфическими индексами GiST и SP-GiST

Справка.

Консистентность - это согласованность данных друг с другом, целостность данных, а также внутренняя непротиворечивость.

> Индексы

Индекс - это объект базы данных, создаваемые с целью повышения производительности поиска данных.

Они требуют: - дополнительного места- дополнительных ресурсов в словаре данных

- больше ресурсов на Insert/Update
- отдельное обслуживание

Какие бывают индексы.

BTree

BTree - это сбалансированное дерево поиска с несколькими ветвями, разработанное для дисков или других устройств хранения. (По сравнению с двумя вилками, каждый внутренний узел B-дерева имеет несколько ветвей, то есть несколько вилок). [Подробнее.](#)

Hash

Hash (PostgreSQL only) - позволяет построить Hash-таблицу по заданному полю и производить операции над таблицами, основываясь на этом Hash-значении. ****Недостатки - они не передаются

в логи и не попадают в реплики базы данных

GiST

GiST (Generalized Search Tree) - позволяет осуществлять поиск по любым значениям.

SP-GiST

SP-GiST (Space-partitioned GiST) - SP-GiST представляет собой дерево поиска, ветви которого не пересекаются (в отличие от GiST).

GIN

GIN - индекс, который позволяет производить поиск по технологии “ключ-значение”, которые создаются пользователями.

BRIN

BRIN (Bitmap Range Index, работает только в PostgreSQL) - применим к полям с повторяющимися значениями. Занимает мало места, но поиск по нему занимает большое время, по сравнению с поиском бинарного дерева.

Bitmap

Bitmap индекс (GreenPlum only) - позволяет индексировать поля с повторяющимися значениями. Эти значения хорошо укладываются в битовую карту. Она позволяет получать строки или диапазон строк в поиске очень быстро.

> Последовательность и триггеры

Последовательности (Sequences)

Это специальные объекты, позволяющие получать уникальные и генерировать последовательность уникальных номеров в условиях многопользовательского асинхронного доступа. Обычно элементы последовательности используются для уникальной нумерации элементов таблиц (строк) в операциях модификации данных.

Это может быть отдельный объект или поле типа SERIAL.

Для последовательности можно задать начальное значение. Последовательности не гарантируют получение строго идущих друг за другом идентификаторов, они гарантируют уникальность полученных идентификаторов.

Триггеры

Это процедуры вызываемые при наступлении определенного события внутри базы данных (вставки, создания, удаления, обновления записей).

Триггер позволяет выполнить пользовательскую процедуру и сохранить данные в любую другую таблицу.

> Пользовательские функции

Пользовательские функции – программируемый объект базы данных, возвращающий значение predeterminedного типа. Это могут быть обработчики для триггеров или программировать операторы базы данных. Пользователь может писать функции на:- PL\PGSQL

- PL\Python
- PL\Perl
- PL\TCL
- C

> Партиции и правила

Партиции

Партиции - это разделение хранимых объектов баз данных (таких как таблиц, индексов, материализованных представлений) на отдельные части с раздельными параметрами физического хранения. Используется в целях повышения управляемости, производительности и доступности для больших баз данных.

Партиции позволяют разбивать таблицы по:

- датам
- диапазонам дат/времени
- список значений
- диапазон значений

Правила

Правила - объект, который позволяет переписывать SQL-запросы “на лету”.

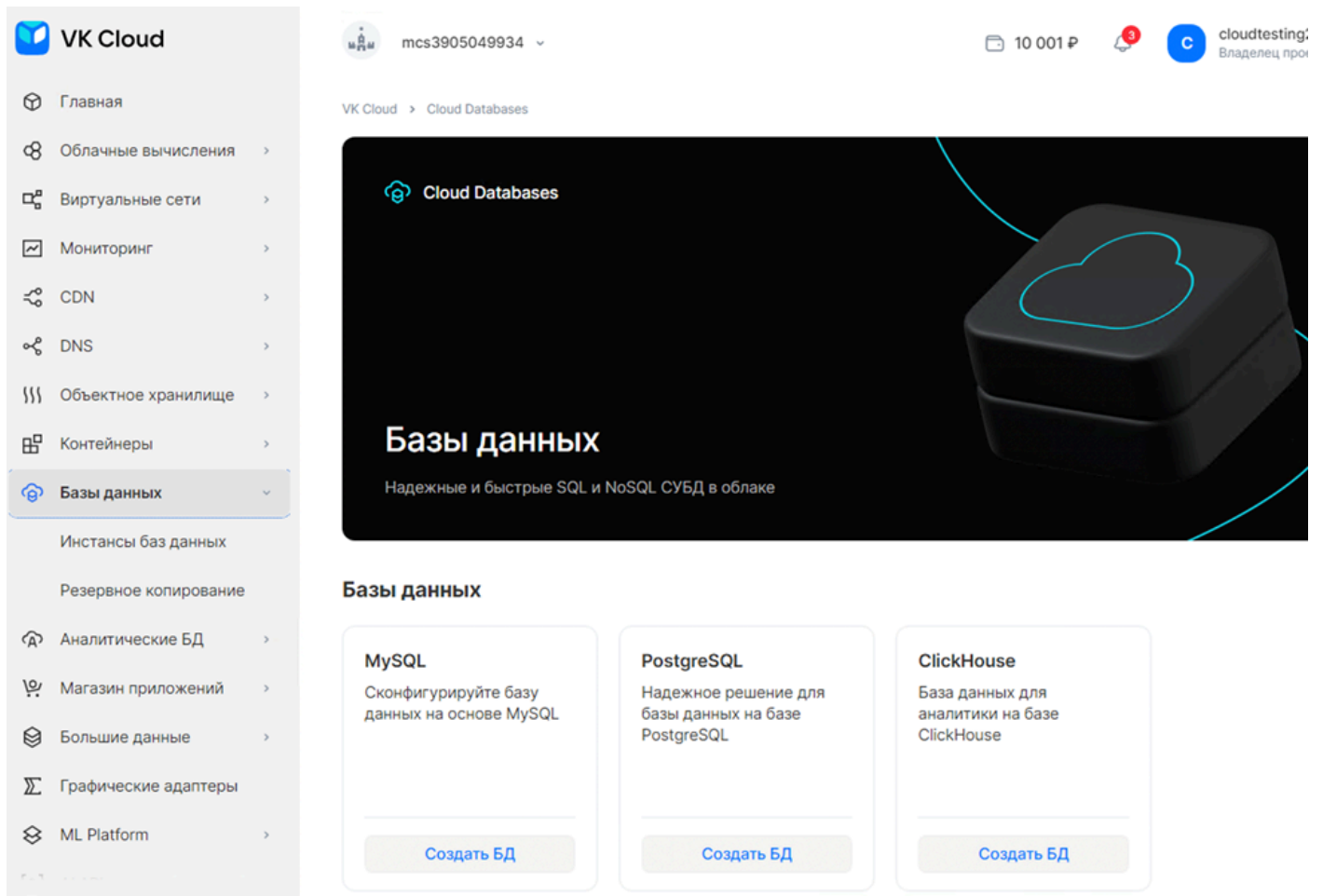
Набор правил, при получении запроса от парсера преобразовать его нужным образом и отправить планировщику переписанный запрос. Правила дают возможность гибко настраивать реакцию базы данных на SQL-запросы.



> Конспект > 3 урок > Подключение к PostgreSQL И GreenPlum. Работа со словарем данных.

> Создание инстанса PostgreSQL в VK Cloud

1. Для того чтобы создать БД заходим в раздел Базы Данных и кликаем на Создать БД на иконке PostgreSQL



2. Выбираем PostgreSQL, версия 14, конфигурация Single нам подходит. Нажимаем кнопку – следующий шаг

 Tarantool Версия Tarantool 2.10 ▾	 PostgreSQL Версия PostgreSQL 14 ▾	 ClickHouse Версия ClickHouse 22.8 ▾	 PostgresPro Enterprise Версия Postgres Pro ... ▾
 redis Версия Redis 5 ▾	 mongoDB Версия MongoDB 4.0 ▾		

Конфигурация

Single Единичный инстанс для разработки и тестирования. 	Master-Replica Реплики данных для максимальной скорости чтения и бесперебойной работы. 	Кластер Отказоустойчивый кластер с синхронной репликацией данных для максимальной доступности. 
--	---	--

3. Выставляем размер диска 10 Гб, отключаем автомасштабирование диска и выставляем флажок «Назначить внешний IP». Нажимаем кнопку «Следующий шаг».

Создание новой базы данных

✓ Конфигурация > 2 Создание инстанса > 3 Инициализация >

Имя инстанса базы данных

PostgreSQL-4150

Категория виртуальной машины ⓘ

Все актуальные типы виртуальных машин

Тип виртуальной машины

STD2-2-8

Intel Cascade Lake

2 CPU

8 ГБ RAM



Зона доступности ⓘ

Москва (GZ1)

Тип диска ⓘ

SSD

High-IOPS SSD

Пропускная способность МБ/с

Чтение ⓘ

31

Запись ⓘ

16

IOPS Операций в секунду

Чтение ⓘ

1 000

Запись ⓘ

500

Показатели верны для размера блока 64КБ. [Подробнее](#)

Размер диска, GB ⓘ

–

10

+

☐ Включить автомасштабирование диска

Сеть

PostgreSQL-7586: subnet_6820

mcs.local ▾

☒ Назначить внешний IP ⓘ

Настройки Firewall ⓘ

default ▾

☐ Создать реплику ⓘ

Ключ для доступа по SSH

CentOS-STD2-4-8-20GB-7lvJeMuy ▾

Резервное копирование

Отключено

Point-in-time recovery

Полное

Периодичность резервного копирования

Раз в сутки ▾

☐ Включить стратегию хранения полных бэкапов (GFS) ⓘ

4. Вводим имя БД, имя пользователя, пароль пользователя и нажимаем кнопку «Следующий шаг».

Создание новой базы данных

✓ Конфигурация > ✓ Создание инстанса > 3 Инициализация

Тип создания

Новая база данных

Восстановить из копии

Имя базы данных для создания

training_pg

Имя пользователя ⓘ

user1

Пароль пользователя

efd6t+14vgqazerT

Сгенерировать



Сохраните пароль. Возможность восстановления пароля не предусмотрена. Допустимо использовать только [разрешенные символы](#).



Создать базу данных

Предыдущий шаг

5. Ждем пока БД будет создана. Это может занять до получаса.

Создание новой базы данных

✓ Конфигурация > ✓ Создание инстанса > ✓ Инициализация > 4 Завершение



Создается инстанс СУБД
Устанавливается приложение.

6. Получаем готовую БД

VK Cloud > Cloud Databases > Инстансы баз данных > PostgreSQL-4150

PostgreSQL-4150

Информация Список баз данных Параметры баз данных Пользователи Расширения Мониторинг

Параметры инстанса СУБД		Значение
ID СУБД		2fc77dd1-708c-48e8-b62a-fd32f1e06b15
Тип СУБД		PostgreSQL 14
Тип репликации		Master-Replica
Количество slave-узлов		0
Объем диска		10 ГБ
Объем WAL-диска (журнал предзаписи)		2 ГБ
Внутренний IP-адрес		10.0.0.12
Внешний IP-адрес		212.233.75.80
Зона доступности		Москва (GZ1)
Цена		4 020 ₽ / месяц

> PSQL

psql — это терминальный клиент для работы с PostgreSQL. Он позволяет интерактивно вводить запросы, передавать их в PostgreSQL и видеть результаты. Также запросы могут быть получены из файла или из аргументов командной строки.

> Установка на Linux (CentOS):

Добавим репозиторий с нужной версией PostgreSQL:

```
1 sudo yum install -y https://download.postgresql.org/pub/repos/yum/reposrpms/EL-7-x86_64/pgdg-redhat-repo-latest.noarch.rpm
```

Для установки клиента нужно выполнить команду:

```
1 sudo yum install -y postgresql12
```

Для подключения к облачному PostgreSQL нужно будет выполнить следующую команду:

```
1 psql -h <адрес хоста> -d <название БД> -U <имя пользователя> -p <порт>
```

После будет предложено ввести пароль. И в случае успешного ввода произойдет подключение к PostgreSQL.

Для проверки успешности соединения можно узнать версию базы с помощью команды:

```
1 SELECT version();
```

Если все выполнилось успешно, тогда вы можете начать работать с базой передавая свои запросы прямо через командную строку.

Для помощи используйте команды:

`\?` - Показывает справочную информацию

`\h` или `\help [ команда ]` - Выводит подсказку по синтаксису указанной команды SQL

Полный список команд можно найти в официальной [документации](#).

> Установка на Linux (Ubuntu):

Для установки клиента нужно выполнить команду:

```
1 sudo apt-get install -y postgresql-client
```

Подключение можно произвести аналогично инструкции для Linux (CentOS).

> Установка на MacOS:

Необходимо открыть терминал и выполнить команду

```
1 brew install libpq
```

По умолчанию rsq1 не добавляется в PATH, поэтому можно добавить путь до бинарника в PATH вручную, или создать symlink

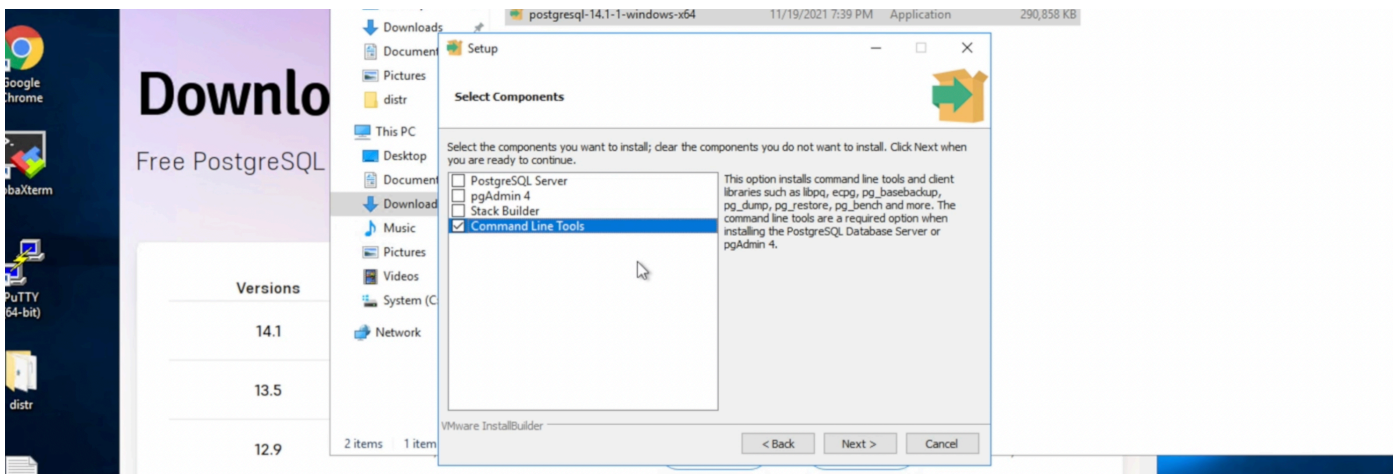
```
1 brew link --force libpq
```

Подключение можно произвести аналогично инструкции для Linux (CentOS).

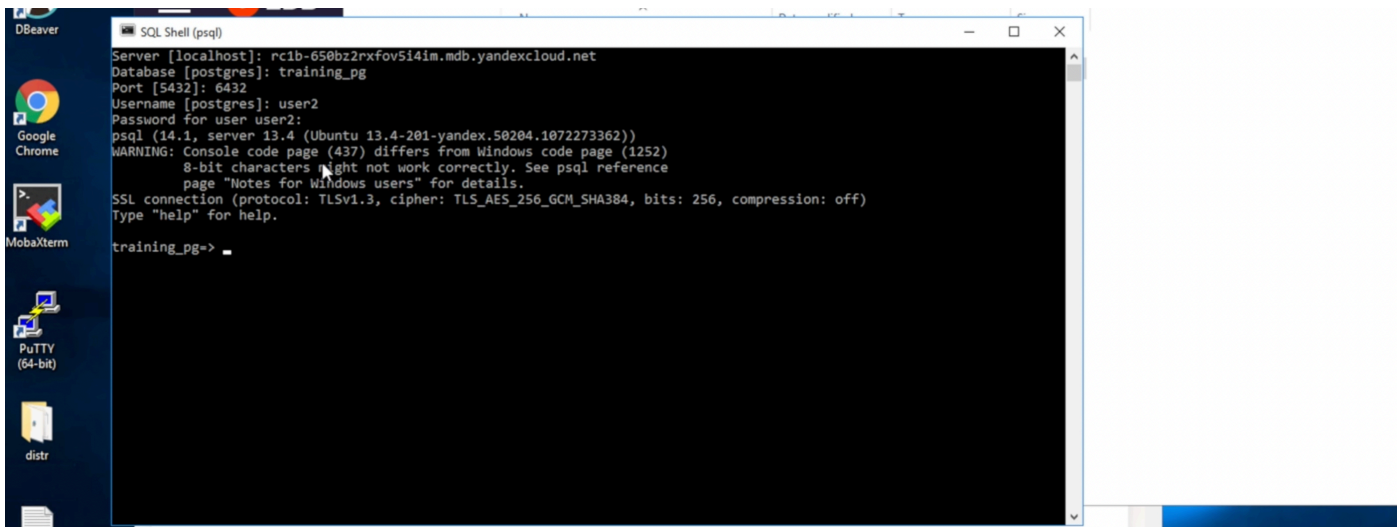
> Установка на Windows:

Сначала необходимо скачать дистрибутив PostgreSQL для Windows. [Ссылка](#).

Устанавливаем его. Если нам необходимо только клиент, выбираем "Command Line Tools".



После завершения установки находим SQL Shell и запускаем его. Далее необходимо будет ввести запрашиваемую информацию по вашей базе.



Пример ввода параметров в SQL Shell из лекции

Все мы подключились и теперь можем пользоваться базой.

При использовании psql можно не указывать постоянно ваш пароль при подключении. Для этого необходимо сохранить ваш пароль в переменную с помощью команды:

```
1 export PGPASSWORD=<password>
```

Ссылка на документацию [VK Cloud](#). Там вы можете найти подробные инструкции разных вариантов доступных подключений.

> DBeaver

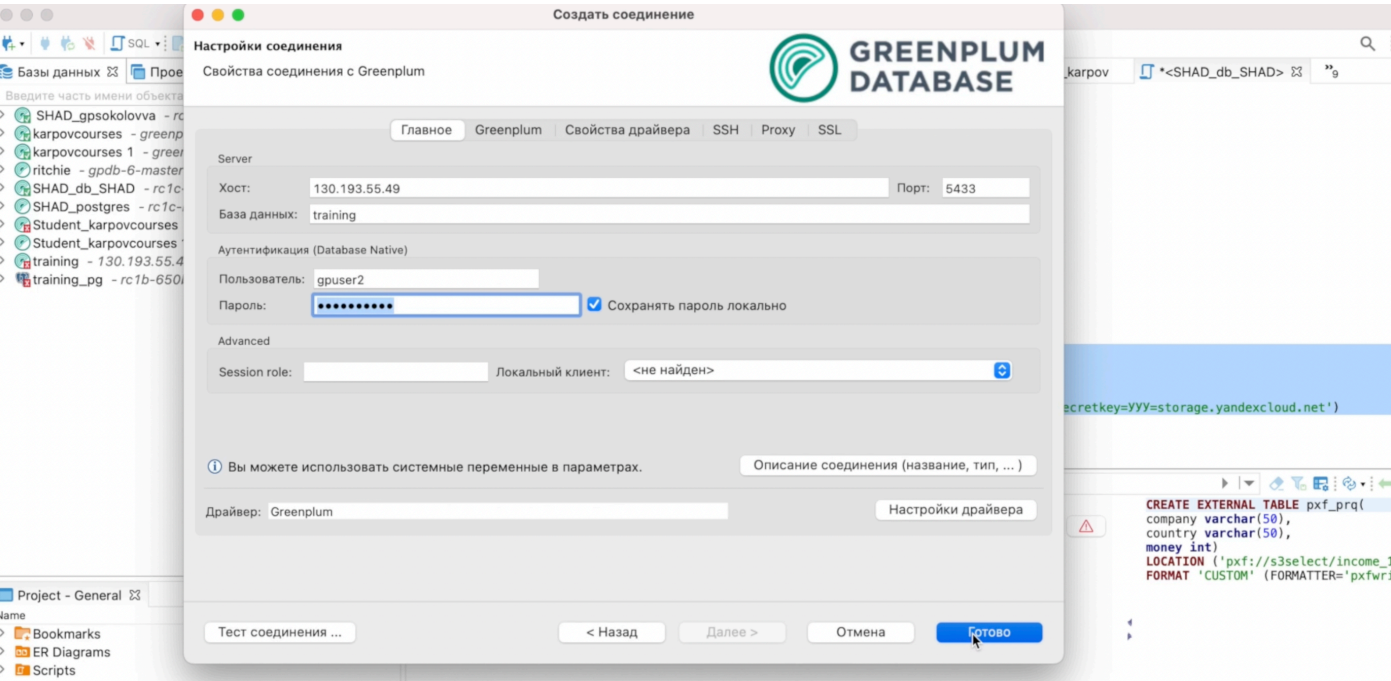
DBeaver - это клиент SQL и инструмент для администрирования баз данных.

DBeaver поддерживает все самые популярные базы данных, такие как: MySQL, PostgreSQL, MariaDB, SQLite, Oracle, DB2, SQL Server, Sybase, MS Access, Teradata, Firebird, Derby и т.д.

> Установка

Для установки DBeaver необходимо скачать дистрибутив для вашей платформы. [Ссылка](#). Подход к установке общий, не зависит от типа платформ.

После установки открываем DBeaver и выбираем в левом верхнем углу "Создать новое соединение". В контекстном меню выбираем необходимую нам базу для подключения, например GreenPlum, и ждем далее. Теперь необходимо ввести параметры вашего соединения. При необходимости DBeaver может попросить установить драйвера, их нужно будет установить.



Пример ввода параметров соединения из лекции

После вы можете начать работать с подключенными базами. DBeaver обладает широким функционалом. Позволяет через свой интерфейс видеть структуры баз данных, удобно работать с таблицами, подсказывает синтаксис и это лишь малая часть всех возможностей.

> Системный каталог

Системный каталог — это место, где система управления реляционной базой данных хранит метаданные схемы, в частности информацию о таблицах и столбцах, а также служебные сведения. Системные каталоги представляют собой обычные таблицы. Поэтому вы можете удалить и пересоздать их, добавить столбцы, изменить и добавить строки, т. е. разными способами вмешаться в работу системы.

В таблице `pg_class` хранится информация обо всех объектах. Для просмотра содержимого нужно просто заселектировать все из таблицы:

```
1 SELECT *
```

```
2 FROM pg_catalog.pg_class
```

В таблице `pg_namespace` содержится информация обо всех схемах.

Например, попробуем достать информацию о таблицах из конкретной схемы:

```
▼  
1 SELECT *  
2 FROM pg_class pc  
3 JOIN pg_namespace pn  
4 ON pn.oid=pc.relnamespace  
5 WHERE nspname=<Название схемы>  
6 ORDER BY relname
```

В результатах можно увидеть индексы таблиц. В поле `relstorage` показано как хранятся наши таблицы:

- h - обычная таблица
- x - внешняя таблица
- c - оптимизированная для добавления таблица

Полный список доступных каталогов можно посмотреть в [документации](#).

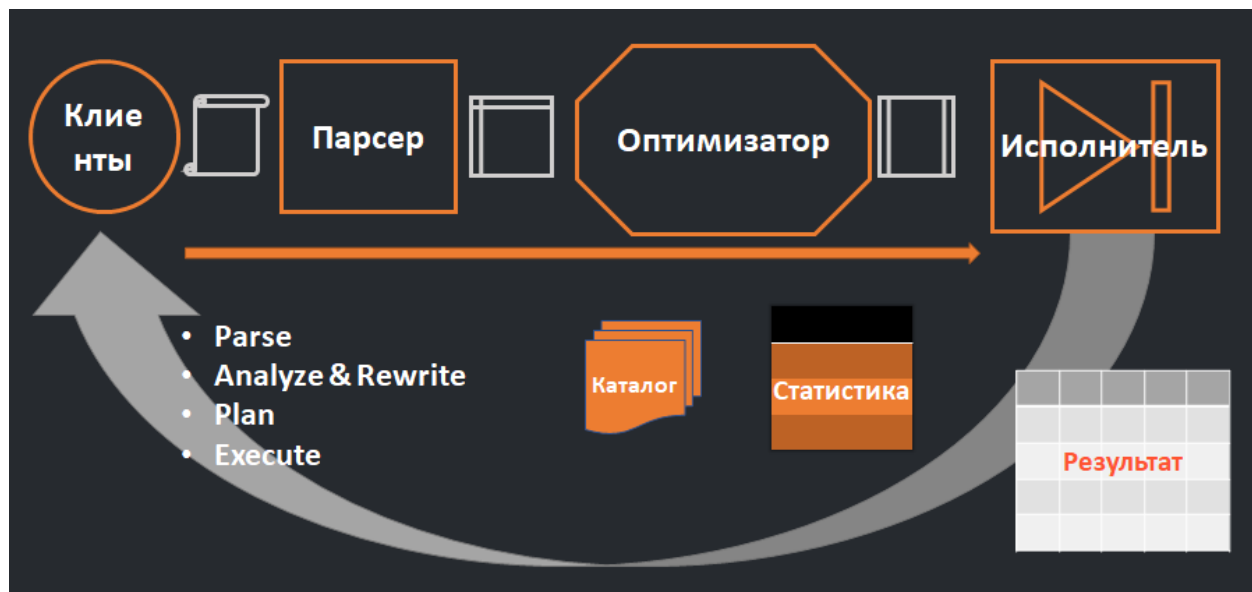


> Конспект > 3 урок > Обработка запросов в обычной СУБД и в MPP СУБД

> Оглавление

- > [Оглавление](#)
- > [Жизненный цикл запроса](#)
- > [Особенности обработки запроса в GreenPlum](#)
- > [Ресурсы доступные планировщику на сервере СУБД](#)
- > [Как оптимизатор принимает решения](#)
- > [Пример плана объединения двух таблиц](#)
 - [План представляет из себя дерево](#)
 - [Что присутствует в плане](#)
- > [Как выполняется план](#)
- > [Какими бывают узлы плана](#)
 - [Листовые узлы плана](#)
 - [Узлы объединения](#)
 - [Узлы плана](#)
 - [Дополнительные операторы плана GreenPlum](#)
- > [Как повлиять на план](#)

> Жизненный цикл запроса



Общая схема жизненного цикла запроса

Клиенты: приложение, аналитики, пользователи, которые пишут запрос к БД. Запрос из клиентского приложения попадает в СУБД.

Парсер: очищает запрос, проверяет синтаксическую правильность, выдаёт ошибки (если есть), преобразует запрос в вид понятный программе оптимизатора.

Оптимизатор: выполняет наиболее важные функции в обработке запроса

- Analyze – анализирует запрос,
- Optimize – оптимизирует, чтобы запрос выполнялся быстрее,
- Rewrite – запрос может быть переписан в соответствии с правилами СУБД,
- Plan – строит план выполнения запроса, опираясь на информацию из **каталога** обо всех объектах, которые участвуют в запросе (их свойствах, где и как они хранятся), и используется **статистика** (какие максимальные, средние значения содержатся в ячейках объектов запроса, насколько высокая селективность у каждой из ячеек, участвующих в запросе). План описывает, как должен выполняться запрос исполнителем, начиная с чтения данных в листьях, обработки в промежуточных узлах плана и предоставлению результата в корневом узле плана.

Исполнитель: получает план и выполняет его по шагам (считывает индексы, данные, производит сортировки, обработки, арифметические операции), собирает

результат, который направляется клиенту, как результат выполнения запроса.

> Особенности обработки запроса в GreenPlum

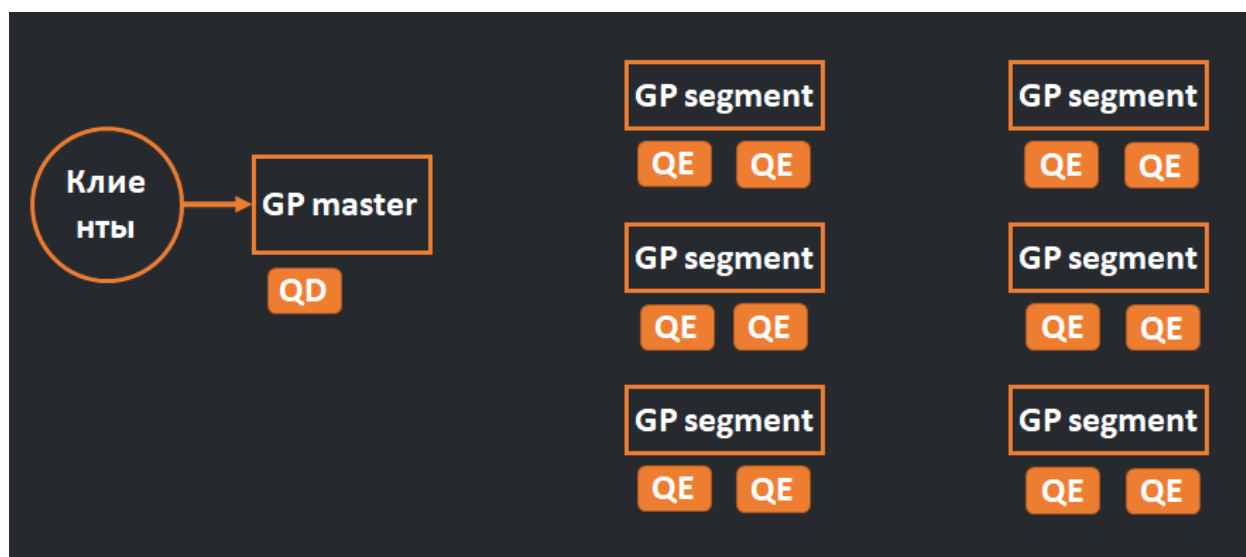


Схема обработки запросов в GreenPlum

Клиенты: формируют запрос и направляют его на GP master.

GP master: единая точка входа:

- принимает все запросы пользователей,
- выполняется парсер и оптимизатор, который подготавливает план запроса для выполнения на сегментах,
- поднимает процесс Query Dispatcher (QD), который открывает соединение к каждому из сегментов GP и передаёт план.

GP segment: сегменты GreenPlum, на которых лежат данные:

- на каждом сегменте поднимается **Query Executor (QE)**, если процесс можно распараллелить, то для ускорения выполнения запроса поднимаются несколько QE,
- QE являются исполнителями, которые выполняют все операции запроса и собирают промежуточный результат,

- может потребоваться уникальная для MPP СУБД операция **Redistribute** – сегментам необходимо обмениваться данными (необходимое по плану количество раз),
- на каждом сегменте будет готов свой финальный набор данных **Result Set (RS)**, каждый QE передаёт свой RS на master в QD.

Далее QD собирает данные из всех сегментов, производит окончательные операции (limit, фильтры) и отдаёт результат запроса клиенту.

> Ресурсы доступные планировщику на сервере СУБД



CPU – вычислительные ядра процессора (L1, L2 Cache)

RAM – оперативная память:

- Process Memory (занимают процессы),
- Shared Buffer (используется для буферов),
- Disk cache (находится недавно использованная информация).

Storage – хранилище, где лежат данные (физические носители)

Network – сеть для GreenPlum, т.к. он содержит несколько сегментов.

> Как оптимизатор принимает решения

Cost Based Optimizer – оптимизаторы принимают решения о том какой выбрать план исходя из стоимости плана.

Стоимость складывается из:

- данных в **словаре данных**: размер объекта, тип хранения (поколоночный, сжатый для GreenPlum), есть ли индексы,
- данных **статистики**: селективность полей (насколько одинаковые или разные значения содержатся в одном поле), гистограмма значений объекта (чтобы оптимизатор понимал насколько часто встречаются те или иные объекты), оценка размеров Result Set.

Создаётся множество вариантов плана и производится расчёт стоимости в условных единицах (для PostgreSQL одна операция ввода-вывода, т.е. одну операцию чтения страницы с данными с диска).

Для каждого узла плана есть базовая стоимость, есть коэффициенты, которые влияют на стоимость.

Выбирается самый дешёвый план и передаётся исполнителю.

> Пример плана объединения двух таблиц

	QUERY PLAN
1	Limit (cost=0.00..970.99 rows=2 width=224)
2	-> Gather Motion 64:1 (slice1; segments: 64) (cost=0.00..970.97 rows=100 width=224)
3	Merge Key: lineitem.l_quantity
4	-> Limit (cost=0.00..970.92 rows=2 width=224)
5	-> Sort (cost=0.00..970.92 rows=5981 width=224)
6	Sort Key: lineitem.l_quantity
7	-> Hash Join (cost=0.00..875.62 rows=5981 width=224)
8	Hash Cond: (lineitem.l_orderkey = orders.o_orderkey)
9	-> Seq Scan on lineitem (cost=0.00..433.70 rows=7513 width=117)
10	Filter: (l_quantity > 10::numeric)
11	-> Hash (cost=431.56..431.56 rows=1515 width=107)
12	-> Seq Scan on orders (cost=0.00..431.56 rows=1515 width=107)
13	Filter: (o_totalprice > 100000::numeric)

План запроса на объединение 2х таблиц, сортировку значений, взятие первых 2 строк

План представляет из себя дерево

- **корневой уровень** – результат возвращается клиенту,
- **ветви** – выполняют операции над данными,
- **листья** – нижний уровень, начало работы с данными, осуществляется физическое чтение данных.

Что присутствует в плане

- **cost** – стоимость операции,
- **rows** – количество строк,
- **width** – примерная ширина возвращаемой строки,
- **Seq Scan** – непосредственный доступ к данным в листах,
- **Hash** – операции расчёта хэша,
- **Hash Join** – операции объединения,
- **Sort, Limit** и др. – операции запроса, которые влияют на стоимость, количество строк, столбцов,
- **Gather Motion** – специфичная для GreenPlum операция, означает что все данные полученные с сегментов собираются на мастере и мастер отдаёт клиенту результат.

План показывает самый дорогой оператор, где наибольшее количество строк, где торможение, где можно что-то подвинуть, поменять оператор, либо ускорить, что-либо сделать с данными.

> Как выполняется план

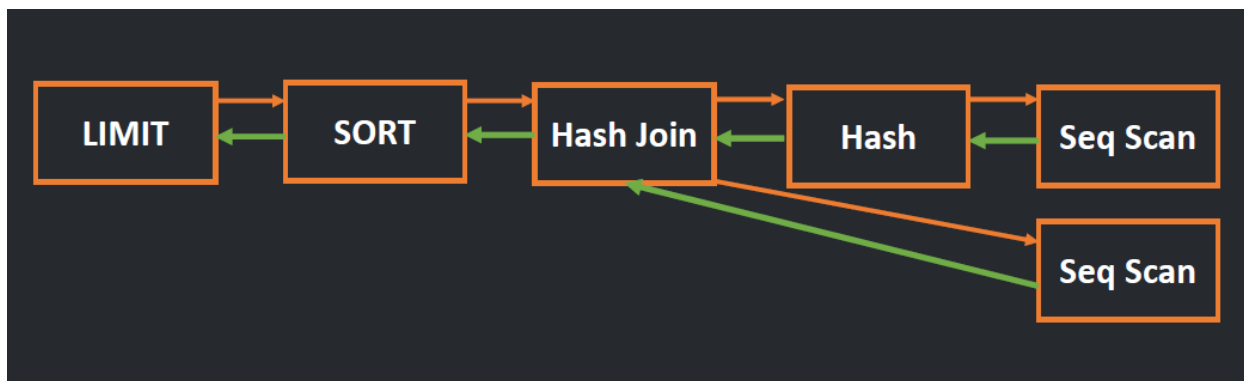


Схема выполнения плана

Последовательность выполнения плана:

1. Исполнитель просматривает план от корневого узла и добирается в листовые узлы,
2. В листовых узлах начинается выполнение плана, осуществляется доступ к данным, происходит последовательное сканирование таблицы или сканирование индекса, получает исходный набор данных,
3. Набор данных отправляется в ближайший узел плана запроса в сторону корня,
4. На каждом узле происходит соответствующая плану обработка данных, результаты отправляются на следующий узел в сторону корня,
5. Исполнитель агрегирует результаты работы по плану во время обработки на корневом узле,
6. Результат предоставляется клиенту.

> Какими бывают узлы плана

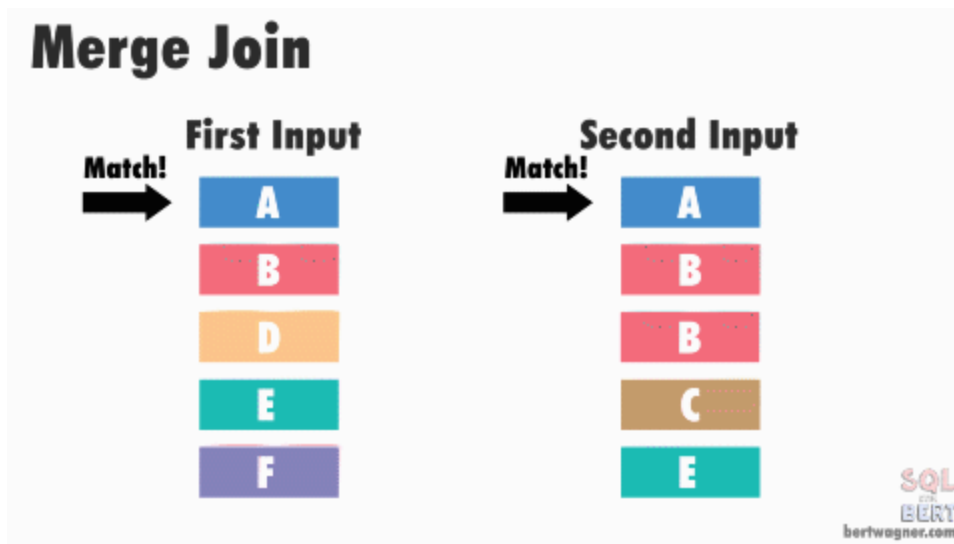
Листовые узлы плана

- **Sequential Scan** – последовательное чтение файла данных с диска,
- **Index Scan** – чтение индекса, чтобы получить ссылки на страницы с данными таблицы и далее чтение таблицы,
- **Index Only Scan** – в индексе содержатся не только поля, но и требуемые данные, нет необходимости читать таблицу,

- **Bitmap Scan** – построение битовой карты при чтении таблицы и выбор нужных результатов подходящих по битовой карте.

Узлы объединения

- **Nested Loop Join** – объединяются два Result Set (один внешний, другой внутренний), сравнивается каждый элемент внутреннего с каждым элементов внешнего, все элементы, для которых выполняется условие соединения, попадают в итоговый Result Set,
- **Hash Join** – для объединения двух объектов строится hash по ключу объединения на одном из объектов и в процессе объединения по полю объединения вычисляется hash второго объекта и происходит сопоставление элементов объектов по hash ключам объединения. Можно создать только отношение равенства,
- **Merge Join** – сравнивает первые строки с обоих отсортированных входных данных. Затем он продолжает сравнивать следующие строки со второго входа, если значения соответствуют значению первого входа.



Узлы плана

- **Sort** – выполняет сортировку Result Set,
- **Aggregate/Group by Aggregate** – выполняет агрегирование данных и выполняет расчёт агрегирующей функции,

- **Limit** – возвращает только часть Result Set.

Дополнительные операторы плана GreenPlum

- **Redistribute Motion** – перераспределение данных между сегментами, чтобы на каждом из сегментов был доступен весь объект,
- **Gather Motion** – сбор результатов расчёта с каждого из сегментов на master и последующая выдача клиенту,
- **External Scan** – узел нижнего уровня, применяется к внешним таблицам (с внешних источников).

> Как повлиять на план

Для начала можно расследовать, почему план работает неэффективно, и понять, что можно было бы улучшить:

- **Изучить план запроса, выполнить команду explain** – посмотреть теоретический план запроса, понять что ему не хватает, что не сходится, где слишком большие цифры,
- **Выполнить команду explain analyze** – фактически выполняет запрос и позволяет понять, что происходило в СУБД в реальности с реальным временем выполнения и реальным количеством полученных строк в результате выполнения каждого из узлов,
- **Подумать**, как эффективней сохранить объект (для GreenPlum есть возможность колоночного хранения, есть возможность сжатия данных, поколоночного сжатия и т.д.).

Влияние на ускорение выполнения плана:

- **Hints** (не используются в PostgreSQL или GreenPlum, но широко используются в Oracle, Microsoft SQL) – позволяют в ручном режиме подсказывать базе, как лучше выполнить запрос,
- **Параметры оптимизатора** – ими можно управлять, например, не использовать Hash Join, Nested Loop Join, сказать оптимизатору, что Sequential Scan будет стоить дорого и лучше воспользоваться Index Scan,

- **Переписать запрос** – например, слишком много join и может возникнуть проблема с выбором оптимального варианта.



> Конспект > 4 урок > Обработка запросов в обычной СУБД и в MPP СУБД: Практика

> Оглавление

- > [Оглавление](#)
- > [Первая итерация explain и explain analyze](#)
 - [Подготовка данных](#)
 - [План выполнения запроса в Greenplum](#)
 - [План выполнения запроса в PostgreSQL](#)
- > [Вторая итерация explain и explain analyze](#)
 - [Подготовка данных](#)
 - [План запроса в Greenplum](#)
 - [Актуальность статистики](#)
 - [План запроса в PostgreSQL](#)
- > [Использование ключей распределения](#)
 - [Изменим ключ распределения](#)
- > [Использование индексов](#)
 - [Индексирование по ключу соединения](#)

[План выполнения запроса в Greenplum](#)

[План выполнения запроса в PostgreSQL](#)

> [Изменение параметров оптимизатора](#)

> [Использование внешних таблиц](#)

> [Explain с другими запросами](#)

[План запроса в Greenplum](#)

[План запроса в PostgreSQL](#)

> [Сжатие таблиц](#)

[Размер таблицы](#)

[Сжатие с построчным хранением](#)

[Сжатие с построчным хранением](#)

> Первая итерация explain и explain analyze

Подготовка данных

```
create table tpch1.orders_1 (like tpch1.orders) distributed randomly;  
create table tpch1.lineitem_1 (like tpch1.lineitem) distributed randomly;
```

Из [документации](#): «Предложение **LIKE** определяет таблицу, из которой в новую таблицу будут автоматически скопированы все имена столбцов, их типы данных и их ограничения на NULL»

Distributed randomly применяется здесь к **Greenplum** и означает, что записи будут распределены по сегментам случайно. Для PostgreSQL этого не требуется.

Вставим в таблицы по 100 записей из изначальных таблиц.

```
insert into tpch1.orders_1 select * from tpch.orders limit 100;  
insert into tpch1.lineitem_1 select * from tpch.lineitem limit 100;
```

План выполнения запроса в Greenplum

Посмотрим на план выполнения запроса в **Greenplum**, указав перед ним оператор **explain**.


```
explain
select * from tpch1.lineitem_1 l join tpch1.orders_1 o on o.o_orderkey=l.l_orderkey;
```

ABC QUERY PLAN	
1	Gather Motion 4:1 (slice3; segments: 4) (cost=0.00..862.20 rows=101 width=226)
2	-> Hash Join (cost=0.00..862.12 rows=26 width=226)
3	Hash Cond: (lineitem_1.l_orderkey = orders_1.o_orderkey)
4	-> Redistribute Motion 4:4 (slice1; segments: 4) (cost=0.00..431.02 rows=25 width=120)
5	Hash Key: lineitem_1.l_orderkey
6	-> Seq Scan on lineitem_1 (cost=0.00..431.00 rows=25 width=120)
7	-> Hash (cost=431.02..431.02 rows=25 width=106)
8	-> Redistribute Motion 4:4 (slice2; segments: 4) (cost=0.00..431.02 rows=25 width=106)
9	Hash Key: orders_1.o_orderkey
10	-> Seq Scan on orders_1 (cost=0.00..431.00 rows=25 width=106)
11	Optimizer: Pivotal Optimizer (GPORCA)

Explain в Greenplum

1. Чтобы уменьшить операции чтения, **Greenplum** ставит в начало меньшую таблицу `orders_1`

```
-> Seq Scan on orders_1 (cost=0.00..431.00 rows=25 width=106)
```

2. Затем создает хэш-таблицу с ключами по полю `o_orderkey` для дальнейшего соединения с таблицей `lineitem_1`.

```
Hash Key: orders_1.o_orderkey
```

3. Производится перераспределение всей таблицы `orders_1` между всеми сегментами **Greenplum**, чтобы на каждом из них была полная копия таблицы

```
> Redistribute Motion 4:4 (slice2; segments: 4) (cost=0.00..431.02 rows=25 width=106)
```

4. Для таблицы `lineitem_1` производится ровно то же самое

```
> Redistribute Motion 4:4 (slice1; segments: 4) (cost=0.00..431.02 rows=25 width=120)
```

```
Hash Key: lineitem_1.l_orderkey
```

```
-> Seq Scan on lineitem_1 (cost=0.00..431.00 rows=25 width=120)
```

5. Выполняется объединение и получение результата

```
> Hash Join (cost=0.00..862.12 rows=26 width 226)
```

```
Hash Cond: (lineitem_1.l_orderkey=orders_1.o_orderkey)
```

6. Оператор **Gather Motion** собирает все результаты на мастер-сегменте и отдает их клиенту

```
Gather Motion 4:1 (slice3, segments: 4) (cost=0.00..862.20 rows=101 width=226)
```

Теперь применим оператор **explain analyze** и посмотрим сколько данных могло бы вернуться клиенту, если бы запрос выполнялся (запрос отработает фактически, но клиент не получит результат).

```
explain analyze
select * from tpch1.lineitem_1 l join tpch1.orders_1 o on o.o_orderkey=l.l_orderkey;
```

ABC QUERY PLAN
Gather Motion 4:1 (slice3; segments: 4) (cost=0.00..862.20 rows=101 width=226) (actual time=3.385..3.818 rows=28 loops=1)
-> Hash Join (cost=0.00..862.12 rows=26 width=226) (actual time=2.138..2.465 rows=11 loops=1)
Hash Cond: (lineitem_1.l_orderkey = orders_1.o_orderkey)
Extra Text: (seg2) Hash chain length 1.0 avg, 1 max, using 19 of 131072 buckets.
Extra Text: (seg3) Hash chain length 1.0 avg, 1 max, using 29 of 131072 buckets.
-> Redistribute Motion 4:4 (slice1; segments: 4) (cost=0.00..431.02 rows=25 width=120) (actual time=0.003..0.010 rows=37 loops=1)
Hash Key: lineitem_1.l_orderkey
-> Seq Scan on lineitem_1 (cost=0.00..431.00 rows=25 width=120) (actual time=0.018..0.021 rows=28 loops=1)
-> Hash (cost=431.02..431.02 rows=25 width=106) (actual time=0.058..0.058 rows=29 loops=1)
-> Redistribute Motion 4:4 (slice2; segments: 4) (cost=0.00..431.02 rows=25 width=106) (actual time=0.013..0.029 rows=29 loops=1)
Hash Key: orders_1.o_orderkey
-> Seq Scan on orders_1 (cost=0.00..431.00 rows=25 width=106) (actual time=0.022..0.025 rows=29 loops=1)
Planning time: 10.107 ms
(slice0) Executor memory: 161K bytes.
(slice1) Executor memory: 60K bytes avg x 4 workers, 60K bytes max (seg0).
(slice2) Executor memory: 60K bytes avg x 4 workers, 60K bytes max (seg0).
(slice3) Executor memory: 1128K bytes avg x 4 workers, 1128K bytes max (seg0). Work_mem: 4K bytes max.
Memory used: 128000kB
Optimizer: Pivotal Optimizer (GPORCA)
Execution time: 27.881 ms

Explain analyze в Greenplum

Здесь появляется реальная информация о результатах каждого шага: затраченное время, количество строк.

Memory used: 120000kB - используемая память

Optimizer: Pivotal Optimizer (GPORCA) - используемый оптимизатор

Execution time: 27.881 ms - время выполнения всего запроса

План выполнения запроса в PostgreSQL

```

              QUERY PLAN
-----
Hash Join (cost=4.25..8.62 rows=100 width=231)
  Hash Cond: (l.l_orderkey = o.o_orderkey)
    -> Seq Scan on lineitem_1 l (cost=0.00..3.00 rows=100 width=119)
    -> Hash (cost=3.00..3.00 rows=100 width=112)
      -> Seq Scan on orders_1 o (cost=0.00..3.00 rows=100 width=112)
(5 rows)

```

Explain в PostgreSQL

Также как и в **Greenplum**, здесь обработка запроса начинается с последовательного сканирования таблицы **orders_1**. Так как у нас всего один инстанс, то ничего не распределяется по сегментам и ожидаемое количество строк - 100.

После строится хэш-таблица по ключу соединения, сканируется **lineitem_1** и таблицы объединяются по ключу.

В результате ожидаем получить 100 строк.

```

              QUERY PLAN
-----
Hash Join (cost=4.25..8.62 rows=100 width=231) (actual time=0.161..0.162 rows=0 loops=1)
  Hash Cond: (l.l_orderkey = o.o_orderkey)
    -> Seq Scan on lineitem_1 l (cost=0.00..3.00 rows=100 width=119) (actual time=0.014..0.024 rows=100 loops=1)
    -> Hash (cost=3.00..3.00 rows=100 width=112) (actual time=0.038..0.038 rows=100 loops=1)
      Buckets: 1024 Batches: 1 Memory Usage: 23kB
      -> Seq Scan on orders_1 o (cost=0.00..3.00 rows=100 width=112) (actual time=0.006..0.016 rows=100 loops=1)
Planning Time: 0.141 ms
Execution Time: 0.202 ms
(8 rows)

```

Explain analyze в PostgreSQL

Запрос выполнен гораздо быстрее, чем в **Greenplum**, так как нет объединений результатов сегментов, все хранится в одном месте. А еще **PostgreSQL** может кэшировать данные в памяти, что, вероятно, произошло при наполнении таблицы данными.

Количество строк в результате - 0. Такое вполне могло произойти, так как наполнение таблиц происходило по взятию 100 случайных строк из первоначальных таблиц и вовсе необязательно, что подтянутся одинаковые ключи с обеих таблиц.

> Вторая итерация explain и explain analyze

Подготовка данных

Удалим данные из таблиц `orders_1` и `lineitem_1` и наполним их большим количеством строк

```
delete from tpch1.orders_1;
delete from tpch1.lineitem_1;

insert into tpch1.orders_1 select * from tpch1.orders limit 100000;
insert into tpch1.lineitem_1 select * from tpch1.lineitem limit 1000000;
```

План запроса в Greenplum

ABC QUERY PLAN
Gather Motion 4:1 (slice3; segments: 4) (cost=0.00..862.20 rows=101 width=226)
-> Hash Join (cost=0.00..862.12 rows=26 width=226)
Hash Cond: (lineitem_1.l_orderkey = orders_1.o_orderkey)
-> Redistribute Motion 4:4 (slice1; segments: 4) (cost=0.00..431.02 rows=25 width=120)
Hash Key: lineitem_1.l_orderkey
-> Seq Scan on lineitem_1 (cost=0.00..431.00 rows=25 width=120)
-> Hash (cost=431.02..431.02 rows=25 width=106)
-> Redistribute Motion 4:4 (slice2; segments: 4) (cost=0.00..431.02 rows=25 width=106)
Hash Key: orders_1.o_orderkey
-> Seq Scan on orders_1 (cost=0.00..431.00 rows=25 width=106)
Optimizer: Pivotal Optimizer (GPORCA)

Explain в Greenplum

Как видно, **Greenplum** все еще думает, что на каждом сегменте находится по 25 строк из таблиц. Но мы знаем, что это уже не так. Всему виной неактуальная статистика.

ABC QUERY PLAN	
Gather Motion 4:1 (slice3; segments: 4) (cost=0.00..862.20 rows=101 width=226) (actual time=65.755..1925.098 rows=1603316)	
-> Hash Join (cost=0.00..862.12 rows=26 width=226) (actual time=63.486..1015.128 rows=404894 loops=1)	
Hash Cond: (lineitem_1.l_orderkey = orders_1.o_orderkey)	
Extra Text: (seg0) Hash chain length 1.9 avg, 7 max, using 13096 of 131072 buckets.	
-> Redistribute Motion 4:4 (slice1; segments: 4) (cost=0.00..431.02 rows=25 width=120) (actual time=0.043..481.488 rows=)	
Hash Key: lineitem_1.l_orderkey	
-> Seq Scan on lineitem_1 (cost=0.00..431.00 rows=25 width=120) (actual time=0.146..131.360 rows=500591 loops=1)	
-> Hash (cost=431.02..431.02 rows=25 width=106) (actual time=55.887..55.887 rows=25214 loops=1)	
-> Redistribute Motion 4:4 (slice2; segments: 4) (cost=0.00..431.02 rows=25 width=106) (actual time=0.062..22.407 row=)	
Hash Key: orders_1.o_orderkey	
-> Seq Scan on orders_1 (cost=0.00..431.00 rows=25 width=106) (actual time=0.622..6.437 rows=25105 loops=1)	
Planning time: 10.363 ms	
(slice0) Executor memory: 193K bytes.	
(slice1) Executor memory: 60K bytes avg x 4 workers, 60K bytes max (seg0).	
(slice2) Executor memory: 60K bytes avg x 4 workers, 60K bytes max (seg0).	

Explain analyze в Greenplum

Запрос `explain analyze` тоже отработал достаточно быстро, благодаря кэшированию данных со стороны операционной системы.

Основной порядок действий сохранился с предыдущего шага, увеличилось лишь количество строк и время обработки.

Актуальность статистики

Для того, чтобы оптимизатор понимал, с чем ему придется работать, важно поддерживать актуальную статистику. Выполним следующий код:

```
analyze tpch1.orders_1;
analyze tpch1.lineitem_1;
```

Выполним `explain` снова:

ABC QUERY PLAN
Gather Motion 4:1 (slice3; segments: 4) (cost=0.00..3254.06 rows=1966337 width=224)
-> Hash Join (cost=0.00..1780.72 rows=491585 width=224)
Hash Cond: (lineitem_1.l_orderkey = orders_1.o_orderkey)
-> Redistribute Motion 4:4 (slice1; segments: 4) (cost=0.00..763.34 rows=500000 width=117)
Hash Key: lineitem_1.l_orderkey
-> Seq Scan on lineitem_1 (cost=0.00..471.43 rows=500000 width=117)
-> Hash (cost=446.23..446.23 rows=25000 width=107)
-> Redistribute Motion 4:4 (slice2; segments: 4) (cost=0.00..446.23 rows=25000 width=107)
Hash Key: orders_1.o_orderkey
-> Seq Scan on orders_1 (cost=0.00..432.88 rows=25000 width=107)
Optimizer: Pivotal Optimizer (GPORCA)

Explain в Greenplum после обновления статистики

Получаем расчет, основанный на актуальных данных с более правильными параметрами и более высоким cost.

План запроса в PostgreSQL

```

QUERY PLAN
-----
Hash Join  (cost=5689.00..79967.00 rows=456700 width=230)
  Hash Cond: (l.l_orderkey = o.o_orderkey)
    -> Seq Scan on lineitem_1 l  (cost=0.00..29142.00 rows=1000000 width=120)
    -> Hash  (cost=2778.00..2778.00 rows=100000 width=110)
      -> Seq Scan on orders_1 o  (cost=0.00..2778.00 rows=100000 width=110)
(5 rows)

```

Explain в PostgreSQL

PostgreSQL самостоятельно обновил статистику и знает, с чем ему придется работать.

```

QUERY PLAN
-----
Hash Join  (cost=5689.00..79967.00 rows=456700 width=230) (actual time=987.374..987.380 rows=0 loops=1)
  Hash Cond: (l.l_orderkey = o.o_orderkey)
    -> Seq Scan on lineitem_1 l  (cost=0.00..29142.00 rows=1000000 width=120) (actual time=0.010..0.84741 rows=1000000 loops=1)
    -> Hash  (cost=2778.00..2778.00 rows=100000 width=110) (actual time=78.360..78.362 rows=100000 loops=1)
      Buckets: 32768  Batches: 4  Memory Usage: 3764kB
      -> Seq Scan on orders_1 o  (cost=0.00..2778.00 rows=100000 width=110) (actual time=0.004..0.8369 rows=100000 loops=1)
Planning Time: 0.137 ms
Execution Time: 987.962 ms
(8 rows)

```

Explain analyze в PostgreSQL

Также, как и с меньшими версиями таблиц, здесь ничего не удалось объединить. Результат вывода - 0 строк.

> Использование ключей распределения

Изменим ключ распределения

Следующий код перераспределит наши таблицы по ключу соединения:

```
alter table tpch1.lineitem_1 set with (reorganize=True) distributed by (l_orderkey);
alter table tpch1.orders_1 set with (reorganize=True) distributed by (o_orderkey);
```

QUERY PLAN

```
Gather Motion 4:1 (slice1; segments: 4) (cost=0.00..3062.58 rows=1966337 width=224)
-> Hash Join (cost=0.00..1589.25 rows=491585 width=224)
    Hash Cond: (lineitem_1.l_orderkey = orders_1.o_orderkey)
    -> Seq Scan on lineitem_1 (cost=0.00..471.43 rows=500000 width=117)
    -> Hash (cost=432.88..432.88 rows=25000 width=107)
        -> Seq Scan on orders_1 (cost=0.00..432.88 rows=25000 width=107)
Optimizer: Pivotal Optimizer (GPORCA)
(7 rows)
```

Explain в Greenplum

Сразу замечаем, что план уменьшился в размере. Нет шага с **Redistribute Motion**, потому что все объединение происходит непосредственно на сегментах. Это уменьшает расчетное время выполнения.

QUERY PLAN

```
Gather Motion 4:1 (slice1; segments: 4) (cost=0.00..3062.58 rows=1966337 width=224) (actual time=17.514..1283.314 rows=1603316 loops=1)
-> Hash Join (cost=0.00..1589.25 rows=491585 width=224) (actual time=16.874..694.860 rows=404894 loops=1)
    Hash Cond: (lineitem_1.l_orderkey = orders_1.o_orderkey)
    Extra Text: (seg0) Hash chain length 1.9 avg, 7 max, using 130% of 131072 buckets.
    -> Seq Scan on lineitem_1 (cost=0.00..471.43 rows=500000 width=117) (actual time=0.095..185.164 rows=501797 loops=1)
    -> Hash (cost=432.88..432.88 rows=25000 width=107) (actual time=15.971..15.971 rows=25214 loops=1)
        -> Seq Scan on orders_1 (cost=0.00..432.88 rows=25000 width=107) (actual time=0.035..3.449 rows=25214 loops=1)
Planning time: 17.232 ms
(slice0) Executor memory: 157K bytes.
(slice1) Executor memory: 9362K bytes avg x 4 workers, 9362K bytes max (seg0). Work_mem: 3497K bytes max.
Memory used: 128000kB
Optimizer: Pivotal Optimizer (GPORCA)
Execution time: 1417.869 ms
(13 rows)
```

Explain analyze в Greenplum

Благодаря тому, что мы распределили таблицы по ключу джойна, фактическое время объединения у нас также сократилось до 1,5 секунд.

> Использование индексов

Индексирование по ключу соединения

Построим индекс по ключу соединения с помощью следующего кода:

```
create index li_ok on tpch1.lineitem_1 (l_orderkey);
create index or_ok on tpch1.orders_1 (o_orderkey);
```

План выполнения запроса в Greenplum

Проверим, будет ли теперь оптимизатор использовать индексы в построении плана.

```
QUERY PLAN
-----
Gather Motion 4:1  (slice1; segments: 4) (cost=0.00..3062.58 rows=1966337 width=224)
-> Hash Join  (cost=0.00..1589.25 rows=491585 width=224)
    Hash Cond: (lineitem_1.l_orderkey = orders_1.o_orderkey)
    -> Seq Scan on lineitem_1  (cost=0.00..471.43 rows=500000 width=117)
    -> Hash  (cost=432.88..432.88 rows=25000 width=107)
        -> Seq Scan on orders_1  (cost=0.00..432.88 rows=25000 width=107)
Optimizer: Pivotal Optimizer (GPORCA)
(7 rows)
```

Explain в Greenplum

Ожидаемо, оптимизатор решил не пользоваться индексами, поскольку **Greenplum** рассчитывает, что на каждом сегменте находится лишь небольшая порция данных и быстрее будет ее считать напрямую в память и работать уже с ней, чем сначала читать файл с индексом, а затем по ссылкам из файла индексов читать данные из таблицы. В 90% случаев читать придется все и выигрыша никакого не получится. Поэтому **Greenplum** идет сразу в файл с данными.

План выполнения запроса в PostgreSQL

```
QUERY PLAN
-----
Merge Join  (cost=0.72..46599.72 rows=456700 width=230)
  Merge Cond: (o.o_orderkey = l.l_orderkey)
    -> Index Scan using or_ok on orders_1 o  (cost=0.29..3554.29 rows=100000 width=110)
    -> Index Scan using li_ok on lineitem_1 l  (cost=0.42..35728.43 rows=1000000 width=120)
(4 rows)
```

Explain в PostgreSQL

А вот PostgreSQL уже решает использовать индексы при выполнении запроса. Таким образом, он понимает, какие строки можно сразу объединять, не сканируя весь файл данных

> Изменение параметров оптимизатора

Так как Greenplum в данный момент используют оптимизатор Pivotal Optimizer, который не воспринимает подсказки, отключим его.

```
set optimizer off;
```

Посмотрим на стоимость разных типов чтения страниц:

```
show seq_page_cost;  
show random_page_cost;
```

Выдача означает, что для оптимизатора дешевле будет использовать последовательное чтение страниц.

Попробуем перевернуть ситуацию и установить для выборочного чтения стоимость = 1 и для последовательного = 100.

```
set seq_page_cost=100;  
set random_page_cost=1;
```

Посмотрим теперь на план запроса:

```
QUERY PLAN  
-----  
Gather Motion 4:1 (slice1; segments: 4) (cost=5318.31..144577.28 rows=1966205 width=224)  
-> Hash Join (cost=5318.31..144577.28 rows=491552 width=224)  
    Hash Cond: (l.l_orderkey = o.o_orderkey)  
    -> Index Scan using li_ok on lineitem_1 l (cost=0.18..97930.43 rows=500000 width=117)  
    -> Hash (cost=4068.14..4068.14 rows=25000 width=107)  
        -> Index Scan using or_ok on orders_1 o (cost=0.17..4068.14 rows=25000 width=107)  
Optimizer: Postgres query optimizer  
(7 rows)
```

Explain в Greenplum с измененными параметрами оптимизатора

Видим, что оптимизатор начал использовать индексы для чтения данных.

Теперь вернём значения обратно и убедимся, что оптимизатор снова стал использовать последовательное чтение.

```
QUERY PLAN
-----
Gather Motion 4:1 (slice1; segments: 4) (cost=2685.00..73309.71 rows=1966205 width=224)
-> Hash Join (cost=2685.00..73309.71 rows=491552 width=224)
    Hash Cond: (l.l_orderkey = o.o_orderkey)
    -> Seq Scan on lineitem_1 l (cost=0.00..29296.00 rows=500000 width=117)
    -> Hash (cost=1435.00..1435.00 rows=25000 width=107)
        -> Seq Scan on orders_1 o (cost=0.00..1435.00 rows=25000 width=107)
Optimizer: Postgres query optimizer
(7 rows)
```

Explain в Greenplum с параметрами по умолчанию

> Использование внешних таблиц

В **Greenplum** можно использовать внешние таблицы для работы. Давайте попробуем сделать объединение по таким таблицам.

`tpch1.orders_ext` и `tpch1.lineitem_ext` представляют собой обращение к внешней программе, которая создаёт данные и представляет их в виде таблицы. В данном случае, оптимизатор не знает статистики этих таблиц и все значения выдаёт навскидку.

```
QUERY PLAN
-----
Gather Motion 4:1 (slice3; segments: 4) (cost=55219.00..135125342.00 rows=1000000000 width=740)
-> Hash Join (cost=55219.00..135125342.00 rows=250000000 width=740)
    Hash Cond: (l.l_orderkey = o.o_orderkey)
    -> Redistribute Motion 4:4 (slice1; segments: 4) (cost=0.00..31000.00 rows=250000 width=382)
        Hash Key: l.l_orderkey
        -> External Scan on lineitem_ext l (cost=0.00..11000.00 rows=250000 width=382)
    -> Hash (cost=31000.00..31000.00 rows=250000 width=358)
        -> Redistribute Motion 4:4 (slice2; segments: 4) (cost=0.00..31000.00 rows=250000 width=358)
            Hash Key: o.o_orderkey
            -> External Scan on orders_ext o (cost=0.00..11000.00 rows=250000 width=358)
Optimizer: Postgres query optimizer
(11 rows)
```

Explain при джойне внешних таблиц

> Explain с другими запросами

Посмотрим, как будет вести себя оптимизатор, когда получит более развернутый запрос, где будут встречаться агрегирующие функции, расчеты и сортировка.

```
select
    l_returnflag,
```

```

    l_linestatus,
    sum(l_extendedprice) as sum_base_price,
    sum(l_extendedprice * (1 - l_discount)) as sum_disc_price,
    sum(l_extendedprice * (1 - l_discount) * (1 + l_tax)) as sum_charge,
    avg(l_quantity) as avg_cty,
    avg(l_extendedprice) as avg_price,
    avg(l_discount) as avg_disc,
    count(*) as count_order
from
    tpch1.lineitem_1
where
    l_shipdate <= date '1998-12-01' - interval '100 days'
group by
    l_returnflag,
    l_linestatus
order by
    l_returnflag,
    l_linestatus;

```

План запроса в Greenplum

```

-----
QUERY PLAN
-----
Gather Motion 4:1 (slice2; segments: 4) (cost=112880.02..112880.04 rows=6 width=248)
  Merge Key: lineitem_1.l_returnflag, lineitem_1.l_linestatus
    -> Sort (cost=112880.02..112880.04 rows=2 width=248)
      Sort Key: lineitem_1.l_returnflag, lineitem_1.l_linestatus
      -> HashAggregate (cost=112879.78..112879.94 rows=2 width=248)
        Group Key: lineitem_1.l_returnflag, lineitem_1.l_linestatus
        -> Redistribute Motion 4:4 (slice1; segments: 4) (cost=112879.36..112879.48 rows=2 width=248)
          Hash Key: lineitem_1.l_returnflag, lineitem_1.l_linestatus
          -> HashAggregate (cost=112879.36..112879.36 rows=2 width=248)
            Group Key: lineitem_1.l_returnflag, lineitem_1.l_linestatus
            -> Seq Scan on lineitem_1 (cost=0.00..34296.00 rows=491146 width=25)
              Filter: (l_shipdate <= '1998-08-23 00:00:00'::timestamp without time zone)
Optimizer: Postgres query optimizer
(13 rows)

```

Explain в Greenplum

Несмотря на то, что таблица распределена по полю `l_orderkey` и это удобно при джойне с использованием данного ключа, здесь все равно есть шаг с **Redistribute Motion**

План запроса в PostgreSQL

```

QUERY PLAN
-----
Finalize GroupAggregate (cost=41725.04..41727.11 rows=6 width=236)
  Group Key: l_returnflag, l_linestatus
  -> Gather Merge (cost=41725.04..41726.44 rows=12 width=236)
      Workers Planned: 2
      -> Sort (cost=40725.02..40725.03 rows=6 width=236)
          Sort Key: l_returnflag, l_linestatus
          -> Partial HashAggregate (cost=40724.77..40724.94 rows=6 width=236)
              Group Key: l_returnflag, l_linestatus
              -> Parallel Seq Scan on lineitem_1 (cost=0.00..24350.33 rows=409361 width=25)
                  Filter: (l_shipdate <= '1998-08-23 00:00:00'::timestamp without time zone)
(10 rows)

```

Explain в PostgreSQL

В PostgreSQL план очень похож на тот, что выдает оптимизатор Greenplum, но здесь уже нет Redistribute Motion и Gather Motion.

> Сжатие таблиц

Размер таблицы

Посмотрим на размер таблицы `tpch1.lineitem`

```
select pg_size_pretty(pg_total_relation_size('tpch1.lineitem'));
```

Применив запрос выше, получаем размер таблицы **34 GB**.

Greenplum умеет сжимать таблицы. У него есть разные алгоритмы и уровни сжатия.

Сжатие с построчным хранением

Создадим таблицу `tpch1.lineitem_compressed`, которая будет сжиматься алгоритмом сжатия **zstd** с уровнем сжатия 7. В ней обычное построчное хранение, но блоки данных сжаты. Также посмотрим на размер этой таблицы.

```

create table tpch1.lineitem_compressed (like tpch1.lineitem) with
  (appendoptimized=true, compressedtype=zstd, compresslevel=7, orientation=row)
distributed randomly;

insert into tpch1.lineitem_compressed select * from tpch1.lineitem;

select pg_size_pretty(pg_total_relation_size('tpch1.lineitem_compressed'));

```

`tpch1.lineitem_compressed` получилась почти в 3,5 раза меньше, и занимает всего 11 GB.

При обращении к такой таблице, у нас будет меньше операций ввода-вывода, а они самые дорогие для Greenplum. Мы быстро читаем блоки данных, поднимаем их в память, процессор их разжимает и может использовать. Для такой таблицы может быть выгодно использовать индексы, потому что в случае дискретной выборки это позволит поднимать меньше блоков данных и меньше разжимать.

Сжатие с поколоночным хранением

Создадим еще одну таблицу `tpch1.lineitem_compressed_columnar`, которая будет сжиматься тем же алгоритмом сжатия `zstd` с уровнем сжатия 7. Это будет сжатая таблица с поколоночным хранением. Каждая из колонок на диске представляет собой отдельный файл, который также сжимается. Посмотрим сколько места будет занимать такая таблица.

```
create table tpch1.lineitem_compressed_columnar (like tpch1.lineitem) with
  (appendoptimized=true, compressedtype=zstd, compresslevel=7, orientation=column)
distributed randomly;

insert into tpch1.lineitem_compressed_columnar select * from tpch1.lineitem;

select pg_size_pretty(pg_total_relation_size('tpch1.lineitem_compressed_columnar'));
```

Она занимает всего **7,5 GB**, что почти в 5 раз меньше, чем исходная несжатая таблица.



> Конспект > 5 урок > Применение R, Python, Geospatial в расчетах на GreenPlum

> Аналитическая экосистема GreenPlum

Для расширения возможностей GreenPlum, к нему можно подключить различные модули.

Аналитическая экосистема GreenPlum состоит из:

- Analytical Functions - аналитические функции
- Procedural Languages - процедурные языки
- Data Transformation - преобразования данных
- Machine & Deep Learning - машинное обучение
- Graph - работа с графами
- Time Series - работа с временными сериями
- Geospatial - работа с географией
- Text processing - обработка текстов

Далее рассмотрим каждый из этих пунктов подробнее.

> Аналитические функции

Агрегатные функции

Позволяют за одно чтение данных с диска произвести всевозможные вычисления и выдать нужный результат, что сильно ускоряет время действия запроса.

- COUNT - подсчитывает количество входных строк
- SUM - сумма всех непустых входных значений
- MIN/MAX - минимум/максимум среди всех непустых входных значений
- AVG - арифметическое среднее всех непустых входных значений

Расширенный набор агрегатных функций:

- MEDIAN - вычисляет медиану среди всех непустых входных значений
- PERCENTILE_COUNT/PERCENTILE_DISC - вычисляет процентиль для непрерывного/дискретного распределения
- SUM(array[]) - позволяет производить суммирование списков (суммирование ведется по соответствующим индексам)

```
1 WITH matrix AS (  
2     SELECT ARRAY[[1, 2], [3, 4]] AS value  
3     UNION ALL  
4     SELECT ARRAY[[0, 1], [1, 0]] AS value  
5 )  
6 SELECT  
7     SUM(value)  
8 FROM matrix  
9 ;  
10  
11 -----+  
12 sum      |  
13 -----+  
14 {{1, 3}, {4, 4}} |
```

- UNNEST(array[]) - позволяет из одной строчки со списком создать несколько строк, в каждой из которой будет запись из переданного списка

Оконные функции

Позволяют выполнять вычисления над набором строк таблицы, который некоторым образом соотносится с текущей строкой.

Синтаксис определения рамки в упрощенном виде задается как:

`OVER ([PARTITION BY ... ], [ORDER BY ... ])`, где указание секции PARTITION BY группирует строки исходной таблицы в разделы, а порядок строк в разделе задается секцией ORDER BY.

- FIRST_VALUE/LAST_VALUE - позволяют в рамках окна найти соответственно первое и последнее значение
- LAG/LEAD - позволяют работать со строками, которые находятся на некотором расстоянии от текущей позиции в рассматриваемом окне
- CUME_DIST - позволяет создать функцию распределения

```
1 WITH table AS (  
2     SELECT 1 AS id, 10 AS val  
3     UNION ALL  
4     SELECT 2 AS id, 20 AS val  
5     UNION ALL  
6     SELECT 3 AS id, 30 AS val  
7     UNION ALL  
8     SELECT 4 AS id, 50 AS val  
9     UNION ALL  
10    SELECT 5 AS id, 70 AS val  
11 )  
12 SELECT  
13     id AS "ID",  
14     val AS "VAL",  
15     CUME_DIST() OVER (ORDER BY val) AS "Cume_Dist"  
16 FROM table  
17 ORDER BY val  
18 ;  
19  
20 -----+  
21 ID|VAL|Cume_Dist|  
22 --+---+-----+  
23 1| 10|      0.2|  
24 2| 20|      0.4|  
25 3| 30|      0.6|  
26 4| 50|      0.8|  
27 5| 70|      1.0|
```

- RANK/DENSE_RANK - ранг текущей строки с пропусками / без пропусков

```
1 SELECT  
2     dep_id,
```



```

3      last_name,
4      salary,
5      RANK() OVER (
6          PARTITION BY dep_id
7              ORDER BY salary
8      ) AS "RANK",
9      DENSE_RANK() OVER (
10         PARTITION BY dep_id
11             ORDER BY salary
12     ) AS "DRANK"
13 FROM employees
14 WHERE dep_id = 60
15 ORDER BY "RANK", last_name
16 ;
17
18 -----+-----+-----+-----+
19 dep_id |last_name|salary|RANK|DRANK|
20 -----+-----+-----+-----+
21      60|Lorentz  | 4200|  1|  1|
22      60|Austin   | 4800|  2|  2|
23      60|Pataballa| 4800|  2|  2| -- RANK=DRANK из-за равенства зарплат
24      60|Ernst    | 6000|  4|  3| -- RANK пропускает 3, а DENSE_RANK нет
25      60|Hunold   | 9000|  5|  4| -- RANK учитывает общее кол-во элементов

```

- NTILE - разделяет упорядоченный набор данных на несколько групп (бакетов) и определяет, в какую группу попадет каждая строка

```

1 SELECT
2     last_name,
3     salary,
4     NTILE(4) OVER (
5         ORDER BY salary DESC
6     ) AS quartile
7 FROM employees
8 WHERE department_id = 100
9 ORDER BY last_name, salary, quartile
10 ;
11
12 -----+-----+-----+
13 last_name|salary|quartile|
14 -----+-----+-----+
15 Chen      | 8200|      2|
16 Faviet    | 9000|      1|
17 Greenberg| 12000|     1|
18 Popp      | 6900|      4|
19 Sciarra   | 7700|      3|

```

- ROW_NUMBER - номер текущей строки в её разделе
- PERCENT_RANK - относительный ранг текущей строки

> Процедурные языки

К GreenPlum можно подключить различные процедурные языки. По умолчанию есть PL/pgSQL.

Дополнительно можно подключить:

- PL/Python
- PL/Perl

В проприетарных версиях можно подключить:

- PL/Java
- PL/R

Исполнение каждого из процедурных языков можно контейнеризировать (PL/Container).

С точки зрения скорости PL/pgSQL самый быстрый. Остальные процедурные языки нужно использовать аккуратно, так как это вносит дополнительные траты ресурсов машин на сегментах.

Пример на PL/Python:

Рассмотрим синтаксис. Нам требуется объявить функцию, которая возвращает указанный тип. Внутри мы пишем тело функции на python.

```
▼
1 CREATE FUNCTION funcname (argument-list)
2     RETURNS return-type
3 AS $$
4     -- pl/python function-body
5 $$ LANGUAGE plpythonu;
```

Пример простой функции, которая из двух переданных чисел возвращает максимальное из них:

```
▼
1 CREATE FUNCTION pymax (a integer, b integer)
2     RETURNS integer
3 AS $$
4     if a > b:
```

```
5         return a
6     return b
7 $$ LANGUAGE plpythonu;
8
9 SELECT румax(1, 5)
```

Можно использовать как plpython 2, так и 3 (они не совместимы друг с другом). Важно помнить, что у python 2 поддержка уже закончилась, лучше использовать 3.

> Преобразование данных

В GreenPlum поддержан очень широкий набор различных функций для преобразования данных из одних форматов в другие, например из:

- JSON(b) <-> SQL array
- Xml <-> SQL array
- Python list <-> SQL array
- Создание json на лету

Поверх json (или бинарного json) можно создавать индексы, что заметно ускоряет работу с нестандартизированными документами.

В качестве примера работы с xml, можно выделить функции: `table_to_xml` / `schema_to_xml` / `database_to_xml`, которые позволяют перевести таблицу/схему/бд в xml формат.

Для работы с json можно выделить следующие операции:

Json operations

Работа с json поддерживается на уровне СУБД.

> Машинное обучение

В GreenPlum внутри sql запросов использовать методы машинного обучения. Для этого используется библиотека Apache MADlib, которая содержит:

- методы DataScience
- преобразование данных
- описательная и индуктивная статистика
- обучение "с учителем" или "на примерах"

- глубокое обучение с Keras & TensorFlow
- модули для работы с графами (необходим специальный формат таблиц, в которых мы храним вершины и ребра. Есть возможность поиска кратчайшего пути, поиска связных компонент и. т. п.)
- решение систем линейных уравнений
- функции обработки текстов
- анализ временных рядов ARIMA

Подробнее про работу с временными рядами:

Обработка и хранение данных временных рядов достаточно удобно, так как мы можем использовать:

- append-optimized таблицы
- колоночное хранение (будут считываться только нужные для анализа колонки)
- партиционирование (по дате и времени события)
- сжатие
- параллельная обработка на разных сегментах

Пример с построением линейной регрессии

Создадим таблицу, в которой есть идентификаторы, значения того, что нужно предсказать и x1, x2.

```
1 CREATE TABLE regr_example (  
2     id int,  
3     y int,  
4     x1 int,  
5     x2 int  
6 );  
7 INSERT INTO regr_example VALUES  
8     (1, 5, 2, 3),  
9     (2, 10, 7, 2),  
10    (3, 6, 4, 1),  
11    (4, 8, 3, 4);
```

Потом применяем к ней функцию `linregr_train` из библиотеки MADlib:

```
1 SELECT madlib.linregr_train (  
2     'regr_example',      -- source table
```

```

3 'regr_example_model',    -- output model table
4 'y',                    -- dependent variable
5 'ARRAY[1, x1, x2]'      -- independent variables
6 );

```

В результате выводится краткий отчет с основными коэффициентами регрессии, необходимыми для анализа полученной модели:

```

1 SELECT * FROM regr_example_model;
2
3 -[ RECORD 1 ]-----+-----
4 coef           |
   {0.111111111111127,1.14814814814815,1.01851851851852} |
5 r2             | 0.968612680477111
6 std_err        |
   {1.49587911309236,0.207043331249903,0.346449758034495} |
7 t_stats        |
   {0.0742781352708591,5.54544858420156,2.93987366103776} |
8 p_values        |
   {0.952799748147436,0.113579771006374,0.208730790695278} |
9 condition_no    | 22.650203241881
10 num_rows_processed | 4
11 num_missing_rows_skipped | 0
12 variance_covariance |
   {{2.23765432098598,-0.257201646090342,-0.437242798353582}, |
13    {-0.257201646090342,0.042866941015057,0.0342935528120456}, |
14    {-0.437242798353582,0.0342935528120457,0.12002743484216}} |

```

С помощью полученной модели можно строить предсказания. Для этого нужно использовать функцию `linregr_predict`

```

1 SELECT
2   regr_example.*,
3   madlib.linregr_predict ( ARRAY[1, x1, x2], m.coef ) AS predict,
4   y - madlib.linregr_predict ( ARRAY[1, x1, x2], m.coef ) AS residual
5 FROM regr_example, regr_example_model m;
6

```


Функционал:

- Index & Search в Solr - индексы могут храниться как в GreenPlum, так и кластере Apache Solr
- данные как в GreenPlum так и вне
- faceting, NER (Named entity recognition) , POS (Part-of-speech)
- proprietary
- большое количество функции, их больше, чем в GreenPlum DataBase Text Search, но это проприетарно

> Работа с географией

Для работы с геоданными в GreenPlum удобно использовать библиотеку PostGIS.

Она включается в себя:

- геометрические типы
- вычислительные и аналитические функции
- GIST индексы
- инструменты импорта/экспорта
- GDAL - Geospatial Data Abstraction Library
- параллельную обработку

Примеры работы с PostGIS

Для проверки того, что одна гео-сущность входит в другую можно использовать функцию `ST_Contains` .

Создадим временную табличку и добавим у ней гео-колонку типа полигон: `CREATE temporary TABLE gtest ( ID int4, NAME varchar(20) );`
`SELECT AddGeometryColumn('', 'gtest','geom',-1,'POLYGON',2);`



```
1 CREATE temporary TABLE gtest ( ID int4, NAME varchar(20) );
2 SELECT AddGeometryColumn('', 'gtest', 'geom', -1, 'POLYGON', 2);
```

Заполним эту таблицу полигоном, который является квадратом со стороной равной 10:



```

1 INSERT INTO gtest (ID, NAME, GEOM)
2 VALUES (
3     1,
4     'First Geometry',
5     ST_GeomFromText('POLYGON((0 0, 0 10, 10 10, 10 0, 0 0))', -1)
6 );
7
8 SELECT id, name, ST_AsText(geom) AS geom FROM gtest;
9 -----+-----+-----+
10 id | name          | geom
11 -----+-----+-----+
12 1 | First Geometry | POLYGON ((0 0, 0 10, 10 10, 10 0, 0 0)) |

```

Попробуем найти в получившейся таблице такую геометрию, которая включала бы в себя нужную нам линию, получим наш квадрат:

```

1 SELECT id, geom
2 FROM gtest
3 WHERE ST_Contains(geom, 'LINESTRING(2 3,4 5,6 5,7 8)');
4
5 -----+-----+
6 id|geom
7 -----+-----+
8 1|POLYGON ((0 0, 0 10, 10 10, 10 0, 0 0))|

```

Geohash

Для удобной работы с координатами можно использовать geohash. Геохэши позволяют из любой точки создать набор символов, который с некоторой точностью задает нашу точку.

Их преимущество заключается в том, что:

- можно явно задавать точность
- каждый геохэш более высокого порядка входит в геохэш более низкого порядка, поэтому легко производить операции включения/исключения
- использование геохешей экономит место, так как их хранить проще, чем координаты

Например, зададим точку и посмотрим, какой геохэш будет ей соответствовать:

```

1 SELECT ST_GeoHash(ST_SetSRID(ST_MakePoint(-126,48),4326));
2
3 -----+

```



```

4 st_geohash |
5 -----+
6 c0w3hf1s70w3hf1s70w3|

```

Здесь `ST_SetSRID` - устанавливает SRID для геометрии в определенное целочисленное значение. SRID (spatial referencing system identifier) - уникальный идентификатор, однозначно определяющий систему координат, например, используемый нами код 4326 соответствует географической системе координат WGS84.

Уменьшить длину геохеша, например, до 5. Видим, что полученное значение входит в полученный ранее более точный геохэш:

```

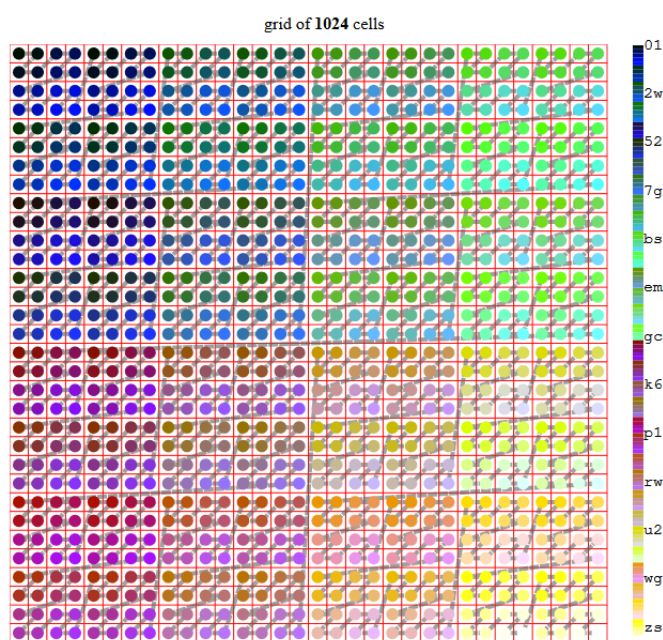
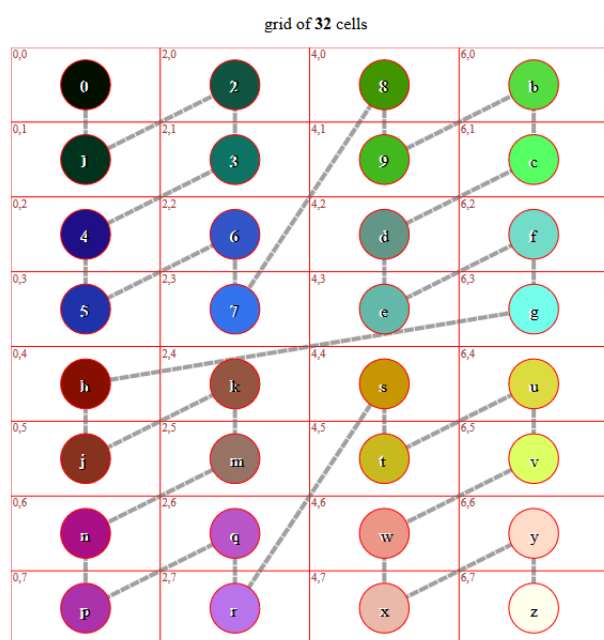
1 SELECT ST_GeoHash(ST_SetSRID(ST_MakePoint(-126,48),4326),5);
2
3 -----+
4 st_geohash|
5 -----+
6 c0w3h |

```

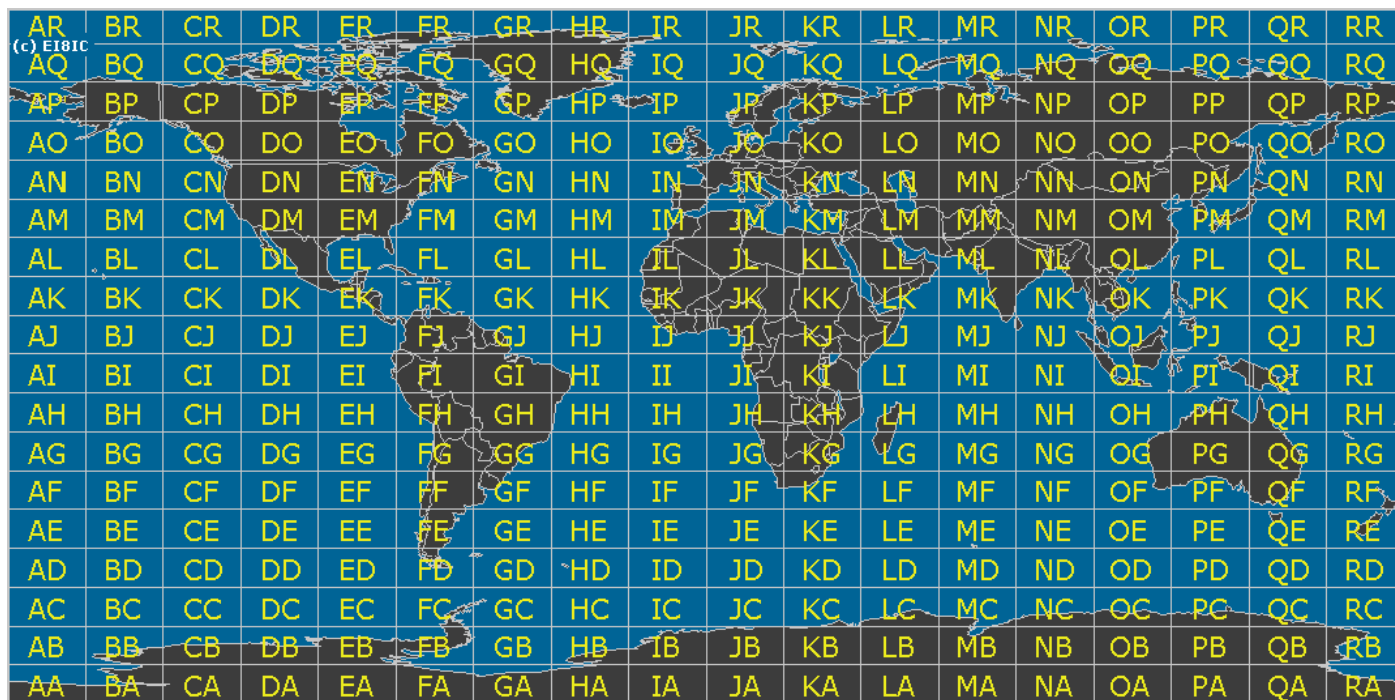
Подобными образом можно явно задавать точность геохэша. Например, геохэш в 6 символов дает погрешность ≈ 600 м.

Чтобы лучше понять, как устроены геохэши разберем алгоритм их создания.

Разобьем карту на 32 квадрата, далее нумеруем эти квадраты от 1 до z. Каждый отдельный квадрат мы повторно разбиваем на 32 блока и снова нумеруем:



Таким образом, например, можно разбить карту мира на 64 блока, тогда каждый из этих блоков будет представлять из себя geohash уровня 2:



> Дополнительные материалы

Ссылки на документации:

- [аналитические функции](#)
- [про PL/Container](#)
- [преобразование json](#)
- [преобразование xml](#)
- [MADlib](#)
- [PostGIS](#)
- [сравнение GreenPlum DataBase Text Search и GreenPlum Text](#)



> Конспект > 6 урок > Дополнительные возможности GreenPlum: Практика

> **Оглавление**

- > [Оглавление](#)
- > [Как работает PostGIS](#)
- > [Что можно сделать с Greenplum Text](#)
- > [Как применить PL\Python](#)
- > [Глоссарий](#)

> **Как работает PostGIS**

Окружение - это кластер Greenplum, собранный на одном хосте. На него установлен PostGIS версия 2.1.5. В нем есть Greenplum Text - расширение для работы с текстом. Также установлено расширение PL\Python.

Можно использовать и PL\Perl, или Java, или R

PostGIS – расширение для географических и пространственных данных, при помощи которого можно:

- обрабатывать точки, кривые, фигуры;
- получать статистику по ним;
- строить карты или находить маршруты;
- сохранять полученные данные в таблицу;
- преобразовывать географические и/или пространственные данные в друг друга;
- индексировать и обрабатывать полученные данные.

Для использования расширения можно:

- в коммерческой версии скачивается и устанавливается .package;
- или это расширение можно собрать из исходников и установить самостоятельно

Для примера создадим таблицу, где одним из типов данных будет geometry.

Полученные геометрические данные можно обрабатывать на сегментах, частями ускоряя обработку массива. Можно вставить:

- полигон (**POLYGON**)
- линию (**LINESTRING**), прямую или ломаную
- ломаную линию, из которой будет создан полигон (**MULTIPOINTS**)

```
CREATE TABLE geom_test (gid int4, geom geometry,
  name varchar(25) );

INSERT INTO geom_test ( gid, geom, name )
VALUES (1, 'POLYGON((0 0 0,0 5 0,5 5 0,5 0 0,0 0 0))', '3D Square');
INSERT INTO geom_test ( gid, geom, name )
VALUES (2, 'LINESTRING((1 1 1,5 5 5,7 7 5))', '3D Line');
INSERT INTO geom_test ( gid, geom, name )
VALUES (3, 'MULTIPOINT((3 4,8 9))', '2D Aggregate Point');
INSERT INTO geom_test ( gid, geom, name )
VALUES (1, ST_Polygon(ST_GeomFromText('LINESTRING(75.15 29.53,77 30,77.6 29.5 75.15 29.53)'), 4326, '3D Poly');
```

Можно выбрать из таблицы, какие типы данных содержатся в каком поле с каким именем

```
SELECT geometrytype (geom), NAME FROM geom_test
```

Можно выбрать все содержимое таблицы и получить наглядное представление об информации, которая хранится в геометрическом поле.

```
SELECT * FROM geom_test;
```

Еще одной возможностью DBeaver является его способность отобразить объект на карте и посмотреть, что это такое.

```
SELECT ST_Polygon (ST_GeomFromText('LINESTRING(75.15 29.53,77 30,77.6 29.5, 75.15 29.53)'), 4326);
```

Можно поставить точку для произвольных координат и посмотреть ее местонахождение на реальной карте.

Расширение PostGIS оно также работает в PostgreSQL. При необходимости анализа большого объема информации она складывается в Greenplum и обрабатывается параллельно на всех сегментах.

PostGIS поддерживает создание GiST, что позволяет ускорить обработку географических запросов и проще выполнять операции поиска-вхождения или принадлежности к одной и той же зоне разных объектов. Такой географический GiST индекс будет представлять собой дерево, в котором будут последовательно отображаться все объекты.

Геометрические поля можно добавлять к уже существующим таблицам. Для этого применяется специфический оператор PostGIS

```
AddGeometryColumn
```

```
DROP TABLE geotest;  
CREATE TABLE geotest (id INT4, name VARCHAR(32) );  
SELECT AddGeometryColumn ('geotest', 'geopoint', 4326, 'POINT', 2);
```

После создания таблицы, можно добавить к ней колонку с геометрическими данными, которые можно дополнять и смотреть их на карте.

```
INSERT INTO geotest (id, name, geopoint)  
VALUES (1, 'Olympia', ST_GeometryFromText('POINT(-122.90 46.97)', 4326));  
INSERT INTO geotest (id, name, geopoint)  
VALUES (2, 'Renton', ST_GeometryFromText('POINT(-122.90 47.50)', 4326));  
  
SELECT name, ST_AsText(geopoint) FROM geotest;  
  
SELECT geopoint FROM geotest WHERE name = 'Olympia'
```

Таким образом, расширение PostGIS позволяет манипулировать большими объемами картографических и географических данных

> Что можно сделать с Greenplum Text

- Разбиение текста на лексемы (группы ассоциированных слов);
- Построение индексов по тексту;
- Ранжирование текст по релевантности результатов поиска;

Greenplum Text доступен в open-source версии, в коммерческой версии это [VMware Tanzu](#) - он разворачивает [Apache Solr](#) кластер рядом с GreenPlum и использует его возможности по индексированию и хранению индексов текста для ускорения и облегчения поиска текста.

Основные единицы хранения и использования текста:

- документ, тип данных **tsvector**
- запрос, который представлен типом **tsquery**

Можно создать таблицу, в которой будет храниться документ и его нормализованная форма типа **tsvector**. К этому полю можно обращаться при помощи типов **tsquery**, чтобы понять вхождение поисковых слов.

GreenPlum позволяет ранжировать документы по количеству и качеству розыскиваемых лексем.

Посмотрим синтаксис запроса, с помощью которого мы пытаемся оценить вхождение необходимых нам слов в документ.

```
SELECT 'a fat cat sat on a mat and ate a fat rat'::tsvector @@ 'cat & rats'::tsquery;
```

Результатом будет true or false (правда или ложь). Так как документ не приведен к лексемам по набору правил, поэтому результат будет false. Это произошло из-за того, что слово **rat** и **rats** являются для Greenplum Text разными. Поэтому в своей работе вам необходимо сперва создать лексемы и по ним оценивать результативность вашего запроса.

Greenplum Text позволяет преобразовать текст в формат **tsvector**. Для этого необходимо вызвать функцию **to_tsvector**, которая преобразует документ в формат **tsvector**.

С помощью функции **to_tsvector** можно получать комбинированное представление документа, создавая объединенный **tsvector** из разных полей таблицы. Эти поля могут быть отранжированы по значимости тех или иных полей (на ваш выбор)

Пример запроса по имени, объединенный по типу, с рассмотрением комментариев.

```
SELECT to_tsvector('english',p_name) || to_tsvector('english',p_type) || to_tsvector('english',p_comment)
FROM tpch1.part LIMIT 50
```

Используя такое составное поле, можно искать по всему документу, найдя вхождение интересующих нас запросов

> Как применить PL\Python

Greenplum поддерживает использование следующих языков:

- PL\Perl
- Java
- R
- PL\Python
- C/C++

Можно вставлять текст, написанный на этих языках программирования и это все будет выполняться над данными, которые хранятся или будут храниться в Greenplum.

Все дополнительные языки нужно прописать как отдельные расширения.

Командой `create extension` можно создать расширение для языка PL\Python

Расширения могут быть безопасными и небезопасными. Последние выполняются в операционной системе и могут получить доступ к файлам операционной системы. Поэтому такие расширения могут создавать суперпользователи.

Можно создать функцию, которая проверяет существует ли файл в системе. Выполнение самой функции будет происходить в самой операционной системе, создавая такую функцию мы можем использовать все модули доступные Python, который установлен вместе с Greenplum.

Например, мы хотим проверить существует ли такой файл в домашней директории

```
SELECT pyfileexists ('home/gpadmin/gpinitssystem_config.txt')
```

Такого файла нет

Формируем новый запрос и проверяем наличие вот этого файла - файл на месте

```
SELECT pyfileexists ('home/gpadmin/gpinitssystem_config')
```

Для примера можно создать функцию шифровальщика, который будет кодировать предлагаемую строку, в соответствии со словарем шифрования.

```
CREATE OR REPLACE FUNCTION pyencoder(instr text,
    mapfrom text DEFAULT 'abcdedfghijklmnopqrstuvwxyz',
    map to text DEFAULT 'bc567ghijk432opqrstuvwxyz' )
RETURN text AS
$$
    import string
    strt = string.maketrans(mapfrom, mapto)
    return instr.lower().translate(strt)
$$
LANGUAGE 'plpythonu' VOLATILE
```

Создаем декодер

```
CREATE OR REPLACE FUNCTION pydecoder(instr text,
    mapfrom text DEFAULT 'abcdedfghijklmnopqrstuvwxyz',
    map to text DEFAULT 'bc567ghijk432opqrstuvwxyz' )
RETURN text AS
$$
    import string
    strt = string.maketrans(mapto, mapfrom)
    return instr.lower().translate(strt)
$$
LANGUAGE 'plpythonu' VOLATILE
```

И проверяем как работает первая часть, полученный ответ нас удовлетворяет

```
select pyencoder('Welcome to Africa');  
x735p27 up bhsj5b
```

Безопасное использование языков подразумевает использование контейнеров. Для этого необходимо создать контейнер, установить язык программирования и импортируемые модули. Такой порядок действий защищает операционную систему, так как вызываемая функция выполняется в контейнере. Такую функцию может выполнять любой пользователь.

Это подключаемый модуль PL/Container

> Глоссарий

Geometry - это плоский пространственный тип данных в евклидовом пространстве (плоской системе координат)

Лексемы — это группа ассоциированных слов или слово как абстрактная единица морфологического анализа. Сейчас в данном значении используют термин семантическое поле.